

ABSTRACT

Quadruped robots have emerged as a promising alternative to traditional wheeled systems, owing to their superior mobility and stability on uneven and unstructured terrain. This capability makes them well-suited for applications such as industrial inspection, disaster response, and healthcare assistance. This project aims to design and develop an intelligent quadruped robotic dog capable of autonomous locomotion, voice-based interaction, obstacle avoidance, and human following, while maintaining dynamic stability throughout operation.

The hardware architecture is built around two complementary processing units. An ESP32 microcontroller is employed for real-time control of actuation and low-level sensor processing, while an NVIDIA Jetson Orin Nano handles computationally intensive tasks related to AI and visual perception. Each leg is actuated using three MG996R servo motors, providing three degrees of freedom per leg, whereas two MG90S servos enable supplementary movement of the head and tail. These actuators are controlled through two PCA9685 PWM driver modules. Environmental perception is achieved using a GY-87 inertial measurement unit, a VL53L0X time-of-flight sensor, a camera module, a LiDAR sensor, and a digital microphone, which together enable obstacle detection, distance estimation, and voice input.

On the software front, gait generation is achieved using Reinforcement Learning approach, specifically Proximal Policy Optimization (PPO), which allows the robot to adapt its locomotion patterns dynamically rather than relying on pre-programmed gait sequences. Object detection and human tracking are performed using YOLOv8, while Simultaneous Localization and Mapping (SLAM) and an Extended Kalman Filter are used for environment mapping and sensor fusion, respectively. An offline voice recognition module further allows the robot to respond to spoken commands without dependence on internet connectivity.

The overall system follows a modular architecture, wherein perception, planning, control, and actuation are implemented as independent subsystems. This design approach enhances scalability, simplifies maintenance, and facilitates future upgrades. To ensure reliable operation under varying conditions, the system incorporates several safety mechanisms, including emergency stop routines, watchdog timers, battery monitoring, and thermal management.

The successful implementation of this robotic dog validates the feasibility of integrating embedded systems, artificial intelligence, reinforcement learning, and modern robotics within a single compact autonomous platform. The resulting system establishes a foundation for future enhancements, including outdoor navigation, robotic manipulation, cloud connectivity, multi-robot collaboration, and advanced autonomous decision-making.

CHAPTER 1

INTRODUCTION

1.1 Background

Robotics is a rapidly growing interdisciplinary field that continues to expand automation across industries such as manufacturing, healthcare, and logistics, improving operational efficiency and workplace safety. Within this field, legged robots have attracted particular interest for their ability to traverse environments inaccessible to wheeled systems, such as staircases and uneven terrain, by imitating animal locomotion to achieve greater adaptability.

Among legged configurations, quadruped robots offer a practical balance between agility and mechanical complexity. Their wider support polygon provides greater stability than bipedal designs while remaining mechanically simpler than multi-legged systems, making them well-suited for industrial inspection, military reconnaissance, and search-and-rescue missions.

These capabilities have been made possible by recent advances in embedded computing. Platforms such as the NVIDIA Jetson series and ARM Cortex-M microcontrollers now allow quadruped robots to run computationally demanding AI algorithms on-board, enabling real-time perception, navigation, voice interaction, object recognition, and reinforcement learning-based locomotion with minimal reliance on cloud computing. Parallel progress in computer vision, deep learning, and sensor fusion has further improved a robot's ability to perceive and respond to complex environments.

Together, these advances have transformed quadruped systems from simple, pre-programmed machines into adaptive autonomous agents capable of learning from experience, responding to voice commands, following human operators, recognizing objects, avoiding obstacles, and maintaining balance under changing terrain conditions. This evolution has accelerated research toward intelligent robotic companions for both structured environments, such as factories and hospitals, and unstructured environments, such as disaster sites.

This project focuses on developing an intelligent quadruped robotic dog that combines robust mechanical design with AI-driven autonomy capable of climbing stairs of moderate height (12–15 cm), executing voice commands offline, and performing autonomous navigation through sensor fusion and computer vision.

1.2 Evolution of Quadruped Robots

Quadruped robots have evolved from basic research gait models into highly capable autonomous systems. Commercial platforms such as Boston Dynamics' Spot, Unitree Go2, and ANYmal demonstrate advanced mobility, but their high cost and proprietary restrictions limit accessibility for academic research.

Spot, developed by Boston Dynamics, was among the most influential of these platforms, demonstrating exceptional mobility through advanced dynamic control – walking, running, climbing stairs, avoiding obstacles, and performing industrial inspection in construction sites, oil and gas facilities, and other hazardous environments. However, its high cost makes it inaccessible to most educational institutions. The Unitree Go2 offers a more affordable alternative, equipped with LiDAR, cameras, and autonomous navigation for tasks such as human-following and obstacle avoidance, while ANYmal (ETH Zurich) focuses on autonomous industrial inspection in harsh environments. The MIT Mini Cheetah further showcased lightweight, agile locomotion through running, jumping, and self-righting behaviors.

Despite their capabilities, these platforms remain expensive, proprietary, and difficult to modify, with limited access to low-level hardware and software. This restricts experimentation with new locomotion algorithms and embedded control architectures, limiting their adoption in undergraduate and smaller research settings.

This project addresses that gap by offering a cost-effective, modular research platform built on a dual-controller architecture – an ESP32 for deterministic motor control and an NVIDIA Jetson Orin Nano for AI perception. It incorporates offline voice recognition, reinforcement learning-based gait generation, real-time object detection using YOLOv8, human-following, obstacle avoidance, and sensor fusion via an Extended Kalman Filter, enabling stable locomotion across uneven terrain and autonomous staircase climbing (12–15 cm step height). Its modular design further allows future integration of robotic manipulation, autonomous charging, cloud connectivity, GPS-based outdoor navigation, and multi-robot collaboration making it a practical platform for robotics research and engineering education.

1.3 Problem Statement

Autonomous mobile robots are growing more important for applications such as industrial inspection, disaster response, healthcare, and surveillance. Yet a significant gap exists between expensive, proprietary commercial platforms and low-cost educational kits that lack autonomous features. Most affordable quadruped robots offer only basic walking ability, without intelligent perception, adaptive movement, terrain adaptation, or natural language interaction — limiting their value for AI and robotics research.

This gap is made more pressing by the demands of real-world environments. Traditional wheeled and tracked robots are largely confined to smooth, structured surfaces, with mobility dropping sharply on rough trails, gravel, grassy fields, or staircases. This restricts their use in disaster-affected areas, construction sites, and industrial facilities with elevation changes, pointing to a clear need for legged systems capable of handling varying lighting conditions and maintaining stability across complex terrain.

A further challenge lies in integrating multiple intelligent subsystems into a single platform. Robots must simultaneously process sensor data, maintain balance, detect obstacles, recognize voice commands, and make real-time navigation decisions — tasks that require an architecture capable of separating high-level AI processing from low-level, real-time motor control while keeping both systems reliably synchronized. Many existing systems compound this challenge by relying on cloud-based services for speech recognition and AI, introducing latency, increasing power consumption, and limiting functionality in poorly connected areas. Reliable field operation therefore demands local processing of speech, perception, and decision-making, with minimal response delay.

Locomotion itself adds significant complexity. Walking on uneven terrain requires continuous orientation estimation, precise joint coordination, and rapid balance correction — challenges that intensify when the robot must climb staircases (12–15 cm), navigate irregular obstacles, and transition between terrain types without compromising stability. Reliable navigation further depends on fusing data from multiple sensing modalities — inertial sensors, cameras, distance sensors, and LiDAR — to accurately identify obstacles, recognize staircases, and estimate safe paths, all within limited onboard computational resources.

These challenges highlight the need for an affordable, modular, and intelligent quadruped platform that integrates robust mechanical design, embedded control, AI, reinforcement learning, and computer vision into a single system. The proposed robotic dog addresses this gap through a hybrid architecture combining the ESP32 microcontroller with the NVIDIA Jetson Orin Nano — enabling reliable locomotion over rough terrain, autonomous stair climbing, offline voice recognition (supporting commands such as "stand," "sit," "walk," "follow me," and "climb stairs"), human-following, obstacle avoidance, and adaptive balance

control, all while remaining upgradeable for future additions such as robotic manipulators, cloud AI, and multi-robot collaboration.

The successful realization of this system will provide a cost-effective, scalable research platform that bridges the gap between costly commercial robots and basic educational prototypes, while demonstrating the practical integration of embedded systems, artificial intelligence, and autonomous mobile robotics in real-world applications.

1.4 Applications of the Proposed Robotic Dog

The rapid advancement of robotics and artificial intelligence has significantly expanded the application domains of autonomous quadruped robots. Their ability to navigate challenging environments, maintain stability on uneven terrain, and interact intelligently with humans makes them suitable for a wide range of real-world tasks. The robotic dog proposed in this project has been designed as a versatile platform capable of operating in both indoor and outdoor environments while performing intelligent perception, navigation, and human interaction. The major application areas are discussed below.

1.4.1 Search and Rescue Operations

Disasters such as earthquakes, floods, and building collapses often create environments that are hazardous or inaccessible for human rescuers. The proposed robotic dog can navigate narrow passages, debris, and damaged staircases while using computer vision and distance sensors to monitor its surroundings. Its ability to climb stairs (12–15 cm), avoid obstacles, and maintain balance on rough terrain allows it to assist rescue teams in locating victims and assessing dangerous areas without putting personnel at risk.

1.4.2 Industrial Inspection

Industrial plants, warehouses, and power stations require regular equipment inspection to ensure operational safety. The robotic dog can autonomously patrol predefined routes, capture images, and identify abnormal conditions in environments where wheeled robots often struggle with uneven flooring. Future integration of thermal cameras and gas sensors could further enhance its inspection capabilities.

1.4.3 Military and Border Surveillance

Quadruped robots are increasingly used for reconnaissance and border security due to their mobility on difficult terrain. The proposed system can patrol rocky paths, detect obstacles, and operate using offline voice commands – making it suitable for environments with limited communication infrastructure. Future enhancements may include encrypted communication and GPS-based mission planning.

1.4.4 Smart Home and Personal Assistance

The robotic dog can function as a household assistant, responding to voice commands, following users, and navigating to predefined locations without internet access. Future integration with smart home devices could allow it to control lighting, security systems, and other IoT appliances.

1.4.5 Healthcare and Elderly Assistance

The growing elderly population has increased demand for robotic assistants capable of supporting daily activities. The robotic dog can accompany elderly individuals, follow them through the home, and respond to voice instructions. Additional sensors could enable fall detection and emergency notifications, extending its role into healthcare support.

1.4.6 Agricultural Monitoring

Agricultural fields often contain uneven surfaces and obstacles that limit conventional robot mobility. The proposed system can traverse rough terrain while collecting environmental data, with future integration of multispectral cameras and soil sensors enabling precision agriculture applications such as crop health analysis.

Beyond these deployment scenarios, the open and modular architecture of the proposed system also makes it a practical platform for academic research. It allows students and researchers to experiment with new locomotion algorithms, reinforcement learning models, and sensor fusion strategies without the restrictions imposed by proprietary commercial platforms — making it suitable for robotics competitions and coursework alike. The integration of offline voice recognition, object detection, and person-following further positions the system as a useful testbed for studying human–robot interaction, including natural language interfaces and adaptive robot behavior, with potential extension into social and service robotics applications.

Overall, the proposed robotic dog demonstrates strong potential across multiple application domains by combining autonomous locomotion, AI, and intelligent perception within a single modular platform suited for both research and practical deployment.

1.5 Objectives of the Project

1.5.1 General Objective

The primary objective of this project is to design, develop, and implement an intelligent autonomous quadruped robotic dog capable of stable locomotion, intelligent perception, and

human-robot interaction, by integrating embedded systems, artificial intelligence, computer vision, reinforcement learning, and autonomous navigation into a modular, cost-effective platform. The system is intended to operate efficiently in both indoor and outdoor environments, adapting to different terrains while performing complex tasks with minimal human intervention.

1.5.2 Specific Objectives

To accomplish the overall goal of the project, the following specific objectives have been identified:

- To design and fabricate a mechanically robust quadruped robot with 12 degrees of freedom using MG996R servo motors, ensuring sufficient strength, stability, and mobility for real-world operation.
- To develop a real-time embedded control system using the ESP32 microcontroller for accurate servo control, sensor acquisition, and low-latency motion execution.
- To integrate an NVIDIA Jetson Orin Nano for executing AI algorithms, computer vision models, reinforcement learning policies, and autonomous navigation functions.
- To enable stable locomotion across multiple terrain types including smooth floors, grass, gravel, rocky paths, and inclined surfaces while maintaining dynamic balance.
- To implement autonomous stair-climbing for step heights of 12–15 cm through adaptive gait planning and balance control.
- To develop an offline voice command system recognizing commands such as stand, sit, walk, stop, and follow me, without requiring internet connectivity.
- To implement environmental perception using cameras, Time-of-Flight sensors, IMU data, and future LiDAR integration for obstacle detection and safe navigation.
- To develop a human-following system capable of detecting, tracking, and maintaining a safe distance from a designated individual using computer vision and sensor fusion.
- To implement reinforcement learning-based locomotion using PPO for adaptive gait generation and efficient movement across varying terrain.
- To integrate sensor fusion using an Extended Kalman Filter (EKF) for accurate estimation of body orientation, pose, and environmental state.
- To design a modular hardware and software architecture allowing future integration of manipulators, cloud connectivity, GPS navigation, and autonomous charging without major redesign.
- To ensure reliable and safe operation through battery monitoring, thermal protection, emergency stop mechanisms, watchdog timers, and fault detection.

1.5.3 Expected Outcomes

Upon successful completion, the proposed robotic dog is expected to achieve:

- Stable quadruped locomotion with dynamic balance control
- Autonomous navigation across rough and uneven terrain
- Reliable stair climbing for heights between 12 cm and 15 cm
- Offline voice-based interaction with low response latency
- Real-time object detection and obstacle avoidance
- Accurate human-following capability
- Adaptive locomotion using reinforcement learning
- Efficient integration of embedded systems and artificial intelligence
- A modular platform suitable for future academic and industrial research
- A cost-effective alternative to expensive commercial quadruped robots

1.6 Scope of the Project

The scope of this project covers the complete design, development, implementation, and evaluation of an intelligent autonomous quadruped robotic dog intended for indoor and semi-structured outdoor environments. It brings together mechanical design, embedded systems, artificial intelligence, computer vision, and autonomous robotics into a single modular platform, primarily intended for research and educational use rather than heavy industrial deployment.

The project's coverage extends across the full robot lifecycle — from mechanical structure and embedded hardware to AI-based perception, locomotion control, and navigation — with each subsystem designed to be independently upgradeable. This includes core capabilities such as terrain-adaptive walking, autonomous stair climbing (12–15 cm), offline voice interaction, and human-following, as outlined in the project objectives.

The current work does not extend to large-scale outdoor exploration, operation in extreme environmental conditions (such as underwater or high-temperature settings), or full GPS-based outdoor navigation. Features such as robotic manipulation, cloud connectivity, autonomous charging, and multi-robot collaboration are considered part of the platform's future expansion and fall outside the scope of the present implementation.

Within these boundaries, the project focuses on demonstrating a functional, cost-effective autonomous platform suitable for academic research, providing a foundation that can be extended in future work toward broader real-world deployment.

1.7 Organization of the Report

This report is organized into twelve chapters, each addressing a specific aspect of the design, development, implementation, and evaluation of the proposed robotic dog system. Table 1.1 summarizes the content covered in each chapter.

Chapter	Title	Description
1	Introduction	Presents the background, evolution of quadruped robotics, problem statement, objectives, scope, and potential applications of the proposed robotic dog.
2	Literature Review	Reviews existing research on quadruped robotics, reinforcement learning, autonomous navigation, computer vision, and embedded control systems, identifying the research gap addressed by this project.
3	System Architecture	Describes the overall system architecture, including functional requirements, block diagrams, and the hardware and software framework.
4	Mechanical Design	Covers chassis development, leg mechanism, joint configuration, actuator selection, and CAD modeling.
5	Electronic Hardware Implementation	Explains the integration of the ESP32 microcontroller, NVIDIA Jetson Orin Nano, servo motors, PWM drivers, sensors, and power management system.
6	Software Architecture	Presents the embedded firmware development, communication protocols, and integration of AI frameworks used in the platform.
7	Locomotion and Control	Discusses reinforcement learning, gait planning, inverse kinematics, balance control, and terrain adaptation strategies.
8	Perception and Navigation	Describes object detection using YOLOv8, voice command recognition, obstacle avoidance, and human-following algorithms.

9	Experimental Setup and Testing	Presents the testing methodology and performance evaluation under different operating conditions, including stair climbing and terrain traversal.
10	Results and Discussion	Discusses experimental results, performance analysis, limitations, and comparison with existing quadruped platforms.
11	Conclusion	Summarizes the project's achievements, conclusions, and overall contributions.
12	Future Scope	Outlines future research directions, including improvements in AI, navigation, robotic manipulation, and multi-robot collaboration.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Recent advances in artificial intelligence, embedded computing, and autonomous control have significantly accelerated the development of quadruped robots. Unlike wheeled mobile robots, quadruped robots can traverse uneven terrain, climb obstacles, recover from disturbances, and maintain stability in environments where conventional robotic platforms often fail. Consequently, quadruped robots have attracted considerable attention for applications including industrial inspection, disaster response, healthcare assistance, military reconnaissance, warehouse automation, and agricultural monitoring. Modern research increasingly combines reinforcement learning, computer vision, sensor fusion, and embedded edge computing to develop autonomous robots capable of perception, navigation, and adaptive locomotion in complex environments. Comprehensive reviews published in 2024 and 2025 highlight that current research trends focus on intelligent locomotion, lightweight mechanical design, real-time perception, and autonomous decision-making, supported by advances in deep learning and embedded AI hardware [1], [2].

The availability of compact edge computing platforms such as the NVIDIA Jetson Orin Nano and low-cost microcontrollers such as the ESP32 has enabled researchers to deploy computationally intensive deep-learning algorithms directly on mobile robots.

Simultaneously, lightweight object detection models such as YOLOv8 and reinforcement learning algorithms such as Proximal Policy Optimization (PPO) have become widely adopted because they provide an effective balance between computational efficiency and real-time performance [3], [4]. These technological developments have facilitated the construction of affordable quadruped robots capable of autonomous locomotion, obstacle detection, object recognition, and human–robot interaction without relying on expensive commercial platforms.

This chapter reviews recent literature related to quadruped robot platforms, reinforcement learning-based locomotion, embedded computer vision, autonomous navigation, human-following systems, and distributed embedded control architectures. The review critically evaluates current approaches, identifies their limitations, and establishes the research gap addressed by the proposed intelligent quadruped robot.

2.2.1 Recent Advances in Quadruped Robot Platforms

Quadruped robots have experienced significant development during the past few years due to improvements in mechanical design, embedded electronics, and intelligent control algorithms. Recent survey papers emphasize that modern quadruped robots are no longer designed solely for locomotion but increasingly integrate perception, navigation, environmental mapping, and autonomous decision-making within a single robotic platform [1], [2]. Contemporary research therefore focuses on achieving robust locomotion while simultaneously reducing system complexity, computational requirements, and hardware cost.

One major research direction involves developing affordable quadruped platforms suitable for education and laboratory research. Giri et al. presented a low-cost quadruped robot incorporating an ESP32-CAM module for real-time video streaming and web-based teleoperation, an Arduino Uno for low-level gait control and servo processing, MG996R servo motors for leg actuation, and an HC-SR04 ultrasonic sensor for obstacle detection [3]. The proposed distributed architecture demonstrated that separating high-level perception from low-level motor control can successfully achieve autonomous movement while maintaining significantly lower system cost than commercial quadruped platforms. The hierarchical control architecture enabled communication between the ESP32-CAM and Arduino Uno via UART serial communication, allowing simultaneous execution of vision and locomotion tasks without overloading either processor.

Recent developments further indicate a transition toward distributed embedded architectures in which computationally intensive tasks, including computer vision and deep learning inference, are executed on dedicated edge-computing devices while microcontrollers independently manage real-time servo control and sensor acquisition. Such architectures

reduce computational latency, improve real-time performance, and enhance system reliability by isolating time-critical control loops from perception algorithms [3].

Although recent educational quadruped robots successfully demonstrate autonomous locomotion using inexpensive hardware, many continue to rely on predefined gait trajectories and conventional control strategies rather than adaptive learning-based approaches. Furthermore, relatively few low-cost platforms integrate reinforcement learning with onboard computer vision for autonomous navigation, leaving considerable scope for research into intelligent, affordable quadruped robots that combine adaptive locomotion with embedded artificial intelligence.

2.2.2 Reinforcement Learning for Quadruped Locomotion

Reinforcement learning has become one of the most influential approaches for controlling quadruped robots because it enables robots to learn optimal locomotion strategies directly through interaction with simulated environments instead of relying on explicit mathematical models. Recent reviews identify reinforcement learning as one of the dominant research directions in legged robotics due to its ability to improve robustness, adaptability, and stability across diverse terrains [1], [2].

Among existing reinforcement learning algorithms, Proximal Policy Optimization (PPO) remains the most widely adopted because of its stable policy optimization, computational efficiency, and comparatively simple implementation. Choudhury et al. demonstrated that PPO enables quadruped robots to autonomously learn stable locomotion policies capable of navigating uneven terrain while maintaining dynamic balance [4]. Experimental evaluation showed that continuous interaction between the robot and its simulated environment allowed the learned policy to outperform manually designed gait controllers under varying environmental conditions. The PPO algorithm optimizes policy updates within safe bounds, balancing exploration and exploitation to enhance adaptability and minimize instability in control tasks.

Although conventional PPO provides reliable locomotion performance, recent research has focused on improving its ability to handle partial observability and long-term temporal dependencies. Zhao et al. introduced the LA-PPO algorithm, which integrates Long Short-Term Memory (LSTM) networks with Multi-Head Attention (MHA) into the actor-critic framework [5]. The additional recurrent memory enables the controller to preserve historical information across timesteps, while the attention mechanism selectively emphasizes the most relevant historical observations during policy optimization. Experimental results demonstrated approximately 40% higher cumulative rewards, improved stability, and longer successful locomotion episodes compared with conventional PPO when tested on flat, inclined, and irregular terrains in the IsaacGym simulation environment.

Similarly, recent hierarchical reinforcement learning frameworks separate locomotion into multiple control layers responsible for gait generation, body stabilization, and velocity regulation. This decomposition improves learning efficiency and produces smoother gait transitions while reducing unnecessary energy consumption during locomotion. Such hierarchical architectures demonstrate that reinforcement learning research is progressing toward increasingly modular and adaptive control frameworks rather than relying exclusively on monolithic end-to-end neural networks.

Despite these improvements, advanced reinforcement learning algorithms generally require substantially greater computational resources, longer training times, and more complex implementation than conventional PPO. Consequently, standard PPO remains an attractive solution for embedded robotic platforms because it offers an effective compromise between computational efficiency, implementation simplicity, and locomotion performance. For undergraduate research and low-cost quadruped systems, PPO therefore provides a practical baseline while allowing future integration of more sophisticated reinforcement learning techniques.

2.2.3 Computer Vision and Embedded Perception

Computer vision has become an essential component of autonomous quadruped robots, enabling environmental perception, obstacle detection, target recognition, and navigation. Recent advancements in deep learning have significantly improved object detection accuracy while simultaneously reducing computational complexity, making deployment on embedded edge-computing platforms increasingly practical. Among current object detection algorithms, the YOLO (You Only Look Once) family has emerged as one of the most widely adopted approaches because it performs object localization and classification within a single neural network, thereby achieving high inference speed suitable for real-time robotic applications.

The release of YOLOv8 introduced architectural improvements including an anchor-free detection head, enhanced feature extraction backbone, and optimized loss functions, resulting in improved detection accuracy and computational efficiency compared with earlier YOLO versions. Zachariadis et al. conducted a systematic performance benchmarking of multiple YOLOv8 variants on the NVIDIA Jetson Orin NX platform, evaluating inference latency, frame throughput, and computational resource usage across varying input sizes and model complexities [6]. Their findings showed that the TensorRT-optimized model outperformed the PyTorch baseline by 17.7% at a batch size of 2, while lightweight YOLOv8 variants achieved real-time inference speeds sufficient for mobile robotic applications. These results indicate that lightweight models are generally more appropriate for mobile robotic platforms where computational resources and power consumption are constrained.

Similarly, Godase et al. evaluated YOLOv8 for autonomous drone navigation under varying environmental conditions, including urban, rural, and mixed outdoor scenarios [7]. Experimental results showed that YOLOv8s achieved a mean Average Precision of 89.3% at 45.2 FPS on the NVIDIA Jetson AGX Xavier, demonstrating reliable real-time object detection performance during autonomous navigation under illumination changes and dynamic backgrounds. Although the application focused on aerial robotics, the reported results are directly relevant to quadruped robots because both platforms require continuous environmental perception using resource-constrained embedded hardware.

Beyond benchmarking studies, recent research has increasingly integrated YOLOv8 with additional sensing modalities to improve perception robustness. Schakkal et al. proposed a dynamic object catching framework for quadruped robots that combined a fine-tuned YOLOv8 model for object detection with a regression-based trajectory prediction module, enabling the robot to intercept thrown objects using its front legs [8]. The system demonstrated that vision-based perception combined with onboard trajectory prediction can enable complex interactive tasks that extend well beyond simple obstacle detection, achieving an 80% catching success rate across diverse scenarios.

Recent developments also indicate increasing adoption of vision-based terrain understanding for quadruped locomotion. Woźniak et al. proposed an RGB-based staircase detection algorithm verified on the Unitree Go1 quadruped robot, utilizing a YOLO version 11n deep neural network trained on more than 26,000 color images collected across 18 sequences to detect staircases in real time from both front and bottom camera perspectives [9]. Experimental results demonstrated an average mAP@50 of 87.56 across all images, enabling early hazard detection during stair navigation on embedded robotic platforms.

Despite these advances, most existing studies evaluate perception algorithms independently from locomotion control. Only limited research integrates reinforcement learning-based gait generation with real-time onboard object detection within a single embedded quadruped platform. Furthermore, many vision-based systems rely on high-performance GPU hardware that significantly increases system cost. Therefore, developing a low-cost quadruped robot capable of simultaneously executing PPO-based locomotion and lightweight YOLOv8 object detection on embedded hardware remains an important research direction.

2.2.4 Vision-Based Human Detection and Following

Human-following represents one of the most important capabilities of service robots, enabling autonomous interaction in healthcare, domestic assistance, logistics, and industrial inspection. Recent advances in deep learning have substantially improved the robustness of person detection and tracking under varying illumination, partial occlusion, and crowded environments. Consequently, vision-based human-following has become the preferred

solution for autonomous mobile robots because it eliminates the need for wearable tracking devices while allowing natural interaction between humans and robots.

García-Gómez et al. proposed a robust person identification and following framework that combines convolutional neural network-based detection with optical tracking techniques to maintain continuous target tracking on an NVIDIA Jetson TX2 embedded mobile robot platform [10]. The system demonstrated reliable identification of specific individuals while maintaining real-time performance under moderate occlusion and dynamic background conditions. The integration of deep-learning-based detection with optical tracking significantly reduced computational requirements by limiting full neural network inference to periodic updates while using lightweight tracking algorithms between detections. Experimental validation confirmed that the proposed approach successfully followed a designated person in real indoor environments.

Cao et al. demonstrated vision-based moving target tracking for quadruped robots using a deep learning-based detection algorithm combined with a simple but accurate target selection strategy [11]. Their system utilized visual distance and angle information derived from detected bounding boxes to realize autonomous tracking in outdoor environments containing interference factors such as illumination changes, people crossing, and partial occlusion by obstacles. Experimental results confirmed the robustness of the approach across diverse and uncontrolled outdoor conditions.

More recent work has focused specifically on quadruped robots in dynamic environments. Scheidemann et al. proposed an obstacle-aware leader-following framework for the ANYmal quadruped platform that integrated an Extended Kalman Filter (EKF) for leader state estimation with a custom Riemannian Motion Policy-based local obstacle avoidance planner [12]. The sensor fusion architecture combined a custom-built RF transponder, RGB cameras, and a Velodyne VLP-16 LiDAR to provide redundant target tracking, enabling the robot to maintain safe following distances while simultaneously avoiding static and dynamic obstacles in narrow and crowded environments. Experimental evaluation demonstrated stable tracking performance under varying real-world conditions, including temporary occlusions and dynamic passersby.

Kurumbaparambil et al. presented a stereo vision-based person-following algorithm for the Unitree Go1 quadruped platform designed for assistive elderly-care applications [13]. The system employed a ZED 2 stereo camera and a finite state machine implemented within ROS to ensure deterministic target tracking, with a velocity control strategy partitioned into stability, proportional, and braking zones based on real-time depth data. Experimental validation in outdoor environments involving 90-degree turns demonstrated improved tracking robustness compared with the robot's native following mode, highlighting the

increasing importance of safety-conscious perception systems in autonomous robotic assistants.

Although recent studies demonstrate significant improvements in person-following accuracy, most systems have been evaluated using commercial robotic platforms equipped with expensive sensors or high-performance computing hardware. Consequently, relatively little research has investigated low-cost quadruped robots capable of executing real-time human-following algorithms using affordable embedded computing platforms. This limitation motivates the proposed system, which combines lightweight YOLOv8-based person detection with embedded edge computing to achieve autonomous human following on a low-cost quadruped robot.

2.2.5 Sensor Fusion and State Estimation

Reliable state estimation is fundamental for autonomous quadruped locomotion because control algorithms require accurate information regarding robot position, orientation, velocity, and environmental conditions. Recent research increasingly emphasizes sensor fusion techniques that combine multiple sensing modalities to improve localization accuracy while reducing sensitivity to individual sensor failures.

Modern quadruped robots typically integrate inertial measurement units (IMUs), joint encoders, cameras, LiDAR, ultrasonic sensors, and depth sensors to estimate robot motion. Rather than relying on a single sensing modality, recent sensor fusion frameworks combine complementary measurements to improve robustness under dynamic operating conditions. The integration of visual perception with inertial sensing has become particularly common because cameras provide rich environmental information while IMUs deliver high-frequency motion estimates that remain available even when visual features become temporarily unavailable [1], [2].

Scheidemann et al. demonstrated that combining visual detection with EKF-based sensor fusion using RF and camera measurements substantially improved leader localization robustness compared with vision-only systems in cluttered and dynamic environments [12]. Similarly, Becoy et al. showed that integrating multi-sensor streams within a ROS2-based navigation architecture improved mapping consistency and path coverage in complex indoor environments using quadruped robots [14]. These studies indicate that multi-sensor fusion has become the preferred strategy for achieving reliable autonomous navigation in complex and dynamic environments.

Although sophisticated sensor fusion algorithms continue to improve localization performance, they generally require additional hardware, higher computational resources, and increased implementation complexity. Consequently, low-cost quadruped robots frequently

adopt simplified sensor fusion strategies combining IMUs, ultrasonic sensors, and vision-based perception to achieve an effective balance between localization accuracy, computational efficiency, and hardware affordability. This approach is consistent with the objectives of the proposed system, which emphasizes practical implementation using commercially available embedded components including an Extended Kalman Filter for body orientation and pose estimation.

2.2.6 Autonomous Navigation and Path Planning

Autonomous navigation is a fundamental capability of intelligent quadruped robots because it enables operation in unknown and dynamic environments without continuous human intervention. Navigation systems generally consist of localization, mapping, path planning, obstacle avoidance, and motion execution. Recent advances in artificial intelligence and robotic perception have shifted research from traditional rule-based navigation toward adaptive systems capable of making autonomous decisions using sensor fusion and deep-learning techniques.

One of the most significant developments in recent quadruped navigation research is the integration of coverage path planning with topological map representations. Becoy et al. proposed an autonomous navigation framework for quadruped robots that utilizes morphological skeleton maps generated from two-dimensional occupancy grids produced by SLAM to generate a sequence of points of interest [14]. A finite state machine was used to switch between navigation and scanning modes, controlling a Unitree Go2 quadruped through a ROS2-based software stack. Experimental evaluation demonstrated successful autonomous navigation in complex indoor environments with improved path efficiency compared with exploration approaches lacking structural map representations.

Recent studies have also investigated integrating navigation with intelligent perception systems. Rather than separating object detection from navigation, modern robotic architectures combine visual perception, obstacle recognition, and path planning into a unified decision-making framework. This integration allows quadruped robots to dynamically modify their trajectories based on real-time environmental information instead of following predefined paths, improving navigation reliability when operating in cluttered or continuously changing environments.

Leader-following has become another important navigation strategy for service robotics. Scheidemann et al. proposed an obstacle-aware leader-following framework in which the ANYmal quadruped simultaneously tracks a moving leader while continuously avoiding static and dynamic obstacles using a local motion planner adapted for legged platforms [12]. The system combines visual target tracking with reactive obstacle avoidance, enabling smooth navigation without interrupting human-following behaviour. Experimental results

demonstrated stable tracking performance under varying environmental conditions while maintaining safe navigation distances.

Computer vision has further enhanced autonomous navigation through environmental understanding. Woźniak et al. demonstrated RGB-based staircase detection enabling quadruped robots to identify traversable staircases and hazardous terrain in real time before locomotion planning [9], while Godase et al. validated YOLOv8-based perception for autonomous navigation under diverse real-world conditions [7]. These developments indicate that perception-driven navigation is becoming the preferred approach for modern quadruped robots.

Although recent navigation systems achieve impressive autonomy, most rely on expensive sensors including LiDAR, stereo cameras, or high-precision localization systems. Such hardware substantially increases implementation cost and power consumption. Consequently, affordable quadruped robots continue to employ simplified navigation strategies using monocular vision, ultrasonic sensors, and inertial measurements. These approaches provide satisfactory navigation performance while maintaining low system complexity and reduced hardware cost, making them suitable for educational and research-oriented robotic platforms.

2.2.7 Distributed Embedded Control Architecture

Embedded control architecture plays a critical role in determining the responsiveness, computational efficiency, and reliability of autonomous quadruped robots. Modern robotic systems increasingly adopt distributed processing architectures that separate computationally intensive perception tasks from low-level real-time motor control. This hierarchical design reduces computational latency while improving system stability during simultaneous execution of locomotion, perception, and navigation algorithms.

Recent low-cost quadruped platforms commonly utilize microcontrollers such as the ESP32 for servo control while assigning computer vision and artificial intelligence tasks to dedicated edge-computing devices. Giri et al. adopted this architecture by employing an ESP32-CAM module for wireless image acquisition and an Arduino Uno for gait generation and servo control, with UART serial communication at 115,200 baud enabling simultaneous execution of perception and locomotion tasks [3]. Experimental evaluation demonstrated successful locomotion across various surfaces, average control latency of 95 ms for web-based teleoperation, and continuous operation time of 1.8 hours on a 7.4 V Li-Po battery.

The increasing availability of embedded AI platforms such as the NVIDIA Jetson Orin Nano has significantly expanded the capabilities of mobile robotic systems. Compared with conventional microcontrollers, Jetson-based platforms provide GPU acceleration for deep neural network inference, enabling real-time execution of advanced computer vision

algorithms including YOLOv8 detection and reinforcement learning policies. Benchmark studies demonstrated that YOLOv8 variants with TensorRT optimization achieve substantially improved inference throughput on the Jetson Orin NX compared with PyTorch deployment, indicating that embedded GPU platforms have become practical solutions for deploying artificial intelligence directly on autonomous robots [6].

Distributed embedded architectures also improve fault tolerance and scalability. By isolating motor control from perception and decision-making, failures occurring within the vision subsystem do not directly interrupt low-level locomotion control. Similarly, perception algorithms can be upgraded independently without modifying the embedded control software responsible for gait generation. Such modularity simplifies maintenance while facilitating future integration of additional sensors and autonomous behaviours.

Wireless communication protocols have similarly become integral components of embedded robotic systems. ESP32-based controllers support Wi-Fi and Bluetooth communication, allowing remote monitoring, telemetry transmission, firmware updates, and cloud connectivity without requiring additional communication hardware. Combined with ROS2 middleware, these communication capabilities enable distributed robotic architectures suitable for autonomous navigation, multi-robot coordination, and Internet of Things (IoT) applications.

Despite these technological advances, balancing computational capability, power consumption, and hardware cost remains a major challenge in embedded robotics. Many commercial quadruped robots employ powerful industrial computers that significantly increase system cost and energy consumption. Consequently, recent research increasingly emphasizes affordable embedded architectures that combine low-cost microcontrollers with compact edge-computing platforms to achieve real-time autonomy without sacrificing economic feasibility. The proposed system follows this design philosophy by integrating an ESP32-based motor controller with an NVIDIA Jetson Orin Nano for computer vision, reinforcement learning inference, and autonomous navigation.

Table 2.1 presents a consolidated comparative summary of all reviewed works, highlighting the key distinctions between existing studies and the proposed system across platform, locomotion method, perception, human-following capability, navigation, embedded hardware, and cost level.

Ref. (Author, Year)	Objective	Method / Technology	Limitation
Giri et al. [3] (2024)	Developed a low-cost quadruped robot	ESP32-CAM, Arduino, ultrasonic sensor	Uses predefined gait control; no reinforcement learning
Choudhury et al. [4] (2024)	Improve quadruped locomotion	PPO reinforcement learning	High computational requirements
Zhao et al. [5] (2025)	Enhance PPO performance	LA-PPO (LSTM + Attention)	Increased training complexity
Zachariadis et al. [6] (2024)	Benchmark embedded object detection	YOLOv8 + TensorRT on Jetson	Only evaluates inference performance
Godase et al. [7] (2024)	Autonomous navigation	YOLOv8 on Jetson AGX Xavier	Drone application only
Schakkal et al. [8] (2024)	Dynamic object catching	YOLOv8 + trajectory prediction	No locomotion integration
Woźniak et al. [9] (2025)	Staircase detection	YOLO11n vision model	Focused only on terrain detection

García-Gómez et al. [10] (2023)	Human following	CNN + optical tracking	Tested on wheeled robots
Cao et al. [11] (2024)	Moving target tracking	Deep-learning vision	Limited navigation integration
Scheidemann et al. [12] (2024)	Leader following	EKF + LiDAR + RGB camera	Expensive hardware
Kurumbaparambil et al. [13] (2024)	Person following	Stereo vision + ROS	Requires stereo camera
Becoy et al. [14] (2025)	Autonomous navigation	ROS2 + SLAM	Depends on LiDAR

2.3 Research Gap

The reviewed literature demonstrates considerable progress in reinforcement learning, embedded perception, and autonomous navigation for quadruped robots. Nevertheless, several research challenges remain unresolved, particularly for affordable robotic platforms.

First, most reinforcement learning studies evaluate advanced locomotion algorithms using expensive commercial quadruped robots equipped with powerful onboard computers and high-performance sensors [1], [2]. Although systems such as Boston Dynamics Spot and ANYmal demonstrate exceptional locomotion performance, their implementation cost limits accessibility for educational institutions and undergraduate research laboratories.

Second, numerous computer vision studies investigate YOLOv8 deployment on embedded GPU platforms [6], [7]; however, these investigations primarily focus on benchmarking inference performance or aerial robotics applications rather than integrating object detection

with adaptive quadruped locomotion and autonomous navigation within a single low-cost embedded platform.

Third, recent human-following systems have demonstrated robust tracking performance through deep-learning-based detection and sensor fusion [12], [13]. However, many implementations rely on commercial robotic platforms or high-cost sensing hardware, including stereo cameras, LiDAR, and RF transponders, that restrict widespread adoption in low-cost robotic systems.

Fourth, autonomous navigation research increasingly incorporates LiDAR, stereo vision, and sophisticated SLAM algorithms [14]. Although these techniques improve localization accuracy, they also increase computational complexity, hardware requirements, and overall system cost beyond what is feasible for educational robotic platforms.

Finally, relatively few studies have integrated reinforcement learning-based locomotion, lightweight YOLOv8 perception, embedded edge computing, ESP32-based motor control, and autonomous human-following within a single affordable quadruped robot. Existing research typically investigates these technologies independently rather than combining them into a unified embedded robotic platform.

Therefore, the proposed research addresses this gap by developing a low-cost intelligent quadruped robot that integrates PPO-based locomotion, YOLOv8 object detection, ESP32-based real-time motor control, NVIDIA Jetson Orin Nano edge computing, and autonomous human-following capabilities within a modular distributed architecture. The proposed system aims to provide a practical and economically accessible platform for research, education, and future development of intelligent quadruped robots.

2.4 Chapter Summary

This chapter reviewed recent developments in quadruped robotics with emphasis on intelligent locomotion, reinforcement learning, embedded computer vision, autonomous navigation, human-following systems, and distributed embedded control architectures. The review indicates that reinforcement learning — and PPO in particular — has become the dominant approach for adaptive quadruped locomotion, while YOLOv8 has emerged as one of the most effective lightweight object detection algorithms for embedded robotic applications. Recent advances in sensor fusion and autonomous navigation have further enhanced the operational capabilities of quadruped robots operating in complex environments.

Despite these advances, existing systems frequently depend on expensive hardware, computationally intensive algorithms, or commercial robotic platforms that limit their

accessibility for educational and research applications. Furthermore, relatively few studies have integrated reinforcement learning, embedded computer vision, autonomous navigation, and low-cost hardware into a unified quadruped robot.

Based on the identified research gap, the proposed work develops a modular intelligent quadruped robot that combines PPO-based locomotion, real-time YOLOv8 object detection, autonomous human-following, and embedded edge computing using affordable hardware. The proposed architecture contributes toward the development of practical and economically accessible quadruped robotic systems suitable for research, education, and real-world autonomous applications.

CHAPTER 3

PROPOSED SYSTEM DESIGN AND ARCHITECTURE

3.1 System Overview

The proposed system is an autonomous, 12-Degree-of-Freedom (12-DOF) quadruped robotic dog designed for adaptive locomotion across uneven terrains (grass, gravel, low-profile obstacles). The platform utilizes a distributed embedded architecture to separate high-level perception from real-time motor execution, optimizing power consumption and processing efficiency.

1. Processing and Control Architecture

Computational tasks are split into two discrete tiers based on timing requirements:

- **High-Level Computing (NVIDIA Jetson Orin Nano):** Processes computer vision, calculates path planning, and executes real-time inference of the locomotion policy.
- **Low-Level Execution (ESP32 Microcontroller):** Handles time-critical, deterministic tasks including real-time state estimation, sensor data ingestion, and joint trajectory stream timing.
- **Actuator Driver (PCA9685):** Offloads PWM generation from the ESP32. It controls 14 active channels (12 leg joints, 2 head/tail joints) over an I2C bus configured to Fast Mode (400 kHz).

2. Sensor Integration and Environmental Perception

To operate autonomously, the robot integrates a multi-modal sensor array:

- **GY-87 IMU:** Provides continuous angular velocity and linear acceleration data to the ESP32 for closed-loop balance compensation.
- **Onboard RGB Camera:** Feeds data directly to the Jetson Orin Nano for object detection, obstacle identification, and human tracking via a lightweight YOLOv8 deep learning model.
- **VL53L0X Time-of-Flight (ToF):** Delivers localized distance tracking for immediate front-facing obstacle avoidance.
- **I2S Digital Microphone:** Implements offline voice command processing directly on the Jetson, eliminating external cloud dependency.

3. Artificial Intelligence and Locomotion Strategy

Locomotion is governed by a Reinforcement Learning (RL) policy developed via the Proximal Policy Optimization (PPO) algorithm. To meet embedded hardware constraints, the RL network is trained offline within a high-fidelity physics simulator. The optimized, lightweight policy is then deployed onto the Jetson Orin Nano for real-time edge inference.

When confronting steps or small obstacles (5 cm to 8 cm), the robot combines vision-based geometric mapping with Inverse Kinematics (IK) to dynamically alter leg trajectories. Continuous IMU feedback enables real-time adjustments to body tilt and unexpected terrain shifts.

4. Power Architecture and Safety

The robot is powered by a high-discharge 3-cell Lithium Polymer (LiPo) battery pack managed by a dedicated safety system.

- **Battery Management System (BMS):** A 3S BMS is integrated directly with the LiPo battery pack to handle cell balancing, over-discharge protection, and short-circuit monitoring, ensuring safe operation under heavy current draws.
- **Rail Isolation:** High-current DC-DC buck converters step down the voltage to create dedicated, isolated power rails for the servo motors, keeping them separate from the sensitive logic electronics.
- **Stability:** Separating the power path of the 14 servos from the ESP32 and Jetson Orin Nano prevents voltage drops and processor brownouts during high-load stall events.

- **Grounding:** All electronics share a unified common ground network to eliminate signal drift and ensure I²C/UART communication integrity.

3.2 Functional Requirements

Functional requirements define the specific operations and capabilities that the proposed robotic dog must perform to achieve its intended objectives. These requirements serve as the foundation for the mechanical design, embedded system architecture, software implementation, and artificial intelligence algorithms developed throughout the project. The proposed robotic dog is designed to perform the following functions.

3.2.1 Autonomous Locomotion

The robotic dog shall achieve autonomous locomotion using twelve independently controlled servo motors arranged in a 12-DOF quadruped configuration. The low-level controller shall generate coordinated gait patterns—including standing, walking, and trotting—while maintaining smooth balancing transitions between locomotion states.

3.2.2 Rough Terrain Traversal

The locomotion system shall dynamically adapt to rough trails, gravel, grass, and inclined paths. The ESP32 shall continuously process data from the GY-87 Inertial Measurement Unit (IMU) to estimate body orientation and execute real-time postural corrections against external disturbances and uneven surfaces.

3.2.3 Stair Climbing

The robotic dog shall autonomously detect and climb scaled steps ranging from 5 cm to 8 cm in height. The perception system must verify step geometry before initiating the sequence, using adaptive Inverse Kinematics (IK) trajectories to maintain stability, prevent slippage, and manage center-of-mass shifts during ascent and descent.

3.2.4 Obstacle Detection and Avoidance

The robot shall continuously scan its environment using the onboard camera and VL53L0X Time-of-Flight (ToF) sensor. The navigation system must automatically classify obstacles, calculate alternative local paths, and reroute around obstructions within a predefined safety zone without interrupting locomotion stability.

3.2.5 Human Following

The vision system shall utilize deep learning tracking algorithms to identify, lock onto, and follow a designated individual. The system must dynamically adjust walking trajectories to maintain a safe following distance and trigger an automated re-identification routine if visual contact is temporarily broken.

3.2.6 Voice Command Recognition

The system shall process offline, edge-based voice recognition via the I2S digital microphone, completely independent of cloud connectivity. The system must recognize and execute the following explicit vocal commands with low latency: *Stand, Sit, Walk, Stop, Follow Me, Turn Left, Turn Right, Climb Stairs, and Come Here.*

3.2.7 Object Detection

The high-level perception system shall run a lightweight YOLOv8 deep learning model on the Jetson Orin Nano for real-time object detection. The robot must identify humans, critical obstacles, and stairs to enhance environmental awareness, utilizing a modular software structure to support custom object classes in future iterations.

3.2.8 Balance and Stability Control

The closed-loop balance controller must continuously analyze inertial sensor data to apply instant corrective joint movements. The control loop shall actively counteract sudden physical pitches, tilts, or slipping events to significantly minimize the probability of a robot fall.

3.2.9 Sensor Fusion

The control framework shall integrate data streams from the IMU, ToF sensor, and camera using sensor fusion algorithms. This unified environmental map must provide superior localization, safer obstacle boundaries, and cleaner spatial perception than any of the individual sensors operating independently.

3.2.10 Real-Time Servo Control

The ESP32 microcontroller shall stream precise, real-time joint positions to a single PCA9685 16-channel PWM controller over a Fast-Mode (400 kHz) I²C bus. This hardware configuration must synchronize 14 active channels (12 leg servos, 2 auxiliary head/tail servos) to eliminate timing stutters and ensure smooth gait execution.

3.2.11 Power Management and Distribution

The system shall actively monitor power metrics across isolated power paths fed by a 3-cell LiPo battery. High-current buck converters must supply the servo motor rail independently from the regulated logic rails powering the ESP32 and Jetson Orin Nano, preventing system brownouts or signal noise during peak motor loading.

3.2.12 Safety Monitoring

A dedicated 3S Battery Management System (BMS) paired with the control software shall continuously monitor battery voltage levels, overcurrent surges, cell balancing, and communication heartbeats. If a critical hardware fault, critical low voltage, or orientation failure is detected, the system must trigger protective actions, including emergency servo shutdowns or a safe recovery stance.

3.2.13 Platform Expandability

The hardware interfaces and software architecture shall remain strictly modular. The communication framework must support the future integration of hardware and software expansions—including LiDAR modules, robotic arms, GPS receivers, and conversational AI models—without requiring structural redesigns of the core platform.

3.3 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance criteria, reliability, safety, and operational constraints of the proposed robotic dog. Unlike functional requirements, which describe the tasks performed by the robot, non-functional requirements define the expected level of performance and system behavior under various operating conditions. These requirements ensure that the robotic platform operates efficiently, safely, and reliably in real-world environments.

3.3.1 Performance Requirements

The robotic dog shall provide smooth and stable locomotion while maintaining real-time responsiveness during autonomous operation. The embedded control system shall execute servo control, sensor acquisition, and communication with minimal delay to ensure coordinated movement across different terrains. Artificial intelligence algorithms, including object detection and reinforcement learning inference, shall operate in real time without affecting the stability of the locomotion controller.

The robot shall support multiple locomotion modes, including standing, walking, trotting, and running. It shall maintain stable movement across flat surfaces, rough trails, gravel roads, grass, and uneven terrain while autonomously climbing staircases with step heights between **5 cm and 8cm**.

3.3.2 Reliability

The proposed robotic dog shall operate continuously with a high degree of reliability during experimental and field testing. Critical functions such as locomotion, balance control, obstacle detection, and voice recognition shall remain operational even during prolonged use.

Communication between the ESP32 microcontroller and the NVIDIA Jetson Orin Nano shall remain stable throughout robot operation.

The system shall include watchdog timers, fault detection mechanisms, and automatic recovery procedures to minimize failures caused by communication interruptions or unexpected software errors.

3.3.3 Safety

Safety is one of the primary design objectives of the proposed robotic dog. The robot shall continuously monitor body orientation, battery voltage, processor status, and communication health during operation. Whenever abnormal conditions are detected, the control system shall automatically activate appropriate safety mechanisms.

These safety mechanisms include:

- Emergency stop during severe instability
- Servo shutdown during overload conditions
- Low battery warning
- Automatic recovery after communication failure
- Over-current protection
- Thermal monitoring of critical components

These features protect both the robotic platform and nearby users during autonomous operation.

3.3.4 Scalability

The robotic dog shall employ a modular hardware and software architecture capable of supporting future upgrades without requiring complete redesign. Additional sensors, actuators, communication modules, robotic manipulators, GPS receivers, LiDAR, and cloud-based artificial intelligence services shall be integrated through standardized communication interfaces such as UART, I²C, SPI, USB, and ROS2 middleware.

3.3.5 Maintainability

The robot shall be designed using modular mechanical assemblies and standardized electronic components to simplify maintenance and component replacement. Individual subsystems, including servo motors, controllers, sensors, and power modules, shall be accessible without requiring complete disassembly of the robot.

The software architecture shall also remain modular, allowing independent modification of perception, navigation, locomotion, and communication modules.

3.3.6 Energy Efficiency

Efficient energy utilization is essential for extending the robot's operating time. The embedded controllers shall minimize unnecessary processor utilization while the reinforcement learning controller shall generate energy-efficient locomotion patterns.

The power management subsystem shall distribute power independently to the servo motors and electronic subsystems to reduce electrical interference and improve battery utilization.

3.3.7 Real-Time Operation

The robotic dog shall perform all control operations within deterministic timing constraints. Servo control signals shall be generated at fixed update intervals while sensor information shall be processed continuously to maintain accurate perception and balance.

Computer vision algorithms shall execute at sufficient frame rates to support real-time object detection, human tracking, and obstacle avoidance without introducing noticeable delays in robot movement.

3.3.8 Usability

The robotic dog shall provide an intuitive user interface that allows operators to interact with the robot through offline voice commands. The OLED display shall present system status information including battery level, locomotion mode, communication status, and operating conditions.

The robot shall require minimal user intervention during autonomous operation while providing straightforward procedures for calibration, startup, and shutdown.

3.3.9 Portability

The robotic platform shall be compact and lightweight enough for transportation between laboratories, classrooms, competition venues, and testing environments. The overall mechanical structure shall provide sufficient strength while maintaining a favorable weight-to-performance ratio.

3.3.10 Future Expandability

The proposed robotic dog shall serve as a long-term research platform capable of supporting future developments including:

- SLAM-based autonomous mapping
- Outdoor GPS navigation
- Robotic arm integration

- Autonomous charging
- Gesture recognition
- Facial recognition
- Cloud robotics
- Conversational AI
- Multi-robot collaboration
- Swarm robotics

The existing architecture shall support these extensions with minimal hardware modification.

3.4 Overall System Architecture

The proposed robotic dog follows a hierarchical, distributed system architecture. By separating high-level perception and artificial intelligence from low-level, deterministic motion control, the system ensures real-time operational reliability, structural scalability, and high computational efficiency.

1. Mechanical Subsystem

The mechanical framework forms the physical chassis and four articulated legs of the robot. The locomotive structure utilizes a 12-Degree-of-Freedom (12-DOF) configuration, where each leg contains three independently actuated joints (hip roll, hip pitch, and knee pitch) powered by twelve MG996R high-torque, metal-g geared servo motors. Two auxiliary MG90S micro-servos actuate the head and tail assemblies.

2. Embedded Control Subsystem

The embedded control tier handles hard real-time execution. An ESP32 microcontroller acts as the core low-level processing hub, managing IMU data acquisition, closed-loop stance updates, and communication protocols. A single PCA9685 16-channel driver module offloads hardware PWM generation from the ESP32 over an I²C bus, synchronizing positional updates across 14 active channels to eliminate locomotion stuttering.

3. Perception Subsystem

The perception subsystem generates a comprehensive map of the robot's environment through a multi-modal sensor network:

- RGB Camera: Captures environmental frames for object detection, obstacle mapping, and target identification via a lightweight YOLOv8 deep learning architecture.
- GY-87 IMU: Constantly measures angular velocity, 3-axis acceleration, and body tilt angle to drive the low-level balance correction engine.
- VL53L0X Time-of-Flight (ToF): Delivers precise short-range distance metrics for front-facing obstacle avoidance.

4. Artificial Intelligence Subsystem

The NVIDIA Jetson Orin Nano serves as the primary high-level decision-making unit. It executes real-time vision processing, target-following algorithms, and edge-based inference of the neural network locomotion policy. The locomotion policy itself is trained offline using the Proximal Policy Optimization (PPO) reinforcement learning algorithm in a physics simulator, allowing the Jetson to rapidly stream target joint angles to the ESP32 over a high-speed UART connection without computational lag.

5. Power Management and Safety Subsystem

Power is delivered by a high-discharge 3-cell Lithium Polymer (LiPo) battery pack governed by an integrated 3S Battery Management System (BMS) to provide vital under-voltage protection, cell balancing, and overcurrent monitoring. High-amperage DC-DC buck converters step down voltage to supply separate, fully isolated power rails. This isolation keeps high-current servo surge loads completely segregated from the sensitive digital logic of the ESP32 and Jetson Orin Nano, preventing communication corruption or voltage brownouts.

System Communication Architecture

Data flow across the platform relies on standardized, low-latency interfaces:

Interface Type	Connected Subsystems / Devices	Operational Role
UART (Serial)	Jetson Orin Nano $\longleftrightarrow$ ESP32	High-speed command and state-vector streaming
I ² C Bus	ESP32 $\rightarrow$ PCA9685, GY-87, VL53L0X	Fast Mode (400 kHz) sensor polling and driver indexing
Hardware PWM	PCA9685 $\rightarrow$ 14x Servo Motors	Coordinated, synchronous joint actuation

3.5 Working Principle of the Proposed Robotic Dog

The proposed robotic dog operates through the coordinated interaction of multiple hardware and software subsystems, including embedded controllers, artificial intelligence algorithms, sensors, actuators, and communication interfaces. The overall working principle follows a hierarchical control architecture in which the **NVIDIA Jetson Orin Nano** performs high-level perception, decision-making, and artificial intelligence processing, while the **ESP32 microcontroller** executes deterministic real-time control of the servo motors and peripheral devices. Continuous communication between these processing units enables the robot to perceive its surroundings, interpret user commands, make intelligent decisions, and execute coordinated locomotion while maintaining stability across different operating conditions.

Step 1: System Initialization

The operation of the robotic dog begins immediately after power is supplied through the 3-cell Lithium Polymer (LiPo) battery. The power management subsystem distributes regulated voltages to the ESP32 microcontroller, NVIDIA Jetson Orin Nano, servo motors, sensors, OLED display, and communication modules through dedicated DC-DC buck converters. During startup, both processing units perform hardware initialization by verifying communication interfaces, configuring peripheral devices, and performing internal self-diagnostic checks. The ESP32 initializes the PWM controllers, servo drivers, IMU, Time-of-Flight sensor, and OLED display, while the Jetson Orin Nano loads the operating system, artificial intelligence models, reinforcement learning policy, computer vision algorithms, and speech recognition framework.

Once initialization is complete, all servo motors are gradually moved to their predefined neutral positions to ensure a stable standing posture. Sensor calibration procedures are simultaneously executed to establish reference values for body orientation, acceleration, and environmental measurements before locomotion begins.

Step 2: Environmental Perception

After initialization, the perception subsystem continuously monitors the surrounding environment using multiple onboard sensors. The RGB camera captures real-time image frames that are processed by the YOLOv8 object detection model to identify obstacles, humans, and other relevant objects. Simultaneously, the VL53L0X Time-of-Flight sensor

measures the distance to nearby obstacles, while the GY-87 Inertial Measurement Unit continuously provides body orientation, angular velocity, and acceleration data.

Information obtained from these sensors is fused using an Extended Kalman Filter (EKF), generating a reliable estimate of the robot's orientation, position, and surrounding environment. This continuous perception process enables the robot to understand changes in terrain, detect obstacles, recognize users, and maintain balance throughout operation.

Step 3: Voice Command Processing

The robotic dog supports offline voice interaction through an onboard I2S digital microphone connected to the NVIDIA Jetson Orin Nano. During normal operation, the microphone continuously monitors incoming audio while executing wake-word detection to minimize unnecessary computational load. Once the predefined wake word is detected, the embedded speech recognition system converts spoken commands into text using an offline speech recognition engine.

The recognized command is analyzed by the command interpretation module, which determines the user's intended action. Supported commands include **"stand," "sit," "walk," "run," "stop," "follow me," "turn left," "turn right," "come here,"** and **"climb stairs."** The corresponding high-level action is then transmitted to the locomotion controller for execution without requiring internet connectivity, thereby reducing response latency and improving operational reliability.

Step 4: Artificial Intelligence Decision-Making

After receiving environmental information and user commands, the NVIDIA Jetson Orin Nano performs high-level decision-making using multiple artificial intelligence algorithms. Reinforcement Learning based on the **Proximal Policy Optimization (PPO)** algorithm determines the most appropriate locomotion strategy according to the robot's current body posture, terrain characteristics, target velocity, and environmental conditions. Simultaneously, computer vision algorithms identify obstacles, track designated users, and estimate stair geometry when stair-climbing mode is activated.

Based on the combined outputs of perception, reinforcement learning, and navigation algorithms, the Jetson generates desired joint positions and locomotion commands that are transmitted to the ESP32 microcontroller through a high-speed UART communication interface.

Step 5: Locomotion Execution

The ESP32 microcontroller receives high-level motion commands from the Jetson Orin Nano and converts them into synchronized servo movements. Two PCA9685 PWM driver modules generate precise Pulse Width Modulation signals for all sixteen MG996R servo motors

controlling the four legs, together with the MG90S servos responsible for head and tail articulation.

Each walking cycle is executed through coordinated joint movement, ensuring smooth transitions between stance and swing phases. Throughout locomotion, the ESP32 continuously monitors body orientation using IMU feedback and applies corrective adjustments whenever deviations from the desired posture are detected. This closed-loop control architecture enables stable locomotion across smooth surfaces, rough trails, gravel roads, grass, and inclined terrain.

Step 6: Obstacle Detection and Avoidance

During locomotion, the robot continuously evaluates its surroundings for potential obstacles. When the perception subsystem identifies an obstacle within the predefined safety distance, the navigation algorithm generates an alternative collision-free path. Depending on the environment, the robot may slow down, stop, turn, or bypass the obstacle before resuming normal locomotion. Continuous sensor updates ensure safe operation even in dynamic environments containing moving objects or pedestrians.

Step 7: Human Following

When the **"Follow Me"** mode is activated, the vision system identifies the designated person using YOLOv8 object detection and tracking algorithms. The robot continuously estimates the person's position relative to its own coordinate frame and adjusts its walking direction to maintain a safe following distance. If the person temporarily disappears from the camera's field of view, the tracking system attempts re-identification before resuming autonomous following behavior.

Step 8: Stair Climbing

Upon detecting a staircase, the perception subsystem estimates the dimensions of individual steps and verifies that the staircase falls within the robot's operational capability of **5cm to 8cm** step height. The reinforcement learning controller then generates adaptive joint trajectories for climbing.

The climbing sequence begins by positioning the front legs onto the first step while the rear legs stabilize the robot on the lower surface. Body orientation is continuously monitored using IMU feedback to compensate for changes in inclination. The robot gradually transfers its weight forward before lifting the rear legs to complete the climbing cycle. This process is repeated for each step until the staircase has been successfully traversed. A similar strategy is

employed for descending stairs while ensuring safe foot placement and controlled body motion.

Step 9: Continuous Safety Monitoring

Throughout operation, the robot continuously monitors battery voltage, processor status, communication links, servo temperature, and body orientation. Whenever unsafe operating conditions are detected, the control system immediately activates appropriate protection mechanisms such as emergency stop, servo deactivation, reduced-speed locomotion, or safe shutdown procedures. These safety mechanisms protect both the robotic platform and nearby users while ensuring reliable long-term operation.

Step 10: System Shutdown

When the user issues a shutdown command or the battery level reaches the minimum safe operating threshold, the robot gradually returns all servo motors to a neutral standing position before disabling power to the actuators. The Jetson Orin Nano safely terminates all artificial intelligence processes, stores operational logs if required, and performs an orderly operating system shutdown. Finally, the embedded controller disconnects all non-essential subsystems, completing the shutdown sequence while preventing abrupt power loss or hardware damage.

Overall, the proposed robotic dog follows a fully autonomous operating cycle in which environmental perception, artificial intelligence, embedded control, and adaptive locomotion operate simultaneously to provide reliable navigation, human interaction, obstacle avoidance, rough-terrain traversal, and autonomous stair climbing. The integration of distributed computing, reinforcement learning, and real-time embedded control ensures high performance while maintaining safety, scalability, and operational efficiency.

3.6 Hardware Architecture

The hardware architecture of the proposed robotic dog has been designed using a modular and distributed approach that combines high-performance artificial intelligence computing with deterministic embedded control. The system integrates mechanical actuators, embedded controllers, intelligent sensors, communication interfaces, and power management circuits into a unified robotic platform capable of autonomous locomotion, environmental perception, human-robot interaction, and adaptive decision-making. The modular design philosophy allows each subsystem to operate independently while maintaining seamless communication with the remaining hardware components, thereby improving scalability, maintainability, and overall system reliability.

The overall hardware architecture consists of six major subsystems:

- Processing and Control Unit

- Locomotion System
- Perception System
- Human–Robot Interaction System
- Communication System
- Power Management System

Each subsystem is discussed in detail below.

3.6.1 Processing and Control Unit

The robotic dog uses two main computers working together: an ESP32 microcontroller and an NVIDIA Jetson Orin Nano. This dual-controller setup separates easy, fast tasks from heavy AI tasks. This prevents the robot from freezing or lagging while walking.

Low-Level Controller (ESP32)

The ESP32 is the robot's nervous system. It handles fast, repetitive tasks that need perfect timing, such as:

- Sending quick movement updates to the leg motors.
- Reading data from the balance sensor (IMU) and distance sensor (ToF).
- Controlling the small OLED display screen and monitoring battery safety.

The ESP32 has a dual-core processor running at 240 MHz. This speed allows it to make instant balancing adjustments in microseconds so the dog does not fall over. It links directly to the motor driver board using an I²C connection.

High-Level Computing Platform (NVIDIA Jetson Orin Nano)

The NVIDIA Jetson Orin Nano is the robot's brain. It handles heavy artificial intelligence and computing tasks, including:

- Processing camera video using the YOLOv8 model to see obstacles and follow humans.
- Listening to and understanding voice commands without using the internet.
- Running the smart Reinforcement Learning walking rules.

The Jetson contains a powerful processor and an NVIDIA graphics card (GPU). This hardware gives it the power to run complex AI models smoothly while saving battery life.

Inter-Processor Communication

The ESP32 and Jetson Orin Nano talk to each other using a simple, high-speed wired connection called UART (Serial).

The communication works like a two-way street: the ESP32 constantly sends balance and sensor data up to the Jetson brain. The Jetson processes this data through its AI models and sends motor angle commands back down to the ESP32 for the legs to execute.

3.6.2 Locomotion System

The locomotion subsystem handles the physical movement of the robotic dog through twelve independently controlled servo motors arranged across four legs.

Each leg has three degrees of freedom (3-DOF) consisting of:

- Hip Roll Joint
- Hip Pitch Joint
- Knee Pitch Joint

This setup gives the robot a total of 12 Degrees of Freedom (12-DOF) for the legs, which allows for stable walking, balancing, and adaptive movement over different surfaces.

These 12 primary joints are driven by **TowerPro MG996R high-torque metal gear servo motors**. These servos were chosen because their heavy-duty metal gears and high torque output provide the physical strength needed for the robot to stand, walk, balance, and step over low obstacles without stripping internal components.

Two smaller, lightweight **MG90S micro servo motors** are used to control the head and tail movements for camera placement and basic user interaction.

To manage all 14 motors, a single **PCA9685 16-channel PWM controller** board is used. This board connects to the ESP32 and takes over the job of generating precise, constant timing signals for the motors. Offloading this task to a single driver board keeps the main microcontroller from lagging and ensures all joints move together smoothly and in perfect sync.

3.6.3 Perception System

Environmental perception is achieved through the integration of multiple complementary sensors capable of observing both the robot's internal state and its surrounding environment.

The **GY-87 Inertial Measurement Unit (IMU)** continuously measures three-axis acceleration, three-axis angular velocity, magnetic heading, atmospheric pressure, and temperature. The accelerometer and gyroscope provide real-time body orientation and motion information required for balance control, while the magnetometer improves heading estimation during navigation.

Obstacle detection is performed using the **VL53L0X Time-of-Flight (ToF) laser distance sensor**, which measures distances of up to approximately **2 meters** with millimeter-level precision. The sensor enables rapid detection of nearby obstacles, wall edges, and staircase boundaries while supporting autonomous navigation in confined environments.

Visual perception is provided by an RGB camera mounted on the robot's head. Image data acquired by the camera is processed using the YOLOv8 deep learning model running on the Jetson Orin Nano to detect humans, obstacles, staircases, furniture, and other relevant objects. The camera also supports person-following, object recognition, and future gesture recognition applications.

Future versions of the robotic dog will integrate a **360-degree LiDAR sensor** to improve mapping, localization, obstacle avoidance, and autonomous navigation within larger indoor and outdoor environments.

3.6.4 Human–Robot Interaction System

The proposed robotic dog has been designed to interact naturally with users through both visual and auditory interfaces.

An **I2S digital microphone module** captures voice commands from nearby users. The microphone provides high-quality digital audio that is processed locally by the NVIDIA Jetson Orin Nano using offline speech recognition algorithms. This architecture eliminates dependence on cloud services, ensuring low response latency, increased privacy, and reliable operation in environments without internet connectivity.

The robot also incorporates a **0.96-inch OLED display** mounted on the head assembly. The display presents operational information including battery level, locomotion mode, communication status, system health, sensor values, and expressive eye animations that improve human–robot interaction.

Capacitive touch sensors positioned near the display provide an additional user interface for menu navigation, operating mode selection, and manual system configuration.

3.6.5 Communication System

Reliable communication between hardware components is essential for synchronized robot operation.

The robotic dog employs multiple communication protocols, including:

- **UART** communication between the ESP32 and NVIDIA Jetson Orin Nano.
- **I²C** communication with the PCA9685 PWM controllers, GY-87 IMU, and VL53L0X ToF sensor.
- **SPI** communication with the OLED display for rapid graphical updates.
- **PWM** signals generated by the PCA9685 modules to control all eighteen servo motors.
- **USB** interfaces supporting future camera and LiDAR integration.

The use of standardized communication interfaces simplifies hardware integration while enabling future expansion of the robotic platform.

3.6.6 Power Management System

The robotic dog is powered using a **3-cell (11.1 V) Lithium Polymer (LiPo) battery**, selected for its high energy density, lightweight construction, and ability to supply the peak current demanded by multiple servo motors during locomotion.

To ensure stable system operation, the power distribution network employs dedicated **200 W, 20 A DC–DC buck converters** that regulate battery voltage according to subsystem requirements. One converter supplies the servo motors with a stable **5.5–6 V** power rail, while a second converter powers the ESP32, sensors, OLED display, and communication modules.

The NVIDIA Jetson Orin Nano receives an independent regulated power supply to prevent voltage fluctuations caused by high-current servo operation. Electrical isolation between logic and actuator power rails minimizes electromagnetic interference and significantly improves system stability.

Battery voltage, current consumption, and thermal conditions are continuously monitored by the embedded controller. If abnormal operating conditions are detected, the robot

automatically activates protective mechanisms such as low-battery warnings, reduced locomotion speed, or emergency shutdown.

Table 3.3 Hardware Components Used in the Proposed Robotic Dog

Component	Quantity	Function
NVIDIA Jetson Orin Nano	1	Artificial Intelligence & Computer Vision
ESP32CEU6	1	Real-Time Embedded Control
MG996R Servo Motor	12	Leg Actuation
MG90S Servo Motor	2	Head & Tail Movement
PCA9685 PWM Driver	2	Servo Signal Generation
GY-87 IMU	1	Balance & Orientation
VL53L0X ToF Sensor	1	Obstacle Detection
I2S Digital Microphone	1	Offline Voice Commands
OLED Display	1	System Status Display
LiPo Battery (11.1 V)	1	Power Source
DC–DC Buck Converter	2	Voltage Regulation
RGB Camera	1	Object Detection & Human Following
LiDAR (<i>Future Upgrade</i>)	1	Mapping & Navigation

3.7 Software Architecture

The software architecture of the proposed robotic dog has been designed using a modular, layered, and distributed approach to ensure scalability, maintainability, and efficient execution of both real-time control and computationally intensive artificial intelligence algorithms. The architecture separates high-level decision-making from low-level hardware control, allowing each subsystem to operate independently while communicating through standardized interfaces. This design improves overall system reliability and simplifies future software expansion.

The software system is divided into two major computational layers. The **high-level software layer**, executed on the NVIDIA Jetson Orin Nano, performs artificial intelligence, perception, navigation, and decision-making. The **low-level embedded software layer**, executed on the ESP32 microcontroller, performs deterministic servo control, sensor acquisition, actuator synchronization, and hardware safety management. Continuous communication between

these layers enables intelligent autonomous operation while maintaining strict real-time constraints required for stable quadruped locomotion.

The software architecture consists of the following major layers:

- Operating System Layer
- Middleware Layer
- Artificial Intelligence Layer
- Motion Planning Layer
- Embedded Control Layer
- Hardware Abstraction Layer

Each layer is described below.

3.7.1 Operating System Layer

The NVIDIA Jetson Orin Nano operates on **Ubuntu Linux**, which provides a stable environment for executing robotics software, artificial intelligence frameworks, and hardware drivers. Ubuntu supports GPU acceleration through NVIDIA JetPack, CUDA, cuDNN, TensorRT, and OpenCV, enabling efficient execution of deep learning models required for perception and navigation.

The operating system is responsible for process scheduling, memory management, hardware resource allocation, network communication, and file management while supporting simultaneous execution of multiple robotics applications.

3.7.2 Middleware Layer

The proposed robotic dog employs **Robot Operating System 2 (ROS2)** as the primary middleware platform for communication between software modules. ROS2 provides a distributed software framework that organizes the robot into independent nodes capable of exchanging information using publishers, subscribers, services, and actions.

Each major subsystem, including object detection, reinforcement learning, voice recognition, navigation, and locomotion control, operates as an independent ROS2 node. This modular architecture enables individual software components to be developed, tested, and upgraded independently without affecting the remaining system.

ROS2 also provides:

- Real-time communication
 - Topic-based message exchange
 - Service-based communication
 - Action servers for long-duration tasks
 - Parameter management
 - Logging and diagnostics
 - Hardware abstraction
 - Future multi-robot communication
-

3.7.3 Artificial Intelligence Layer

The Artificial Intelligence Layer represents the primary decision-making component of the robotic dog and executes entirely on the NVIDIA Jetson Orin Nano.

This layer consists of several intelligent software modules working simultaneously.

Object Detection

The onboard RGB camera continuously captures image frames that are processed using the **YOLOv8** deep learning model

YOLOv8 detects:

- Humans
- Obstacles
- Furniture
- Staircases
- Walls
- Custom trained objects

The detected objects are transmitted to the navigation system for path planning.

Reinforcement Learning Module

Adaptive locomotion is achieved using the **Proximal Policy Optimization (PPO)** reinforcement learning algorithm.

The RL model receives information including:

- Joint positions
- Body orientation
- Angular velocity
- Terrain information
- Previous servo actions

The trained policy generates optimized joint trajectories that maximize locomotion stability while minimizing energy consumption.

The reinforcement learning policy continuously adapts locomotion according to changing terrain conditions, enabling the robot to traverse rough trails, gravel roads, grass, and staircases.

Speech Recognition Module

Offline speech recognition enables natural interaction between the robot and users.

The speech recognition pipeline consists of:

- Wake-word detection
- Audio preprocessing
- Speech-to-text conversion
- Command interpretation
- Motion command generation

Supported commands include:

- Stand
- Sit
- Walk
- Run
- Stop
- Turn Left
- Turn Right
- Follow Me
- Come Here
- Climb Stairs

Since all processing occurs locally on the Jetson Orin Nano, the robot operates independently of cloud services and internet connectivity.

3.7.4 Motion Planning Layer

The Motion Planning Layer converts high-level decisions generated by artificial intelligence into executable locomotion commands.

This layer consists of:

- Gait Planner
- Trajectory Generator
- Inverse Kinematics Solver
- Balance Controller
- Obstacle Avoidance Planner
- Stair Climbing Planner

The gait planner determines the walking mode according to the desired robot speed and operating conditions.

Supported gait modes include:

- Standing
- Walking
- Trotting
- Running

The trajectory generator calculates smooth foot trajectories while the inverse kinematics solver computes the corresponding joint angles for all sixteen servo motors.

During stair climbing, this layer modifies foot trajectories according to detected stair dimensions while maintaining robot stability.

3.7.5 Embedded Control Layer

The Embedded Control Layer executes on the ESP32 microcontroller.

Its primary responsibilities include:

- Servo position control
- PWM generation
- Sensor acquisition
- OLED display management
- Touch sensor processing

- Battery monitoring
- UART communication
- Safety supervision

Servo commands received from the Jetson Orin Nano are translated into synchronized PWM signals through the PCA9685 controllers.

The ESP32 continuously monitors sensor feedback and applies corrective adjustments whenever deviations from the desired body posture are detected.

3.7.6 Hardware Abstraction Layer

The Hardware Abstraction Layer (HAL) provides standardized software interfaces for all hardware peripherals.

The HAL includes drivers for:

- UART
- I²C
- SPI
- PWM
- GPIO
- Timers
- ADC
- Interrupt Controllers

Using a hardware abstraction layer simplifies software portability and reduces dependency on specific hardware implementations.

Future hardware upgrades can therefore be integrated with minimal software modifications.

3.7.7 Software Execution Sequence

The overall software execution process follows the sequence below:

1. System Boot
2. Hardware Initialization
3. Sensor Initialization
4. ROS2 Node Startup

5. Camera Initialization
6. Speech Recognition Initialization
7. Reinforcement Learning Model Loading
8. Sensor Calibration
9. Continuous Environment Monitoring
10. Artificial Intelligence Processing
11. Motion Planning
12. Servo Command Generation
13. Real-Time Locomotion
14. Safety Monitoring
15. System Shutdown

Each stage continuously exchanges information with adjacent software layers to ensure synchronized robot operation.

Table 3.4 Software Modules

Software Module	Function
Ubuntu Linux	Operating System
ROS2	Middleware Communication
YOLOv8	Object Detection
PPO	Reinforcement Learning
OpenCV	Image Processing
Whisper/Vosk	Offline Speech Recognition
EKF	Sensor Fusion
Motion Planner	Walking & Navigation
Inverse Kinematics	Joint Angle Calculation
ESP32 Firmware	Real-Time Embedded Control
FreeRTOS (<i>Optional Future Upgrade</i>)	Task Scheduling
HAL Drivers	Hardware Interface

3.8 Communication Architecture

The communication architecture of the proposed robotic dog has been designed to provide reliable, real-time, and efficient data exchange between the various embedded controllers,

sensors, actuators, and artificial intelligence modules. Since the robotic platform integrates multiple processing units and peripheral devices, a robust communication framework is essential to ensure synchronized operation and deterministic system behavior.

The proposed robotic dog adopts a **hierarchical communication architecture**, where the **NVIDIA Jetson Orin Nano** functions as the high-level processing unit responsible for artificial intelligence, computer vision, voice recognition, and navigation, while the **ESP32 microcontroller** acts as the low-level real-time controller responsible for servo actuation, sensor interfacing, and hardware management. Communication between these two processing units and all peripheral devices is achieved using standardized serial communication protocols including **UART**, **I²C**, **SPI**, **PWM**, and **USB**.

The communication architecture is designed to minimize latency, reduce processor overhead, and maintain reliable operation during autonomous locomotion. Each protocol has been selected according to the bandwidth, timing, and reliability requirements of the corresponding hardware component.

3.8.1 UART Communication

Universal Asynchronous Receiver-Transmitter (UART) serves as the primary communication interface between the **NVIDIA Jetson Orin Nano** and the **ESP32** microcontroller. This communication channel enables continuous bidirectional data exchange between the artificial intelligence subsystem and the embedded control subsystem.

The Jetson Orin Nano transmits:

- Desired walking speed
- Gait selection
- Joint trajectory commands
- Obstacle avoidance instructions
- Human-following commands
- Voice command execution requests
- Stair-climbing commands
- System configuration parameters

The ESP32 returns:

- IMU measurements
- Servo positions

- Battery voltage
- Current consumption
- System health status
- Error messages
- Motion execution status

To ensure reliable communication, data packets include start bytes, packet identifiers, payload length, checksum values, and end markers for error detection. This protocol minimizes the risk of corrupted or incomplete data during real-time robot operation.

3.8.2 I²C Communication

The **Inter-Integrated Circuit (I²C)** protocol is employed for communication between the ESP32 microcontroller and multiple peripheral devices. I²C enables several devices to share a common two-wire bus consisting of a Serial Data Line (SDA) and a Serial Clock Line (SCL), reducing wiring complexity and simplifying hardware integration.

The following components communicate via the I²C bus:

- PCA9685 PWM Driver (Servo Control)
- GY-87 IMU
- VL53L0X Time-of-Flight Sensor
- OLED Display
- Future environmental sensors

The ESP32 functions as the I²C master, while the connected peripherals operate as slave devices with unique hardware addresses. This architecture allows efficient sequential communication with multiple sensors while minimizing the number of required microcontroller pins.

3.8.3 PWM Communication

Pulse Width Modulation (PWM) signals are responsible for controlling all servo motors used in the robotic dog. Instead of generating PWM signals directly through the ESP32, the proposed system utilizes two **PCA9685 16-channel PWM controllers**, each capable of producing high-resolution PWM outputs independently.

The ESP32 sends desired joint positions to the PCA9685 modules through the I²C interface. The PCA9685 then converts these commands into stable PWM signals that drive:

- Sixteen MG996R servo motors controlling the legs.
- Two MG90S servo motors controlling the head and tail.

Offloading PWM generation to dedicated hardware reduces processor workload, improves timing accuracy, and ensures synchronized movement of all actuators during locomotion.

3.8.4 SPI Communication

The **Serial Peripheral Interface (SPI)** protocol is used for high-speed communication with display modules and future high-bandwidth peripheral devices. Compared to I²C, SPI provides higher communication speeds by using separate data lines for transmission and reception.

In the proposed robotic dog, SPI is primarily used for:

- OLED display updates
- High-speed graphical rendering
- Future memory modules
- Future wireless communication interfaces

SPI enables rapid display updates while minimizing latency during continuous system monitoring.

3.8.5 USB Communication

The **Universal Serial Bus (USB)** interface connects high-bandwidth peripherals directly to the NVIDIA Jetson Orin Nano. USB provides sufficient bandwidth for image acquisition and future sensing devices that require continuous data streaming.

USB devices include:

- RGB Camera
- Future LiDAR
- External storage devices
- Development interfaces
- Wireless communication adapters

The USB interface allows plug-and-play hardware expansion without requiring significant software modifications.

3.8.6 ROS2 Communication

Within the NVIDIA Jetson Orin Nano, software modules communicate using **ROS2** topics, services, and actions. Instead of exchanging information through direct function calls, each subsystem operates as an independent ROS2 node.

Typical communication includes:

Publisher Node	Subscriber Node	Purpose
Camera Node	YOLOv8 Node	Image Processing
YOLOv8 Node	Navigation Node	Object Detection
Navigation Node	Motion Planner	Path Planning
Voice Recognition	Command Manager	Command Execution
PPO Controller	Motion Planner	Gait Generation
Motion Planner	UART Node	Send Motion Commands

This modular communication framework allows independent software development while improving maintainability and fault isolation.

3.8.7 Communication Workflow

The communication sequence begins when the perception subsystem acquires environmental information through the camera, IMU, and ToF sensor. Sensor data is transmitted to the Jetson Orin Nano, where artificial intelligence algorithms analyze the environment, detect objects, identify users, and generate locomotion decisions.

The Jetson communicates the required joint trajectories and motion commands to the ESP32 through UART. The ESP32 processes these commands, updates the servo controllers through the I²C bus, and generates synchronized PWM signals using the PCA9685 modules.

During locomotion, the ESP32 continuously transmits feedback regarding body orientation, battery status, servo positions, and system health back to the Jetson. This closed-loop

communication architecture enables real-time adaptation to environmental changes while maintaining stable robot operation.

3.8.8 Communication Security and Fault Handling

Reliable communication is essential for safe autonomous operation. Therefore, the communication architecture incorporates several fault detection and recovery mechanisms:

- Packet checksum verification
- Communication timeout detection
- Automatic retransmission of corrupted packets
- Watchdog timer monitoring
- Sensor validation before execution
- Emergency stop during communication failure
- Error logging and diagnostic reporting

These mechanisms ensure that the robot continues to operate safely even in the presence of temporary communication disturbances.

Table 3.5 Communication Interfaces

Protocol	Connected Devices	Purpose
UART	Jetson ↔ ESP32	AI and motion command exchange
I²C	ESP32 ↔ PCA9685, IMU, ToF, OLED	Sensor and peripheral communication
PWM	PCA9685 → Servo Motors	Joint actuation
SPI	ESP32 ↔ OLED Display	High-speed display communication
USB	Jetson ↔ Camera, Future LiDAR	High-bandwidth peripherals
ROS2	Software Modules	Internal AI communication

3.9 Data Flow Architecture

The Data Flow Architecture describes the movement of information throughout the proposed robotic dog, from data acquisition by sensors to decision-making by artificial intelligence algorithms and finally to physical actuation through the servo motors. A well-defined data flow architecture is essential for ensuring efficient communication between hardware and software components while maintaining real-time system performance.

The proposed robotic dog follows a **closed-loop data processing architecture**, where environmental information is continuously collected, processed, analyzed, and used to generate control actions. Feedback from sensors is continuously incorporated into the control loop, allowing the robot to adapt its behavior according to changing environmental conditions. This architecture enables stable locomotion, autonomous navigation, obstacle avoidance, voice interaction, and adaptive balance control.

The complete data flow is organized into six major stages:

- Data Acquisition
- Data Processing
- Artificial Intelligence Decision-Making
- Motion Planning
- Actuator Control
- Feedback and System Monitoring

Each stage is explained below.

3.9.1 Data Acquisition Layer

The first stage of the data flow architecture is responsible for acquiring information from the robot's sensors and user interfaces. Multiple sensors continuously collect data describing both the robot's internal state and the surrounding environment.

The **RGB camera** captures image frames used for object detection, person tracking, and stair recognition. The **GY-87 Inertial Measurement Unit (IMU)** measures acceleration, angular velocity, and orientation to monitor the robot's posture and maintain dynamic balance. The **VL53L0X Time-of-Flight (ToF) sensor** measures the distance to nearby obstacles, assisting in collision avoidance and terrain assessment. An **I2S digital microphone** captures spoken commands for offline speech recognition, while the battery monitoring circuit continuously measures voltage and current to evaluate the robot's power status.

All sensor measurements are time-stamped and synchronized before being forwarded to the appropriate processing modules. Accurate synchronization ensures that the perception and control algorithms operate using consistent environmental information.

3.9.2 Data Processing Layer

The acquired sensor data is transmitted to the **NVIDIA Jetson Orin Nano**, where preprocessing is performed before artificial intelligence algorithms are executed.

Image frames are enhanced and resized to improve object detection accuracy, while IMU measurements are filtered to reduce noise and estimate the robot's orientation. Distance measurements obtained from the ToF sensor are validated and converted into obstacle distance information. Voice signals are filtered to remove background noise before speech recognition is performed.

The processed sensor information is then organized into structured data packets and made available to higher-level artificial intelligence modules.

3.9.3 Artificial Intelligence Decision Layer

After preprocessing, the structured sensor information is analyzed by the artificial intelligence subsystem running on the NVIDIA Jetson Orin Nano.

Several software modules execute simultaneously:

YOLOv8 Object Detection

The camera images are analyzed using the YOLOv8 deep learning model to identify:

- Humans
- Obstacles
- Furniture
- Staircases
- Walls
- Custom-trained objects

Detected objects are assigned confidence scores and spatial coordinates before being forwarded to the navigation system.

Reinforcement Learning Controller

The PPO-based reinforcement learning model receives:

- Robot orientation
- Joint positions
- Walking velocity
- Terrain information
- Previous control actions

The model predicts the optimal joint trajectories required to maintain stable locomotion while minimizing energy consumption.

Speech Recognition

Voice commands are converted into executable robot actions.

Navigation Module

The navigation system combines outputs from object detection, reinforcement learning, and sensor fusion to determine the safest movement strategy while avoiding collisions.

3.9.4 Motion Planning Layer

The Motion Planning Layer transforms high-level decisions into executable robot movements.

This layer performs:

- Gait selection
- Foot trajectory generation
- Inverse kinematics
- Balance correction
- Turning trajectory calculation
- Stair-climbing trajectory planning

If an obstacle is detected, the planner modifies the walking path. If a staircase is identified, specialized stair-climbing trajectories are generated. During human-following mode, the planner continuously updates the walking direction according to the target person's position.

The final output of this layer consists of desired joint angles for all sixteen servo motors.

3.9.5 Embedded Control Layer

Desired joint angles generated by the motion planner are transmitted through the UART interface to the ESP32 microcontroller.

The ESP32 performs:

- Servo interpolation
- Motion synchronization
- Speed limiting
- Safety verification
- PWM command generation

Servo commands are then transmitted through the I²C bus to the PCA9685 PWM controllers, which generate precise PWM signals for all servo motors.

The MG996R servo motors execute the commanded movements, producing coordinated locomotion of the robotic dog.

3.9.6 Feedback Layer

Unlike an open-loop robotic system, the proposed robotic dog continuously monitors the consequences of every movement.

After each control cycle:

- IMU measures new body orientation.
- The camera observes environmental changes.
- ToF sensor measures obstacle distance.
- Servo positions are updated.
- Battery voltage is monitored.
- Communication status is verified.

"The commanded target positions are tracked locally by the software stack as reference inputs."

This closed-loop architecture enables:

- Dynamic balance
- Terrain adaptation
- Obstacle avoidance

- Human following
- Stair climbing
- Recovery from disturbances

3.9.8 Advantages of the Proposed Data Flow Architecture

The proposed data flow architecture offers several important advantages:

- Continuous closed-loop feedback for stable locomotion.
- Efficient separation between artificial intelligence processing and embedded control.
- Real-time processing of multiple sensor streams.
- Reliable synchronization between perception and locomotion.
- Reduced computational load on the embedded controller.
- Improved adaptability to rough terrain and environmental changes.
- Modular software design that supports future expansion.
- Scalable architecture for integrating additional sensors and intelligent algorithms.

Table 3.6 Data Flow Between System Modules

Source	Processing Module	Output
RGB Camera	YOLOv8	Objects, Humans, Stairs
IMU (GY-87)	Sensor Fusion (EKF)	Orientation & Stability
ToF Sensor	Distance Processing	Obstacle Distance
Microphone	Speech Recognition	Voice Commands
Battery Monitor	Power Management	Battery Status
YOLOv8	Navigation Module	Obstacle Information
PPO Controller	Motion Planner	Gait Commands
Motion Planner	ESP32	Joint Angles
ESP32	PCA9685	PWM Signals
PCA9685	Servo Motors	Robot Motion
Sensors	Feedback Controller	Continuous Corrections

3.10 Operational Modes of the Proposed Robotic Dog

The proposed robotic dog has been designed to operate in multiple functional modes to perform different tasks efficiently while adapting to varying environmental conditions and user requirements. Each operational mode activates a specific combination of hardware and software modules, allowing the robot to execute specialized functions such as autonomous navigation, voice-controlled movement, human following, and stair climbing. The modular software architecture enables seamless switching between these modes while maintaining system stability and safety.

The major operational modes of the robotic dog are illustrated in **Figure 3.8** and described in the following subsections.

3.10.1 Idle Mode

Idle Mode is the default operating state after system initialization. In this mode, the robot remains stationary while continuously monitoring all onboard sensors and communication interfaces.

During Idle Mode, the system performs the following operations:

- Initializes all hardware components.
- Monitors battery voltage and current.
- Continuously reads IMU data.
- Detects wake-word for voice activation.
- Maintains communication between ESP32 and Jetson.
- Displays system status on the OLED screen.
- Keeps servo motors in a stable standing position.

This mode consumes minimal power while ensuring that the robot is immediately ready to respond to user commands or autonomous tasks.

3.10.2 Manual Voice-Controlled Mode

In Manual Voice-Controlled Mode, the robot is directly controlled through offline voice commands without requiring a remote controller or internet connection.

The I2S digital microphone continuously listens for the predefined wake word. Once activated, the speech recognition module running on the NVIDIA Jetson Orin Nano interprets the spoken command and converts it into robot actions.

Supported commands include:

- Stand
- Sit
- Walk Forward
- Walk Backward
- Turn Left
- Turn Right
- Run
- Stop
- Follow Me
- Climb Stairs
- Return Home (*Future Upgrade*)

After command recognition, the motion planner generates appropriate gait trajectories, which are transmitted to the ESP32 for execution.

3.10.3 Autonomous Navigation Mode

Autonomous Navigation Mode enables the robot to move independently without direct human intervention.

In this mode, the robot continuously performs the following operations:

- Capture environmental images.
- Detects obstacles using YOLOv8 and ToF sensors.
- Estimate safe walking paths.
- Adjust gait according to terrain conditions.
- Maintain body balance using IMU feedback.
- Continuously update motion trajectories.

The robot autonomously avoids collisions while navigating through indoor or outdoor environments.

3.10.4 Rough Terrain Locomotion Mode

This mode is specifically designed for traversing challenging environments such as:

- Gravel roads
- Rocky surfaces
- Grass
- Mud paths
- Uneven trails
- Small slopes

The reinforcement learning controller continuously analyzes body posture and terrain characteristics before modifying gait parameters.

Adaptive adjustments include:

- Step height
- Step length
- Walking speed
- Body inclination
- Center of gravity

These adjustments enable stable locomotion despite irregular ground conditions.

3.10.5 Human Following Mode

In Human Following Mode, the robotic dog autonomously detects and follows a designated person.

The operational sequence consists of:

1. Capture camera images.
2. Detect the person using YOLOv8.
3. Track the person's movement.
4. Estimate relative distance.
5. Generate walking trajectory.
6. Maintain a safe following distance.
7. Avoid obstacles during movement.

If the target temporarily disappears from view, the robot attempts re-identification before resuming normal operation.

This mode is particularly useful for personal assistance, warehouse logistics, healthcare support, and collaborative robotics.

3.10.6 Stair Climbing Mode

One of the distinguishing features of the proposed robotic dog is its ability to autonomously climb staircases with heights ranging from **12 cm to 15 cm**.

The stair climbing sequence consists of:

- Detects staircase using camera and ToF sensor.
- Estimate stair height and depth.
- Verify climbability.
- Generate climbing gait.
- Lift front legs.
- Shift body weight.
- Lift rear legs.
- Repeat until the staircase is completed.

Throughout the climbing process, the IMU continuously monitors body orientation while the reinforcement learning controller compensates for any instability.

A similar procedure is employed during stair descent, ensuring controlled foot placement and minimizing the risk of slipping.

3.10.7 Obstacle Avoidance Mode

Obstacle Avoidance Mode operates continuously during robot movement.

The perception subsystem detects both static and moving obstacles using:

- Camera

- VL53L0X Time-of-Flight Sensor
- Future LiDAR integration

Depending on obstacle characteristics, the navigation algorithm may:

- Stop
- Turn
- Change walking direction
- Reduce speed
- Select an alternative path

This functionality significantly improves operational safety in dynamic environments.

3.10.8 Safety and Emergency Mode

The Safety Mode continuously supervises the robot during all operating conditions.

The ESP32 monitors:

- Battery voltage
- Servo current
- Communication status
- IMU orientation
- Processor health
- Temperature

If abnormal conditions are detected, the robot immediately activates protective measures including:

- Emergency Stop
- Servo shutdown
- Reduced walking speed
- Low battery warning
- Automatic recovery procedure
- Safe shutdown

These mechanisms protect both the robot and nearby users.

3.10.9 Future Intelligent Mode

The modular architecture allows the robot to support several advanced operational modes in future versions.

Planned capabilities include:

- Gesture-controlled operation
- Face recognition
- Autonomous charging
- GPS navigation
- SLAM-based exploration
- Cloud AI integration
- Robotic arm manipulation
- Multi-robot collaboration
- Remote monitoring through a mobile application

These future enhancements can be incorporated without significant modification to the existing hardware architecture.

Table 3.7 Operational Modes

Mode	Description	Main Technologies
Idle Mode	System monitoring and standby	ESP32, Sensors
Manual Voice-Controlled Mode	Voice-based robot control	I2S Mic, Offline Speech Recognition
Autonomous Navigation Mode	Independent movement	YOLOv8, Navigation
Rough Terrain Mode	Adaptive locomotion	PPO, IMU
Human Following Mode	Person detection and tracking	YOLOv8, Computer Vision
Stair Climbing Mode	Autonomous stair climbing (12–15 cm)	Camera, ToF, PPO
Obstacle Avoidance Mode	Collision-free navigation	ToF, YOLOv8

Safety Mode	Fault detection and emergency response	ESP32, Battery Monitoring
Future Intelligent Mode	Advanced AI features	SLAM, GPS, Cloud AI

3.11 Advantages of the Proposed Architecture

The proposed robotic dog has been designed using a modular and distributed architecture that combines advanced embedded systems, artificial intelligence, computer vision, reinforcement learning, and autonomous navigation into a single integrated platform. Unlike conventional quadruped robots that rely on a single controller or basic locomotion algorithms, the proposed system separates high-level artificial intelligence processing from low-level real-time control, thereby improving computational efficiency, reliability, and scalability. This architecture enables the robot to perform complex autonomous tasks while maintaining stable locomotion and safe operation under varying environmental conditions.

One of the primary advantages of the proposed architecture is the **dual-controller design**, consisting of the **NVIDIA Jetson Orin Nano** and the **ESP32 microcontroller**. The Jetson Orin Nano performs computationally intensive tasks such as reinforcement learning inference, YOLOv8 object detection, offline voice recognition, and autonomous navigation, whereas the ESP32 executes deterministic real-time functions including servo control, sensor acquisition, and hardware monitoring. This separation prevents delays caused by artificial intelligence computations from affecting the locomotion controller, resulting in smoother and more reliable robot movement.

The architecture also incorporates a **modular hardware design**, allowing individual subsystems such as sensors, actuators, communication modules, and processing units to be upgraded independently. This flexibility significantly reduces maintenance complexity and enables future expansion without requiring major modifications to the existing platform. Components such as LiDAR, GPS receivers, robotic manipulators, additional cameras, and wireless communication modules can be integrated using the available communication interfaces while preserving compatibility with the existing control system.

Another significant advantage is the implementation of **artificial intelligence-based adaptive locomotion**. Unlike conventional robots that depend solely on predefined gait trajectories, the proposed robotic dog utilizes the **Proximal Policy Optimization (PPO)** reinforcement learning algorithm to generate adaptive walking strategies. This enables the robot to adjust its locomotion according to terrain conditions, maintain dynamic balance, and improve walking efficiency across rough trails, gravel roads, grass, and inclined surfaces.

The integration of **computer vision** further enhances robot intelligence by enabling real-time environmental perception. The YOLOv8 deep learning model allows the robot to detect humans, obstacles, staircases, and surrounding objects with high accuracy. Combined with the Time-of-Flight sensor and the GY-87 IMU, the robot can autonomously avoid collisions, recognize users, and estimate terrain characteristics before executing movement. This capability significantly improves operational safety and autonomy.

A key feature of the proposed architecture is its support for **offline voice recognition**. Unlike many commercial systems that rely on cloud-based speech processing, all speech recognition and command interpretation are executed locally on the NVIDIA Jetson Orin Nano. This eliminates dependence on internet connectivity, reduces response latency, enhances user privacy, and allows reliable operation in remote or communication-limited environments.

The architecture has also been specifically designed to support **rough-terrain locomotion and autonomous stair climbing**. Continuous feedback from the IMU, reinforcement learning controller, and perception system enables the robot to maintain body stability while traversing uneven surfaces and climbing staircases with heights between **12 cm and 15 cm**. This capability considerably expands the range of environments in which the robot can operate compared with conventional wheeled robots or basic quadruped platforms.

From a software perspective, the use of **ROS2** provides a highly modular framework that organizes perception, navigation, locomotion, and communication into independent software nodes. This improves maintainability, simplifies debugging, and enables future integration of advanced algorithms such as Simultaneous Localization and Mapping (SLAM), autonomous exploration, multi-robot collaboration, and cloud-based robotic services.

The proposed architecture also emphasizes **system safety and reliability**. Dedicated safety mechanisms continuously monitor battery voltage, communication status, servo operation, processor health, and body orientation. In the event of abnormal operating conditions, the robot can automatically activate emergency stop procedures, reduce locomotion speed, or safely shut down the system to prevent hardware damage and ensure user safety.

Finally, the proposed robotic dog has been developed as a **cost-effective research platform**. By combining commercially available components with advanced artificial intelligence techniques, the system provides many capabilities found in expensive commercial quadruped robots while remaining accessible for universities, research laboratories, and engineering students. The modular design ensures that the platform can evolve with future technological advancements, making it suitable for long-term research and educational applications.

Table 3.8 Advantages of the Proposed Architecture

Feature	Conventional Quadruped Robots	Proposed Robotic Dog
Processing Architecture	Single controller	Dual-controller (Jetson Orin Nano + ESP32)
Artificial Intelligence	Limited or cloud-based	Local AI processing on Jetson Orin Nano
Locomotion	Predefined gait patterns	PPO-based adaptive locomotion
Terrain Adaptation	Mostly flat surfaces	Rough trails, grass, gravel, slopes
Stair Climbing	Limited capability	Autonomous climbing of 12–15 cm stairs
Object Detection	Basic sensing	YOLOv8 real-time object detection
Voice Interaction	Cloud-dependent or unavailable	Offline voice recognition
Navigation	Basic obstacle avoidance	Intelligent autonomous navigation
Expandability	Limited	Highly modular and scalable
Cost	High	Research-oriented and cost-effective
Communication	Limited interfaces	UART, I ² C, SPI, USB, PWM, ROS2
Future Upgrades	Difficult	LiDAR, GPS, robotic arm, SLAM, cloud AI

3.12 Chapter Summary

This chapter presented the complete system design and architecture of the proposed intelligent quadruped robotic dog. The overall design integrates mechanical engineering, embedded systems, artificial intelligence, computer vision, reinforcement learning, and autonomous navigation into a unified and modular robotic platform capable of performing complex tasks in both indoor and outdoor environments.

The chapter began by introducing the overall system architecture and defining the functional and non-functional requirements of the robotic dog. These requirements established the operational objectives of the system, including autonomous locomotion, rough-terrain traversal, stair climbing, human following, offline voice interaction, obstacle avoidance, and

real-time environmental perception. The requirements also specified the expected performance, reliability, safety, scalability, and maintainability necessary for successful operation.

A distributed hardware architecture based on the **ESP32 microcontroller** and the **NVIDIA Jetson Orin Nano** was presented to separate deterministic real-time control from computationally intensive artificial intelligence processing. The hardware architecture included the locomotion system, perception subsystem, communication interfaces, and power management system, demonstrating how each component contributes to the overall functionality of the robotic platform.

The software architecture was subsequently described as a layered and modular framework consisting of the operating system, ROS2 middleware, artificial intelligence modules, motion planning algorithms, embedded firmware, and hardware abstraction layer. The integration of **YOLOv8** for object detection, **Proximal Policy Optimization (PPO)** for adaptive locomotion, offline speech recognition for voice interaction, and sensor fusion for balance estimation enables the robot to make intelligent decisions while maintaining stable and efficient movement.

The communication architecture illustrated the use of standard communication protocols including **UART, I2C, SPI, PWM, USB, and ROS2**, ensuring reliable data exchange between processing units, sensors, actuators, and software modules. A closed-loop data flow architecture was also presented, explaining how environmental information is acquired, processed, analyzed, and converted into coordinated robot motion while continuously incorporating feedback for dynamic balance and adaptive control.

The operational modes of the robotic dog were discussed in detail, demonstrating the versatility of the proposed system. The robot supports multiple operating modes including idle monitoring, offline voice-controlled operation, autonomous navigation, rough-terrain locomotion, human following, autonomous obstacle avoidance, and stair climbing for stair heights ranging from **12 cm to 15 cm**. Continuous safety monitoring ensures reliable operation by supervising battery status, communication integrity, body orientation, and system health throughout all modes of operation.

Finally, the advantages of the proposed architecture were highlighted. The dual-controller design, modular hardware configuration, local artificial intelligence processing, reinforcement learning-based locomotion, and scalable communication framework provide significant improvements over conventional quadruped robotic systems. These design choices not only enhance the robot's current capabilities but also establish a flexible platform for future integration of technologies such as **LiDAR-based SLAM, GPS-assisted outdoor navigation, robotic manipulation, autonomous charging, cloud robotics, gesture recognition, and multi-robot collaboration**.

Overall, the proposed system architecture provides a robust, scalable, and research-oriented foundation for the development of an intelligent autonomous quadruped robotic dog capable of operating in complex real-world environments. The integration of advanced sensing, artificial intelligence, embedded control, and adaptive locomotion demonstrates the feasibility of developing a cost-effective robotic platform suitable for research, education, and practical applications.

The next chapter presents the **mechanical design and hardware implementation** of the robotic dog, including the structural design, CAD modeling, actuator selection, electronic circuit design, component integration, and fabrication process used to realize the proposed system.

CHAPTER 4

MECHANICAL DESIGN AND HARDWARE IMPLEMENTATION

4.1 Mechanical Design Overview

The mechanical design of the proposed robotic dog forms the foundation of its overall performance, directly influencing locomotion stability, payload capacity, energy efficiency, and terrain adaptability. Unlike wheeled robotic platforms, a quadruped robot requires a carefully engineered mechanical structure capable of coordinating multiple joints while maintaining dynamic balance during walking, running, stair climbing, and obstacle negotiation. Therefore, the mechanical design was developed with emphasis on structural rigidity, lightweight construction, modularity, ease of manufacturing, and future scalability.

The proposed robotic dog adopts a **quadruped configuration with twelve degrees of freedom (12-DOF)**, providing three independently actuated joints in each leg. This configuration was selected to closely emulate the locomotion of biological quadrupeds while providing sufficient flexibility for adaptive gait generation and terrain traversal. Each leg consists of a Hip Pitch joint, Knee joint, and Ankle joint, enabling coordinated movement in multiple directions. The four-leg configuration also provides a larger support polygon compared to bipedal systems, resulting in improved static and dynamic stability.

The robot's main body serves as the central structural frame and houses all major electronic components, including the **NVIDIA Jetson Orin Nano**, **ESP32 microcontroller**, dual **PCA9685 PWM servo drivers**, battery, voltage regulation modules, communication interfaces, and auxiliary sensors. Positioning these components close to the geometric center of the chassis helps maintain a low center of gravity, improving balance during locomotion and reducing the torque required from the leg actuators.

The chassis has been designed using **Autodesk Fusion 360**, allowing accurate three-dimensional modeling of all structural components before fabrication. Computer-Aided Design (CAD) enables verification of component placement, mechanical clearances, joint ranges, wiring paths, and structural integrity before manufacturing. The modular CAD approach also allows individual components such as legs, servo mounts, electronics compartments, and sensor brackets to be modified or replaced independently without affecting the remaining structure of the robot.

To reduce overall weight while maintaining structural strength, the mechanical components are manufactured using **PLA (Polylactic Acid)** through **Fused Deposition Modeling (FDM)** 3D printing technology. PLA was selected due to its lightweight nature, ease of fabrication, dimensional stability, cost-effectiveness, and suitability for rapid prototyping. Areas subjected to higher mechanical loads, such as servo mounting brackets and leg joints, are reinforced with increased wall thickness and optimized internal infill patterns to improve stiffness and durability.

The locomotion mechanism is driven by **twelve MG996R high-torque metal gear servo motors**, each directly controlling one degree of freedom. The metal gear construction of the servos provides improved durability under repeated dynamic loading while delivering sufficient torque to support the robot during standing, walking, turning, and stair climbing. Two additional **MG90S servo motors** are integrated into the head and tail assemblies to provide expressive movement and improve human–robot interaction.

A major design objective was to ensure that the robotic dog could operate on both structured and unstructured terrain. The mechanical geometry of the legs was optimized to provide adequate ground clearance, sufficient stride length, and large joint motion ranges. These characteristics enable the robot to traverse rough trails, gravel, grass, inclined surfaces, and obstacles while maintaining body stability. Particular attention was given to the design of the foot mechanism, which provides improved ground contact and reduces the likelihood of slipping during locomotion.

The mechanical structure has also been designed to support one of the primary objectives of the project—**autonomous stair climbing with stair heights between 12 cm and 15 cm**. Leg lengths, joint rotation limits, and body dimensions were selected to ensure that the front legs can safely reach the next step while the rear legs provide sufficient support during weight transfer. Continuous feedback from the IMU allows the control system to maintain body

orientation throughout the climbing sequence, reducing mechanical stress on individual joints.

Another important consideration during the design process was **maintainability**. The robot consists of modular assemblies connected using standardized fasteners, allowing rapid replacement of servo motors, structural components, sensors, and electronic modules without complete disassembly of the system. This modular design significantly reduces maintenance time and supports future upgrades such as LiDAR integration, robotic manipulators, larger batteries, or improved actuator systems.

The final mechanical design represents a balance between **strength, weight, stability, manufacturability, and cost**. By integrating modern CAD techniques, additive manufacturing, modular construction, and optimized weight distribution, the proposed robotic dog provides a robust mechanical platform capable of supporting advanced artificial intelligence, reinforcement learning, autonomous navigation, human following, and voice-controlled interaction.

The subsequent sections of this chapter describe the detailed design requirements, CAD modeling process, chassis construction, leg mechanism, actuator selection, material selection, electronic integration, and assembly procedure used in the development of the proposed robotic dog.

Figure 4.1 Mechanical Design of the Proposed Robotic Dog

Insert a rendered CAD model of your robotic dog showing:

- Front View
- Side View
- Isometric View
- Top View

Label the following components:

- Head
- Chassis
- Front Legs
- Rear Legs
- MG996R Servo Motors
- MG90S Servos
- Camera Mount
- IMU Position

- Battery Compartment
- Jetson Orin Nano
- ESP32 Controller
- PCA9685 Driver
- OLED Display

Table 4.1 Mechanical Specifications

Parameter	Specification
Robot Configuration	Quadruped
Degrees of Freedom	12 (Legs) + 2 (Head & Tail)
Main Controller	NVIDIA Jetson Orin Nano
Embedded Controller	ESP32CEU6
Leg Actuator	MG996R Servo Motor
Head/Tail Actuator	MG90S Servo Motor
Frame Material	PLA (3D Printed)
Manufacturing Method	FDM 3D Printing
CAD Software	Autodesk Fusion 360
Stair Climbing Capability	12–15 cm
Terrain Capability	Flat, Grass, Gravel, Rough Trails, Slopes
Assembly Type	Modular

4.2 Mechanical Design Requirements

The mechanical design of the proposed quadruped robotic dog was developed by considering the functional objectives, environmental conditions, actuator capabilities, manufacturing constraints, and future scalability of the system. Since the robot is intended to operate on both structured and unstructured terrains while performing autonomous navigation, human following, obstacle avoidance, and stair climbing, the mechanical structure was required to satisfy several performance and reliability requirements. These design requirements served as the foundation for selecting the robot's dimensions, joint configuration, materials, actuators, and fabrication methods.

The primary design objective was to develop a lightweight yet structurally rigid platform capable of supporting all electronic components and actuators while maintaining stable locomotion under varying operating conditions. Excessive structural weight increases the

torque required from the servo motors, leading to higher power consumption, reduced battery life, and lower locomotion efficiency. Conversely, an excessively lightweight structure may lack sufficient mechanical strength to withstand repeated dynamic loading during walking, turning, or stair climbing. Therefore, an optimized balance between weight and structural rigidity was considered throughout the design process.

Another important requirement was the implementation of a **16-degree-of-freedom (16-DOF)** leg mechanism. Each of the four legs was designed with four independently actuated joints to provide sufficient flexibility for adaptive gait generation, body stabilization, and obstacle negotiation. This configuration allows the robot to perform smooth walking, turning, body posture adjustment, and stair climbing while maintaining a stable support polygon throughout locomotion.

The robot was specifically designed to operate on multiple terrain types, including smooth indoor floors, grass, gravel, rocky surfaces, rough trails, and inclined paths. To achieve this capability, the leg geometry was optimized to provide adequate ground clearance, sufficient stride length, and large joint rotation angles. The foot design was also selected to maximize ground contact and improve traction during movement over uneven terrain.

One of the principal design objectives of this project is the robot's ability to autonomously climb staircases with heights ranging from **12 cm to 15 cm**. This requirement significantly influenced the mechanical dimensions of the robot, particularly the leg length, joint rotation limits, and body clearance. The front legs must be capable of reaching the next step while the rear legs simultaneously provide stable support during weight transfer. The chassis geometry was therefore designed to prevent collisions between the robot body and staircase edges during ascent and descent.

The arrangement of internal electronic components was carefully planned to maintain a **low and centrally located center of gravity (CoG)**. Heavy components such as the battery, NVIDIA Jetson Orin Nano, and ESP32 controller are positioned close to the geometric center of the chassis. This configuration improves static stability, reduces body oscillations during locomotion, and minimizes actuator torque requirements during turning and stair climbing.

Modularity was another major design requirement. The chassis, leg assemblies, sensor mounts, and electronic compartments were designed as independent modules connected using standardized fasteners. This approach allows damaged components to be replaced quickly without requiring complete disassembly of the robot. Furthermore, modular construction facilitates future upgrades such as LiDAR integration, robotic arm attachment, larger battery packs, or improved actuator systems.

Manufacturability was also considered during the design stage. All structural components were designed to be fabricated using **Fused Deposition Modeling (FDM) 3D printing**, minimizing production cost while allowing rapid prototyping and easy design modifications. Features such as support-free printing orientations, uniform wall thickness, standardized

mounting holes, and simplified assembly were incorporated to reduce fabrication complexity and improve dimensional accuracy.

Since the robot is intended for extended experimental use, durability and maintainability were included as key design considerations. Structural components subjected to repeated loading, including servo brackets and leg joints, were reinforced to withstand cyclic mechanical stresses. Electronic components were positioned within protective enclosures to reduce the risk of damage caused by accidental impacts, dust, or wiring interference during operation.

Finally, the mechanical design was required to remain compatible with the embedded control and artificial intelligence architecture. Adequate space was reserved for wiring, cooling, communication modules, sensors, and future hardware expansion. This ensures that the mechanical structure can accommodate additional technologies without requiring significant redesign, thereby extending the operational life and research potential of the robotic platform.

Table 4.2 Mechanical Design Requirements

Requirement	Design Objective
Robot Configuration	Quadruped (4 Legs)
Degrees of Freedom	12-DOF (3 per leg)
Weight	Lightweight with high structural rigidity
Material	3D Printed PLA
Manufacturing Method	FDM 3D Printing
Terrain Capability	Flat, Grass, Gravel, Rough Trails, Slopes
Stair Climbing	12–15 cm Step Height
Center of Gravity	Low and centrally located
Chassis Design	Modular and compact
Assembly	Easy maintenance and component replacement
Expandability	Support future sensors and hardware upgrades
Stability	High static and dynamic stability
Durability	Reinforced joints and servo mounts
Power Integration	Internal battery compartment with protected wiring

4.3 Computer-Aided Design (CAD) Modeling

Computer-Aided Design (CAD) plays a crucial role in the development of modern robotic systems by enabling engineers to visualize, analyze, and optimize mechanical structures before fabrication. In this project, the complete mechanical model of the proposed quadruped robotic dog was developed using **Autodesk Fusion 360**, which provides an integrated environment for three-dimensional modeling, assembly design, motion verification, and engineering documentation. The use of CAD significantly reduced design errors, simplified component integration, and minimized manufacturing costs by allowing the complete robotic structure to be evaluated digitally before physical fabrication.

The CAD model was developed using a **bottom-up assembly approach**, where individual components were designed separately and later assembled into the complete robotic system. Each mechanical component, including the chassis, leg assemblies, servo brackets, sensor mounts, battery compartment, and electronic enclosures, was modeled independently. This modular approach simplified modifications during the design phase and allows damaged components to be replaced individually without redesigning the entire robot.

The design process began with the development of the **main chassis**, which serves as the structural backbone of the robotic dog. The chassis was dimensioned to accommodate all major electronic components, including the NVIDIA Jetson Orin Nano, ESP32 microcontroller, PCA9685 servo drivers, battery pack, voltage regulators, communication modules, and wiring harnesses. Internal mounting holes and cable-routing channels were incorporated directly into the CAD model to improve assembly quality and reduce wiring complexity.

Following the chassis design, the four leg assemblies were developed. Each leg consists of four articulated joints providing four degrees of freedom. Servo mounting brackets were designed according to the dimensions of the MG996R servo motors, ensuring accurate alignment between adjacent joints while minimizing backlash and mechanical interference. Particular attention was given to the range of motion of each joint to guarantee sufficient flexibility for walking, turning, rough-terrain locomotion, and autonomous stair climbing.

The dimensions of the leg links were optimized to achieve an appropriate balance between stride length, ground clearance, and mechanical stability. Longer links increase the robot's reachable workspace but also increase the torque required from the actuators. Therefore, the final link dimensions were selected to maximize mobility while remaining within the torque capabilities of the MG996R servo motors. This optimization contributes to efficient locomotion and reduces unnecessary power consumption during operation.

To improve overall stability, the heavy components of the robot were positioned near the center of the chassis during the CAD design stage. The battery compartment was placed close to the geometric center of the robot to lower the center of gravity, while the Jetson Orin Nano

and ESP32 controller were positioned to maintain balanced weight distribution between the front and rear legs. This arrangement reduces body oscillations during walking and minimizes actuator loading during turning and stair-climbing operations.

The head assembly was designed to accommodate the RGB camera, I2S digital microphone, OLED display, and future sensors without obstructing the robot's field of view. A dedicated MG90S servo motor enables controlled head movement, allowing the camera to adjust its viewing direction during human-following and object detection tasks. The tail assembly, actuated by a second MG90S servo motor, provides expressive movement that enhances human-robot interaction while contributing minimally to the robot's overall weight.

Fusion 360 assembly constraints were employed to verify the relative movement between adjacent mechanical components. Joint limits were defined according to the physical capabilities of the servo motors, and interference detection tools were used to ensure that no structural collisions occurred throughout the complete range of motion. Motion simulations were performed to evaluate walking postures, turning configurations, and stair-climbing poses before fabrication. These simulations allowed potential design issues to be identified and corrected early in the development process.

The CAD model was also optimized for **Fused Deposition Modeling (FDM) 3D printing**. Wall thickness, fillet radii, support requirements, and print orientation were carefully considered to reduce printing time while maintaining adequate structural strength. Components subjected to high mechanical loads, such as servo brackets and hip joints, were reinforced with increased wall thickness and higher infill density. Mounting holes were standardized throughout the design to simplify assembly using commercially available fasteners.

The completed CAD assembly provides an accurate digital representation of the final robotic dog, allowing dimensional verification, mass estimation, component placement, and manufacturing documentation to be generated efficiently. Furthermore, the modular design allows future hardware upgrades, including LiDAR integration, robotic manipulators, GPS modules, larger battery packs, and additional sensors, without requiring major structural modifications.

Overall, the CAD modeling process established a reliable and optimized mechanical foundation for the proposed robotic dog. The use of Autodesk Fusion 360 enabled efficient design iteration, improved structural integrity, simplified manufacturing, and ensured compatibility between the mechanical, electronic, and embedded subsystems prior to physical implementation.

Table 4.3 CAD Design Specifications

Parameter	Specification
CAD Software	Autodesk Fusion 360
Design Method	Bottom-Up Assembly
Robot Configuration	Quadruped
Degrees of Freedom	12 (Legs) + 2 (Head & Tail)
Assembly Type	Modular
Manufacturing Method	FDM 3D Printing
Material	PLA
Component Modeling	Individual Parts + Final Assembly
Motion Verification	Fusion 360 Joint Simulation
Collision Detection	Yes
Future Expandability	Supported

4.4 Chassis Design

The chassis serves as the primary structural framework of the proposed quadruped robotic dog and provides the foundation for integrating all mechanical, electronic, and computational subsystems. It supports the leg assemblies, houses the embedded controllers, protects sensitive electronic components, and maintains the structural integrity of the robot during locomotion. The chassis was designed to achieve an optimal balance between strength, weight, stability, manufacturability, and modularity while accommodating future hardware expansion.

The overall chassis was designed using **Autodesk Fusion 360** and optimized for fabrication using **Fused Deposition Modeling (FDM) 3D printing**. The geometry was developed to provide sufficient internal space for the **NVIDIA Jetson Orin Nano**, **ESP32 microcontroller**, **dual PCA9685 PWM drivers**, **11.1 V LiPo battery**, **voltage regulation modules**, communication interfaces, and wiring harnesses. Careful placement of these components ensured efficient utilization of the available internal volume while maintaining adequate ventilation for heat dissipation.

One of the most important considerations during the chassis design process was the **center of gravity (CoG)**. Heavy components such as the battery and Jetson Orin Nano were positioned close to the geometric center of the robot to reduce body oscillations during locomotion. Maintaining a centrally located CoG improves static and dynamic stability, decreases actuator loading, and enhances the robot's ability to traverse uneven terrain and climb stairs. This

balanced weight distribution also reduces energy consumption by minimizing unnecessary corrective movements during walking.

The chassis adopts a **rectangular modular structure** with reinforced side walls and internal support ribs to increase stiffness without significantly increasing weight. The front and rear sections provide mounting interfaces for the leg assemblies, while dedicated internal compartments secure the electronic hardware using standardized screws and mounting brackets. This arrangement protects the electronics from vibration and accidental impacts while allowing easy access during maintenance.

To improve mechanical strength, stress-concentrated regions around the leg mounting points were reinforced with additional material and filleted edges. These reinforcements distribute loads more evenly throughout the chassis, reducing the likelihood of crack initiation during repeated walking, turning, and stair-climbing operations. Rounded corners and smooth transitions between structural elements further reduce stress concentrations and improve the durability of the printed components.

The chassis design also incorporates an organized **cable management system**. Internal cable-routing channels and mounting clips guide power and communication cables between electronic components while preventing interference with moving leg mechanisms. Proper cable routing minimizes the risk of wire damage, simplifies assembly, and improves the overall appearance of the robot.

Thermal management was considered during the design process to ensure reliable operation of the embedded computing hardware. Ventilation openings were incorporated into the upper and side panels of the chassis to facilitate natural airflow around the **Jetson Orin Nano** and voltage regulation modules. Additional mounting provisions have been included to support future installation of cooling fans if higher computational workloads require active thermal management.

The front section of the chassis supports the **camera module**, **I2S digital microphone**, and **OLED display**, allowing these components to obtain an unobstructed view of the surrounding environment while enabling natural human–robot interaction. The head assembly is mounted separately to allow expressive movement without transmitting excessive vibration to the main body.

The lower section of the chassis houses the **LiPo battery compartment**, which includes a removable cover for convenient battery replacement and maintenance. Positioning the battery near the bottom of the chassis lowers the overall center of gravity and improves stability during rapid acceleration, turning, and obstacle negotiation. Protective walls surrounding the battery compartment reduce the likelihood of mechanical damage during accidental impacts.

To support future research and development, the chassis includes additional mounting holes and standardized interfaces for integrating optional hardware such as **LiDAR sensors**, **GPS modules**, **robotic manipulators**, **wireless communication devices**, and **environmental sensors**.

This modular design philosophy extends the operational lifespan of the robotic platform by allowing future upgrades without major structural redesign.

The final chassis design satisfies the primary objectives of the project by providing a lightweight, structurally rigid, and modular platform capable of supporting autonomous locomotion, reinforcement learning, computer vision, offline voice interaction, and intelligent navigation. Through careful optimization of weight distribution, structural reinforcement, internal component placement, and manufacturability, the chassis provides a reliable mechanical foundation for the complete robotic dog.

4.4.1 Chassis Design Features

The major features incorporated into the chassis design include:

- Lightweight 3D-printed PLA structure.
- Modular construction for easy maintenance.
- Reinforced leg mounting points.
- Low center of gravity for improved stability.
- Internal compartments for electronic components.
- Dedicated battery housing with removable cover.
- Cable management channels for organized wiring.
- Ventilation openings for passive cooling.
- Standardized mounting holes for future upgrades.
- Compact dimensions suitable for indoor and outdoor operation.

Table 4.4 Chassis Design Specifications

Parameter	Specification
Chassis Material	PLA
Manufacturing Method	FDM 3D Printing
CAD Software	Autodesk Fusion 360
Structure Type	Modular
Battery Location	Bottom Center
Main Controller Location	Center Compartment

Servo Mounting	Reinforced Side Panels
Cable Routing	Internal Channels
Cooling Method	Passive Ventilation
Future Expansion	LiDAR, GPS, Robotic Arm Ready

4.5 Leg Mechanism Design

The leg mechanism is the primary locomotion subsystem of the proposed quadruped robotic dog and plays a fundamental role in determining its mobility, balance, terrain adaptability, and overall performance. Unlike wheeled robots that are limited to relatively smooth surfaces, the four-legged configuration enables the robotic dog to traverse uneven terrain, climb stairs, avoid obstacles, and maintain stability under dynamic operating conditions. The leg mechanism was therefore designed to provide sufficient flexibility, strength, and precision while remaining lightweight and easy to manufacture.

The proposed robotic dog employs a **12-Degree-of-Freedom (12-DOF)** locomotion system, where each of the four legs possesses **three independently actuated rotational joints**. This configuration closely resembles the kinematic structure of biological quadrupeds and provides greater adaptability than traditional 3-DOF leg mechanisms. The additional degree of freedom allows improved body posture control, smoother gait transitions, enhanced balance, and more efficient navigation over complex terrain.

Each leg consists of the following joints:

- Hip Pitch Joint
- Knee Joint
- Ankle Joint

These joints work together to generate coordinated leg movements required for walking, turning, running, stair climbing, and body stabilization.

The **Hip Pitch Joint** is responsible for forward and backward movement of the leg and primarily determines the stride length of the robot. During locomotion, this joint generates the majority of the forward propulsion required for walking and running.

The **Knee Joint** controls leg extension and retraction, allowing the robot to adjust body height and foot clearance. Increasing knee flexion enables the robot to step over obstacles and climb stairs, while extension provides stable support during the stance phase of locomotion.

The **Ankle Joint** controls the orientation of the foot during ground contact. It improves foot placement accuracy, maintains ground contact on irregular surfaces, and contributes significantly to overall balance during stair climbing and rough-terrain locomotion.

The four joints operate in a coordinated manner through synchronized servo control, allowing each leg to execute smooth swing and stance phases during every walking cycle. The distributed control architecture ensures that all sixteen joints move synchronously while continuously compensating for body inclination and external disturbances.

To generate stable locomotion, the robot employs alternating gait patterns. During normal walking, two diagonal legs remain in contact with the ground while the remaining two legs move forward. This gait maintains continuous support and minimizes body oscillations. Higher-speed locomotion is achieved by modifying stride length, step frequency, and body posture through the reinforcement learning controller.

The leg geometry was carefully optimized to satisfy several design objectives. The overall leg length was selected to provide adequate ground clearance while maintaining sufficient mechanical stiffness. Excessively long legs increase actuator torque requirements and reduce structural rigidity, whereas shorter legs restrict obstacle negotiation and stair-climbing capability. The final dimensions therefore represent a compromise between mobility, stability, and energy efficiency.

Each structural link was designed with lightweight construction while incorporating reinforcement around servo mounting regions. Filleted transitions between structural elements reduce stress concentration and improve fatigue resistance during repetitive walking cycles. Standardized mounting interfaces simplify servo replacement and mechanical maintenance.

The foot mechanism was designed to maximize ground contact and improve traction across different terrain conditions. A slightly compliant foot pad distributes contact pressure more uniformly while reducing impact forces generated during landing. This design improves stability on grass, gravel, rocky surfaces, and uneven ground while reducing the likelihood of slipping during stair climbing.

A significant feature of the proposed leg mechanism is its capability to negotiate **staircases with step heights ranging from 12 cm to 15 cm**. During stair ascent, the front legs first establish stable contact with the higher step while the rear legs continue supporting the robot on the lower surface. Once body stability has been confirmed through IMU feedback, the rear legs sequentially climb onto the next step. This coordinated movement minimizes body pitching and ensures safe weight transfer throughout the climbing process.

Dynamic balance is achieved through continuous feedback obtained from the **GY-87 Inertial Measurement Unit (IMU)**. Body roll, pitch, and yaw are continuously monitored, allowing the embedded controller to adjust individual joint angles whenever external disturbances or uneven terrain cause deviations from the desired posture. The reinforcement learning controller further enhances stability by adapting gait parameters according to terrain characteristics and locomotion speed.

The leg mechanism was also designed with future expandability in mind. The modular construction allows individual links, servo motors, and brackets to be replaced or upgraded independently. Future improvements such as higher-torque actuators, force sensors, compliant joints, or carbon-fiber structural components can be integrated without requiring a complete redesign of the locomotion system.

Overall, the proposed leg mechanism combines a biologically inspired 16-DOF configuration with lightweight construction, adaptive control, and modular design to provide stable and efficient locomotion across a wide range of operating environments. The combination of optimized kinematics, reinforcement learning-based gait generation, and real-time balance control enables the robotic dog to perform complex locomotion tasks while maintaining high reliability and operational flexibility.

4.5.1 Kinematic Structure of One Leg

Each leg consists of four rotational joints arranged in a serial kinematic chain.

Joint Configuration:

Joint	Degree of Freedom	Function
Hip Pitch	1	Forward and backward leg movement
Knee	1	Leg extension and obstacle clearance
Ankle	1	Foot orientation and terrain adaptation

Total DOF per Leg: 3

Total Robot DOF: 12

4.5.2 Leg Motion During Walking

Each walking cycle consists of two phases:

Swing Phase

During this phase:

- The leg lifts off the ground.
- The knee flexes to increase foot clearance.
- The hip pitch moves the leg forward.
- The ankle adjusts foot orientation before landing.

Stance Phase

During this phase:

- The foot contacts the ground.
- The leg supports the robot's body weight.
- The hip generates forward propulsion.
- The ankle stabilizes ground contact.

The continuous alternation between swing and stance phases generates smooth and energy-efficient locomotion.

Table 4.5 Leg Mechanism Specifications

Parameter	Specification
Number of Legs	4
Degrees of Freedom per Leg	3
Total Degrees of Freedom	12
Main Actuator	MG996R Servo Motor
Material	PLA (3D Printed)
Joint Type	Revolute
Control Method	PWM via PCA9685
Walking Modes	Stand, Walk, Trot, Run
Stair Climbing Capability	12–15 cm
Terrain Capability	Flat, Grass, Gravel, Rough Trails

4.6 Actuator Selection

Actuators are the primary components responsible for generating movement in robotic systems by converting electrical energy into mechanical motion. In quadruped robots, the selection of suitable actuators directly influences locomotion performance, payload capacity, stability, precision, energy efficiency, and overall system reliability. Since the proposed robotic dog is designed to perform autonomous walking, rough-terrain traversal, stair climbing, and human interaction, careful consideration was given to actuator selection during the design phase.

The locomotion system of the proposed robotic dog utilizes a total of **fourteen servo motors**, consisting of **twelve MG996R high-torque servo motors** for the leg joints and **two MG90S**

micro servo motors for the head and tail mechanisms. This combination provides an appropriate balance between torque, speed, weight, cost, and ease of control while meeting the mechanical and operational requirements of the robot.

The **MG996R servo motor** was selected as the primary actuator because of its high torque capability, metal gear construction, and compatibility with Pulse Width Modulation (PWM) control. Each MG996R servo is capable of delivering sufficient torque to support the robot's body weight while performing walking, turning, balancing, and stair-climbing operations. The metal gear train offers improved durability compared to plastic gear servos, making it suitable for repeated dynamic loading during long-term operation.

Each of the sixteen MG996R servos independently controls one rotational joint of the quadruped mechanism. Four servos are assigned to each leg, providing four degrees of freedom that enable coordinated motion for adaptive locomotion. The servo motors receive PWM control signals from the dual PCA9685 controllers, which are managed by the ESP32 microcontroller. This arrangement ensures synchronized joint movement and accurate positioning while reducing computational overhead on the embedded controller.

Although the MG996R servo does not provide industrial-grade precision, it offers an excellent compromise between performance and affordability for educational and research applications. Its operating speed allows smooth execution of multiple gait patterns, including standing, walking, trotting, and stair climbing. The servo's wide operating voltage range also simplifies power system design by allowing direct operation from the regulated power supply used throughout the robot.

The **MG90S micro servo motor** was selected for the head and tail assemblies because these mechanisms require lower torque but benefit from reduced weight and compact dimensions. The head servo enables the robot to rotate its camera and microphone toward the direction of interest, improving object detection, human following, and voice interaction. The tail servo provides expressive movement that enhances human-robot interaction without significantly increasing the robot's overall power consumption.

The combination of high-torque MG996R servos and lightweight MG90S servos allows efficient allocation of actuator resources according to mechanical requirements. High-load joints responsible for supporting the robot's weight employ MG996R servos, while low-load mechanisms utilize smaller actuators to minimize total system weight and energy consumption.

During actuator selection, several engineering factors were evaluated, including torque output, rotational speed, positioning accuracy, physical dimensions, operating voltage, current consumption, mechanical durability, control compatibility, availability, and overall cost. The MG996R servo satisfied these requirements while remaining economically feasible for a research-oriented robotic platform.

The selected actuators are capable of generating sufficient torque to support locomotion on uneven terrain, grass, gravel, and inclined surfaces. Furthermore, the torque margin provided by the MG996R servos enables the robot to climb staircases with heights between **12 cm and 15 cm** while maintaining stable body posture and minimizing the risk of joint stalling during weight transfer.

Power consumption was another important consideration during actuator selection. Since sixteen high-torque servo motors may operate simultaneously, the electrical power system was designed to supply sufficient current without excessive voltage fluctuations. Dedicated DC–DC buck converters provide regulated voltage to the servo motors, while separate power rails isolate actuator loads from the embedded controllers and sensors. This arrangement improves electrical stability and prevents processor resets caused by sudden current surges during locomotion.

From a maintenance perspective, the selected servo motors are widely available and easy to replace. Standardized mounting dimensions simplify mechanical assembly and allow individual actuators to be replaced without modifying the surrounding mechanical structure. This modularity significantly reduces maintenance time and supports future upgrades to higher-performance actuators if required.

Overall, the selected actuator configuration provides an effective balance between mechanical performance, reliability, affordability, and ease of implementation. The combination of sixteen MG996R servo motors and two MG90S servo motors enables the proposed robotic dog to achieve stable locomotion, adaptive terrain traversal, autonomous stair climbing, and expressive human–robot interaction while maintaining a modular and scalable hardware architecture.

4.6.1 MG996R Servo Motor Specifications

The MG996R servo motors are responsible for all primary locomotion functions of the robotic dog.

Major Features

- High torque metal gear servo
- PWM position control
- 180° rotation range
- Metal gear transmission
- Suitable for robotic joints
- High durability

- Easy integration with PCA9685

Table 4.6 MG996R Servo Specifications

Parameter	Specification
Servo Model	MG996R
Quantity	12
Operating Voltage	4.8–7.2 V
Stall Torque	Up to 11 kg·cm (6 V)
Rotation Range	Approximately 180°
Gear Material	Metal
Control Signal	PWM
Application	Hip, Knee, and Ankle Joints

4.6.2 MG90S Servo Motor Specifications

The MG90S servo motors are used for lightweight mechanisms where high torque is unnecessary.

Applications include:

- Head rotation
- Tail movement

These actuators improve human–robot interaction while maintaining low power consumption.

Table 4.7 MG90S Servo Specifications

Parameter	Specification
Servo Model	MG90S
Quantity	2
Operating Voltage	4.8–6 V
Stall Torque	Approximately 2.2 kg·cm
Rotation Range	Approximately 180°
Gear Material	Metal

Control Signal	PWM
Application	Head and Tail

4.6.3 Actuator Allocation

Robot Part	Servo Model	Quantity
Front Left Leg	MG996R	3
Front Right Leg	MG996R	3
Rear Left Leg	MG996R	3
Rear Right Leg	MG996R	3
Head	MG90S	1
Tail	MG90S	1
Total	—	14 Servos

4.7 Material Selection and 3D Printing

The selection of appropriate materials and manufacturing techniques is a critical aspect of robotic system development, as these factors directly influence the structural strength, weight, durability, manufacturing cost, and overall performance of the robot. For the proposed quadruped robotic dog, the mechanical components were designed to be lightweight, structurally rigid, and easily manufacturable while supporting autonomous locomotion across various terrains. To achieve these objectives, **Polylactic Acid (PLA)** was selected as the primary structural material, and **Fused Deposition Modeling (FDM)** was adopted as the manufacturing process.

The use of additive manufacturing provides several advantages over conventional machining methods, including rapid prototyping, reduced material wastage, lower production cost, and the ability to manufacture complex geometries that would be difficult to produce using traditional manufacturing techniques. Since the robotic dog required numerous customized components, including servo brackets, chassis sections, leg links, sensor mounts, and electronic enclosures, 3D printing offered an efficient solution for fabricating these parts with high dimensional accuracy.

PLA was selected because it possesses a favorable combination of **lightweight characteristics, sufficient mechanical strength, ease of printing, dimensional stability, and environmental friendliness**. Compared to materials such as ABS or Nylon, PLA exhibits lower printing temperatures and reduced thermal shrinkage, minimizing warping during fabrication. These

properties allow large structural components, including the chassis and leg assemblies, to be printed reliably without requiring specialized printing equipment.

Another important consideration during material selection was the relationship between **structural weight and actuator performance**. Since the robotic dog employs twelve MG996R servo motors for locomotion, reducing the overall weight of the structure directly decreases the torque required from each actuator. Lower actuator loading results in smoother movement, reduced power consumption, improved battery life, and enhanced stair-climbing performance. The lightweight nature of PLA therefore contributes significantly to the efficiency of the complete robotic system.

Although PLA provides adequate strength for educational and research applications, regions subjected to higher mechanical stress were reinforced through geometric optimization rather than simply increasing material usage. Servo mounting brackets, hip joints, and load-bearing structural members were designed with thicker walls, internal ribs, and filleted transitions to distribute stresses more uniformly. This approach improves structural durability while maintaining a low overall mass.

The complete mechanical assembly was designed in **Autodesk Fusion 360** and exported in **STL (Stereolithography)** format before slicing. Slicing software was used to define print parameters including layer height, infill density, wall thickness, printing speed, nozzle temperature, bed temperature, and support structures. Each component was individually optimized according to its functional requirements to achieve an appropriate balance between strength, printing time, and material consumption.

The chassis components were printed with a higher wall thickness and moderate infill density to improve structural rigidity, whereas lightweight covers and decorative components were manufactured using lower infill percentages to reduce weight. Servo brackets and leg joints employed increased infill density and additional perimeters to withstand repeated mechanical loading during locomotion.

Following fabrication, all printed components were carefully inspected for dimensional accuracy and surface quality. Minor surface irregularities and support marks were removed through manual finishing before assembly. Standardized fasteners were then used to integrate the printed parts with the servo motors, electronic components, and structural hardware. The modular design of the printed components simplified the assembly process and allowed damaged parts to be replaced individually without requiring complete reconstruction of the robot.

The use of additive manufacturing also provides excellent flexibility for future development. If modifications to the leg geometry, chassis dimensions, or sensor mounts are required, only the affected components need to be redesigned and reprinted. This rapid iteration capability significantly accelerates the research and development cycle while reducing manufacturing costs compared to conventional fabrication methods.

For future versions of the robotic dog intended for outdoor or heavy-duty applications, alternative materials such as **PETG**, **ABS**, **Carbon Fiber Reinforced PLA (CF-PLA)**, or **Nylon** may be considered to improve impact resistance, thermal stability, and long-term durability. However, for the present research prototype, PLA provides an effective balance between mechanical performance, cost, ease of fabrication, and rapid prototyping capability.

Overall, the combination of PLA material and FDM 3D printing technology provides a lightweight, cost-effective, and highly customizable manufacturing solution for the proposed robotic dog. This approach supports rapid design iteration, modular construction, and efficient integration of mechanical and electronic subsystems while satisfying the structural requirements necessary for autonomous locomotion, rough-terrain traversal, and stair climbing.

4.7.1 Selection of PLA Material

The primary reasons for selecting PLA are:

- Lightweight construction.
- Easy to print with minimal warping.
- High dimensional accuracy.
- Cost-effective for prototyping.
- Good surface finish.
- Environmentally friendly (biodegradable).
- Suitable for educational and research robots.
- Compatible with standard FDM 3D printers.

4.7.2 FDM 3D Printing Process

The manufacturing workflow followed in this project is illustrated below:

1. Design CAD model in Autodesk Fusion 360.
2. Export components as STL files.
3. Import STL files into slicing software.
4. Configure printing parameters.
5. Generate G-code.

6. Print individual components using an FDM 3D printer.
7. Remove support material and perform finishing.
8. Assemble printed parts with actuators and electronics.

Table 4.8 3D Printing Parameters

Parameter	Value
Printing Technology	FDM (Fused Deposition Modeling)
Material	PLA
CAD Software	Autodesk Fusion 360
File Format	STL
Nozzle Diameter	0.4 mm
Layer Height	0.20 mm
Nozzle Temperature	200–210°C
Bed Temperature	55–60°C
Infill Density	30–50% (depending on part)
Wall Thickness	2–3 mm
Print Speed	50–60 mm/s
Support Material	Enabled where required

4.8 Electronic Hardware Integration

The electronic hardware integration process combines all embedded controllers, sensors, actuators, communication modules, and power management circuits into a unified robotic platform capable of autonomous locomotion, environmental perception, and intelligent decision-making. Proper integration of these components is essential to ensure reliable operation, minimize electrical interference, simplify maintenance, and provide sufficient computational capability for real-time robotic applications.

The proposed robotic dog adopts a **distributed electronic architecture** consisting of two primary processing units: the **NVIDIA Jetson Orin Nano** and the **ESP32CEU6 microcontroller**. The Jetson Orin Nano serves as the high-level processing unit responsible for artificial intelligence, computer vision, reinforcement learning, offline voice recognition, and autonomous navigation. The ESP32 functions as the low-level embedded controller,

executing real-time servo control, sensor acquisition, PWM generation, communication management, and system monitoring.

Both processing units are securely mounted inside the central chassis using dedicated mounting brackets designed in Autodesk Fusion 360. Their positions were selected to minimize cable lengths, improve weight distribution, and facilitate airflow for passive cooling. The Jetson Orin Nano is positioned near the upper section of the chassis to provide convenient access to USB ports for the RGB camera and future LiDAR sensor, while the ESP32 is located adjacent to the PWM drivers to simplify communication with peripheral devices.

The locomotion system employs **two PCA9685 16-channel PWM driver modules** to generate control signals for the servo motors. These drivers are mounted on dedicated brackets close to the ESP32 microcontroller, reducing I²C bus length and minimizing electrical noise. The dual-driver configuration distributes the servo load evenly, with one PCA9685 controlling the left-side leg servos and the second controlling the right-side leg servos. This arrangement simplifies wiring and improves maintainability.

The **GY-87 Inertial Measurement Unit (IMU)** is mounted near the geometric center of the chassis to provide accurate measurements of body orientation while minimizing the influence of rotational acceleration. The central placement improves the accuracy of roll, pitch, and yaw estimation used by the balance control system.

The **VL53L0X Time-of-Flight (ToF) sensor** is positioned at the front of the robot with an unobstructed forward-facing field of view. This location enables early detection of obstacles, edges, and staircases during autonomous navigation. The sensor communicates with the ESP32 through the I²C interface, allowing rapid distance measurements for collision avoidance and terrain assessment.

The robot's **RGB camera** is mounted within the head assembly to maximize its viewing angle and support computer vision applications such as object detection, human following, and stair recognition. Positioning the camera at an elevated location reduces blind spots and improves the performance of the YOLOv8 object detection algorithm. The camera connects directly to the NVIDIA Jetson Orin Nano through the USB interface for high-speed image acquisition.

An **I2S digital microphone** is integrated adjacent to the camera module to capture voice commands while minimizing interference from the servo motors. Audio data is transmitted directly to the Jetson Orin Nano, where offline speech recognition algorithms process user commands without requiring internet connectivity.

The **0.96-inch OLED display** is installed on the front panel of the robot and connected to the ESP32 microcontroller. The display provides real-time information including battery status, locomotion mode, communication status, operating conditions, and diagnostic messages. This interface enables users to monitor the robot without requiring an external computer.

Power is supplied by an **11.1 V Lithium Polymer (LiPo) battery**, positioned within the lower central compartment of the chassis to maintain a low center of gravity. Two **DC–DC buck converters** regulate the battery voltage for different subsystems. One converter supplies the servo motors with a stable 6 V output, while the second powers the ESP32, sensors, OLED display, and communication modules. The NVIDIA Jetson Orin Nano receives a dedicated regulated supply to ensure stable operation during periods of high computational load.

The power distribution network employs **separate power rails** for high-current servo motors and low-power electronic circuits. This electrical isolation reduces voltage fluctuations and minimizes electromagnetic interference generated by rapid servo movements. Appropriate decoupling capacitors and protective circuitry further improve system stability and prevent unexpected processor resets during demanding locomotion tasks.

Cable management was carefully considered during hardware integration. Dedicated routing channels and cable clips guide power and communication wires through the chassis while preventing interference with moving leg mechanisms. Connectors are clearly labeled to simplify maintenance and troubleshooting. The modular wiring arrangement allows individual components to be replaced without disturbing the remaining electronic subsystems.

Communication between hardware components is achieved using multiple interfaces. The ESP32 communicates with the NVIDIA Jetson Orin Nano through **UART**, while peripheral sensors and PWM drivers are connected using the **I²C** protocol. The OLED display uses **SPI**, and the RGB camera communicates with the Jetson through **USB**. PWM signals generated by the PCA9685 modules drive all servo motors with precise timing and synchronization.

Following integration, comprehensive functional testing was performed to verify correct operation of all electronic subsystems. Communication links, power distribution, sensor measurements, servo responses, and embedded software execution were validated before mechanical testing commenced. Successful integration confirmed that all hardware components operated together as a unified robotic platform capable of supporting autonomous locomotion, reinforcement learning, object detection, voice interaction, and stair climbing.

The completed electronic hardware integration provides a reliable and scalable foundation for the proposed robotic dog. The modular arrangement of controllers, sensors, communication interfaces, and power electronics simplifies future expansion and supports additional technologies such as LiDAR, GPS, robotic manipulators, wireless communication modules, and autonomous charging systems.

4.8.1 Electronic Subsystems

The electronic hardware of the robotic dog is organized into the following subsystems:

- **Processing Unit:** NVIDIA Jetson Orin Nano and ESP32.
- **Locomotion Control:** Two PCA9685 PWM driver modules controlling 12 MG996R and 2 MG90S servo motors.
- **Perception System:** RGB camera, GY-87 IMU, and VL53L0X ToF sensor.
- **Human–Robot Interaction:** I2S digital microphone and OLED display.
- **Power System:** 11.1 V LiPo battery with dual DC–DC buck converters.
- **Communication System:** UART, I²C, SPI, USB, and PWM interfaces.

Table 4.10 Electronic Hardware Integration

Component	Location	Function
NVIDIA Jetson Orin Nano	Upper Chassis	AI processing, YOLOv8, PPO, speech recognition
ESP32	Center Chassis	Real-time embedded control
PCA9685 Driver 1	Left Electronics Bay	Left-side servo control
PCA9685 Driver 2	Right Electronics Bay	Right-side servo control
GY-87 IMU	Center of Chassis	Orientation and balance sensing
VL53L0X ToF Sensor	Front Panel	Obstacle and stair detection
RGB Camera	Head Assembly	Vision and object detection
I2S Microphone	Head Assembly	Offline voice commands
OLED Display	Front Panel	System status display
LiPo Battery	Bottom Compartment	Primary power source
Buck Converter 1	Power Bay	Servo power regulation
Buck Converter 2	Power Bay	Logic power regulation

Table 4.11 Communication Connections

Source	Destination	Protocol
Jetson Orin Nano	ESP32	UART
ESP32	PCA9685 Drivers	I ² C
ESP32	GY-87 IMU	I ² C
ESP32	VL53L0X ToF	I ² C

ESP32	OLED Display	SPI
Jetson Orin Nano	RGB Camera	USB
PCA9685	MG996R & MG90S Servos	PWM

4.9 Wiring and PCB Design

The electrical wiring and interconnection system of the proposed robotic dog plays a critical role in ensuring reliable communication, stable power distribution, and efficient integration of all hardware components. Since the robot consists of multiple embedded controllers, sensors, actuators, and power modules operating simultaneously, careful wiring organization was necessary to minimize electrical interference, simplify maintenance, and improve overall system reliability.

The electronic architecture follows a **modular wiring approach**, where individual subsystems including processing, sensing, actuation, communication, and power management are interconnected using standardized connectors and organized cable routing. This modular arrangement simplifies troubleshooting and allows components to be replaced or upgraded independently without affecting the remaining electrical system.

The **NVIDIA Jetson Orin Nano** functions as the high-level computing platform and communicates with the **ESP32 microcontroller** through a dedicated UART serial interface. This communication link transfers locomotion commands, navigation information, voice command execution requests, and system status between the artificial intelligence subsystem and the embedded control subsystem. A common ground connection is maintained between both controllers to ensure reliable serial communication.

The **ESP32** serves as the central embedded controller and interfaces with the remaining peripheral devices through the I²C and SPI communication buses. The two PCA9685 PWM servo drivers are connected to the ESP32 through the I²C bus, enabling synchronized control of all eighteen servo motors. The GY-87 IMU and VL53L0X Time-of-Flight sensor are also connected to the same I²C bus using unique device addresses, reducing wiring complexity while allowing efficient communication with multiple peripherals.

The **OLED display** is connected to the ESP32 using the SPI communication interface to provide high-speed graphical updates. The RGB camera is connected directly to the NVIDIA Jetson Orin Nano through a USB interface, ensuring sufficient bandwidth for real-time image processing using the YOLOv8 object detection model. Similarly, the I2S digital microphone is connected to the Jetson using its dedicated digital audio interface, enabling offline speech recognition with low latency.

Servo power distribution is isolated from the logic circuitry to prevent electrical disturbances caused by high-current actuator operation. The twelve MG996R servo motors and two MG90S servo motors receive regulated 6 V power from a dedicated DC–DC buck converter, while the ESP32, sensors, and peripheral electronics operate from a separate regulated 5 V supply. The NVIDIA Jetson Orin Nano is powered through an independent regulator designed to meet its higher current requirements. This separation of power rails minimizes voltage fluctuations and prevents processor resets during demanding locomotion tasks.

Power distribution is implemented using appropriately sized conductors capable of carrying the current required by the servo motors. High-current power lines are routed separately from low-voltage signal cables to reduce electromagnetic interference. Signal wiring for UART, I²C, SPI, and PWM communication is kept as short as possible to improve communication reliability and reduce susceptibility to electrical noise.

Cable management was carefully considered during assembly. Wiring harnesses are routed through dedicated channels integrated into the chassis and secured using cable ties and mounting clips. This arrangement prevents wires from interfering with moving leg mechanisms and protects the cables from mechanical wear during repeated locomotion cycles. Color-coded wiring and labeled connectors further simplify assembly, maintenance, and future hardware upgrades.

Protection circuits were incorporated into the electrical system to improve operational safety. Appropriate fuses and voltage regulators protect sensitive electronic components against overcurrent and voltage fluctuations. Reverse polarity protection safeguards the system during battery connection, while decoupling capacitors positioned near major electronic modules improve voltage stability during rapid changes in servo current demand.

Although the present prototype utilizes modular wiring with commercially available development boards, the electrical design has been developed with future migration to a **custom Printed Circuit Board (PCB)** in mind. A dedicated PCB would integrate the ESP32, power regulation circuits, communication interfaces, and peripheral connectors into a compact module, reducing wiring complexity, improving reliability, and simplifying assembly. Future versions of the robotic dog may also employ multi-layer PCB technology to enhance electromagnetic compatibility and facilitate integration of additional sensors and communication modules.

The completed wiring architecture provides a robust and maintainable electrical infrastructure that supports reliable communication, stable power delivery, and synchronized operation of all hardware components. This organized wiring system contributes significantly to the overall reliability and serviceability of the robotic platform.

4.9.1 Wiring Architecture

The electrical wiring is divided into five major groups:

- **Power Distribution Wiring:** Battery, buck converters, voltage regulators, and servo power.
- **Communication Wiring:** UART, I²C, SPI, USB, and I2S connections.
- **Servo Signal Wiring:** PWM signals from the PCA9685 drivers to the servo motors.
- **Sensor Wiring:** IMU, ToF sensor, camera, microphone, and OLED display.
- **Ground Network:** Common ground connections between all electronic subsystems.

4.9.2 Power Distribution Network

The robot employs separate regulated power rails to improve electrical stability.

11.1 V LiPo Battery

Supplies the entire robotic system.

6 V Servo Rail

Powers:

- 12 MG996R servo motors
- 2 MG90S servo motors

5 V Logic Rail

Powers:

- ESP32
- PCA9685 Drivers
- GY-87 IMU
- VL53L0X ToF Sensor
- OLED Display

Jetson Power Rail

Provides an independent regulated supply to the NVIDIA Jetson Orin Nano.

This architecture prevents high-current servo operation from affecting sensitive computing hardware.

4.9.3 Future PCB Design

For future versions, a custom PCB is proposed to provide:

- Integrated ESP32 microcontroller
- Dual PCA9685 interface
- Battery management circuitry
- Voltage regulation
- Power distribution
- Sensor connectors
- UART, I²C, SPI, and USB interfaces
- Status LEDs
- Protection circuitry
- Expansion headers for future sensors

The custom PCB would reduce wiring complexity, improve reliability, and simplify maintenance.

Table 4.12 Electrical Connections

Source	Destination	Interface
LiPo Battery	Buck Converter (6 V)	Power
LiPo Battery	Buck Converter (5 V)	Power
Buck Converter	Servo Motors	Power
Buck Converter	ESP32	Power
Buck Converter	Jetson Orin Nano	Power
Jetson Orin Nano	ESP32	UART
ESP32	PCA9685 Drivers	I ² C
ESP32	GY-87 IMU	I ² C
ESP32	VL53L0X ToF	I ² C
ESP32	OLED Display	SPI
Jetson Orin Nano	RGB Camera	USB
Jetson Orin Nano	I2S Microphone	I2S

PCA9685 Drivers	Servo Motors	PWM
-----------------	--------------	-----

Table 4.13 Proposed PCB Features

PCB Section	Function
ESP32 Core	Embedded control
Power Regulation	Voltage conversion
Battery Input	LiPo connection
Servo Headers	MG996R and MG90S connections
I ² C Bus	Sensor communication
SPI Interface	OLED display
UART Port	Jetson communication
Expansion Headers	Future LiDAR, GPS, and sensors
Protection Circuit	Reverse polarity and overcurrent protection

4.10 Assembly Procedure

The assembly procedure of the proposed robotic dog involved the systematic integration of the mechanical structure, actuators, embedded controllers, sensors, communication modules, and power management circuitry into a fully operational robotic platform. A modular assembly approach was adopted to simplify manufacturing, maintenance, troubleshooting, and future hardware upgrades. By assembling the robot in stages, each subsystem could be individually inspected and tested before being integrated into the complete system.

The assembly process began with the inspection of all 3D-printed components. Each printed part was examined for dimensional accuracy, surface quality, and structural integrity. Support material generated during the printing process was removed, and minor finishing operations such as sanding and cleaning were performed to ensure proper fitting between mating components. Threaded holes and mounting interfaces were checked to verify compatibility with the selected fasteners.

The **main chassis** was assembled first because it serves as the structural foundation of the robot. The upper and lower chassis sections were joined using stainless steel screws and threaded inserts, forming a rigid frame capable of supporting the remaining mechanical and electronic components. Internal mounting brackets for the battery, controllers, and electronic modules were installed before integrating the leg mechanisms.

Following chassis assembly, the **leg mechanisms** were constructed individually. Each leg consists of three articulated joints actuated by MG996R servo motors. Servo motors were first centered electronically to their neutral positions before being installed in the printed servo brackets. This calibration ensured that all joints began with identical reference angles, reducing alignment errors during gait generation. The upper and lower leg links were then connected to the servo horns using precision fasteners, allowing smooth rotational movement throughout the required operating range.

After completing the four leg assemblies, they were mounted to the chassis at the designated attachment points. Care was taken to ensure that each leg was aligned symmetrically with respect to the robot's body. The complete mechanical structure was manually inspected to verify unrestricted joint movement and to ensure that no mechanical interference occurred during full-range articulation.

The **electronic hardware** was installed after completion of the mechanical framework. The NVIDIA Jetson Orin Nano and ESP32 microcontroller were mounted on their respective brackets inside the central compartment of the chassis. Two PCA9685 PWM driver modules were positioned adjacent to the ESP32 to minimize communication cable lengths and improve wiring organization. The LiPo battery was secured within the lower battery compartment using adjustable retaining straps to prevent movement during locomotion.

The **sensor modules** were then integrated into the robot. The GY-87 IMU was installed near the geometric center of the chassis to provide accurate orientation measurements, while the VL53L0X Time-of-Flight sensor was mounted on the front panel for obstacle detection. The RGB camera, I2S microphone, and OLED display were assembled within the head section to support object detection, human following, voice interaction, and user feedback. Wiring for each sensor was routed through dedicated cable channels to prevent interference with the moving leg mechanisms.

Once all electronic components had been installed, the complete wiring harness was assembled. Separate power and signal cables were routed through the chassis using cable clips and protective sleeves. High-current power cables supplying the servo motors were isolated from communication lines to reduce electromagnetic interference. Connectors were labeled and secured to simplify future maintenance and reduce the possibility of incorrect wiring during component replacement.

After completing the mechanical and electrical assembly, the robot underwent **system calibration**. Servo motors were initialized to their default positions, and the IMU was calibrated to establish reference values for roll, pitch, and yaw. Communication between the Jetson Orin Nano and ESP32 was verified through UART testing, while individual sensors were tested to confirm correct operation. Each servo motor was actuated independently before executing synchronized leg movements to verify proper joint orientation and wiring.

The final stage involved **functional integration testing**. The robot was powered using the regulated power system, and all subsystems—including locomotion, sensor acquisition, communication, object detection, and voice recognition—were tested together. Walking trials, turning maneuvers, rough-terrain movement, obstacle detection, and stair-climbing experiments were conducted to verify successful integration of the mechanical, electrical, and software systems.

The modular assembly strategy adopted in this project significantly reduced construction complexity while improving accessibility to internal components. Individual subsystems can be removed and serviced without disassembling the entire robot, allowing rapid replacement of actuators, sensors, or embedded controllers. This approach also supports future hardware upgrades, making the robotic dog a flexible research platform for continued development.

Overall, the assembly procedure transformed the individually manufactured mechanical and electronic components into a fully integrated autonomous robotic system. Careful alignment, organized wiring, systematic calibration, and staged testing ensured reliable operation and established a robust foundation for the experimental evaluation presented in subsequent chapters.

4.10.1 Assembly Sequence

The complete assembly process followed these steps:

1. Inspect and finish all 3D-printed components.
2. Assemble the main chassis.
3. Mount all MG996R servo motors.
4. Assemble the four leg mechanisms.
5. Attach the legs to the chassis.
6. Install the NVIDIA Jetson Orin Nano.
7. Install the ESP32 microcontroller.
8. Mount the PCA9685 PWM driver modules.
9. Install the LiPo battery and power modules.
10. Mount the IMU, ToF sensor, camera, microphone, and OLED display.
11. Route and secure all wiring.
12. Perform servo calibration.
13. Calibrate sensors.

- 14. Verify communication between hardware modules.
- 15. Conduct functional testing.
- 16. Perform locomotion and stair-climbing validation

Table 4.14 Assembly Components

Assembly Stage	Components Installed
Stage 1	Chassis and structural frame
Stage 2	MG996R and MG90S servo motors
Stage 3	Four leg assemblies
Stage 4	NVIDIA Jetson Orin Nano
Stage 5	ESP32 microcontroller
Stage 6	PCA9685 PWM drivers
Stage 7	Battery and power distribution modules
Stage 8	Camera, IMU, ToF sensor, microphone, OLED
Stage 9	Wiring harness and connectors
Stage 10	Calibration and testing

Table 4.15 Assembly Tools

Tool	Purpose
Screwdriver Set	Mechanical assembly
Hex Key Set	Fastening servo brackets
Soldering Station	Electrical connections
Wire Stripper	Cable preparation
Heat Shrink Tubing	Wire insulation
Digital Multimeter	Electrical testing
Laptop	Firmware upload and diagnostics
3D Printer Tools	Component finishing

4.11 Mechanical Analysis

The mechanical analysis of the proposed quadruped robotic dog was performed to evaluate the structural integrity, stability, weight distribution, joint mobility, and overall suitability of the mechanical design for autonomous locomotion. Since the robot is expected to operate on rough terrain, climb staircases, avoid obstacles, and maintain dynamic balance, it is essential to ensure that the mechanical structure can withstand the loads generated during these operations without excessive deformation or failure.

The analysis focused on several key aspects of the mechanical design, including structural strength, center of gravity, joint motion, actuator loading, stability, and manufacturability. Computer-Aided Design (CAD) models developed in Autodesk Fusion 360 were used to verify component alignment, assembly compatibility, and the range of motion of each leg before fabrication.

One of the primary objectives of the analysis was to minimize the overall weight of the robot while maintaining sufficient structural rigidity. Since the quadruped uses sixteen MG996R servo motors, reducing the structural weight decreases the torque required from each actuator, leading to lower power consumption and improved battery life. The chassis and leg components were therefore optimized to remove unnecessary material while reinforcing high-stress regions such as servo mounting brackets, hip joints, and leg connections.

The weight distribution of the robot was carefully analyzed to ensure that the **center of gravity (CoG)** remained close to the geometric center of the chassis. Heavy components including the LiPo battery, NVIDIA Jetson Orin Nano, and ESP32 microcontroller were positioned centrally and close to the lower section of the chassis. This arrangement improves both static and dynamic stability by reducing body roll and pitch during locomotion, turning, and stair climbing.

The **range of motion** of each leg was also verified during the design phase. Fusion 360 joint simulations confirmed that the Hip Pitch, Knee, and Ankle joints could move through their required operating ranges without mechanical interference between adjacent components. The motion analysis ensured that the robot could achieve the leg positions required for walking, turning, rough-terrain traversal, and climbing staircases with heights between **12 cm and 15 cm**.

Structural analysis also considered the loads transmitted through the legs during walking. During locomotion, each supporting leg experiences compressive forces generated by the weight of the robot and dynamic forces resulting from acceleration and deceleration. Reinforced servo brackets and optimized leg geometries distribute these loads uniformly, reducing stress concentrations and improving the durability of the printed components.

The stair-climbing capability of the robot was evaluated by analyzing the leg reach, body clearance, and joint movement required to negotiate staircases. The selected leg dimensions

provide sufficient reach for the front legs to access the next step while maintaining stability through support from the rear legs. Adequate chassis clearance prevents the underside of the robot from colliding with stair edges during ascent or descent.

Dynamic stability during locomotion is achieved through the combined action of the mechanical structure and the embedded control system. The GY-87 Inertial Measurement Unit continuously monitors body orientation, allowing the control algorithm to compensate for disturbances by adjusting individual joint angles. This feedback mechanism reduces body oscillations and improves the robot's ability to maintain balance on uneven terrain.

The modular construction of the mechanical system was another important aspect of the analysis. Each leg assembly can be removed independently for maintenance or replacement, reducing repair time and allowing rapid hardware upgrades. Standardized fasteners and interchangeable components further simplify servicing and improve the long-term usability of the robotic platform.

The analysis also confirmed that the selected PLA material, combined with optimized wall thickness and internal reinforcement, provides sufficient mechanical strength for research and educational applications. Although PLA is not intended for heavy industrial environments, it offers an excellent balance between strength, weight, cost, and ease of manufacturing for the objectives of this project.

Overall, the mechanical analysis demonstrates that the proposed quadruped robotic dog satisfies the structural and functional requirements necessary for autonomous locomotion, rough-terrain navigation, human following, obstacle avoidance, and stair climbing. The optimized weight distribution, reinforced structural design, adequate joint mobility, and modular construction provide a reliable mechanical foundation for the experimental evaluation presented in the subsequent chapters.

4.11.1 Mechanical Performance Evaluation

The mechanical design was evaluated based on the following criteria:

- Structural strength
- Weight optimization
- Center of gravity location
- Joint range of motion
- Ground clearance
- Stair-climbing capability
- Modular construction

- Ease of maintenance
- Manufacturability
- Future expandability

Table 4.16 Mechanical Analysis Summary

Parameter	Evaluation
Structural Strength	Reinforced PLA structure
Overall Weight	Optimized for servo-driven locomotion
Degrees of Freedom	16 (Legs) + 2 (Head & Tail)
Center of Gravity	Low and centrally located
Ground Clearance	Suitable for rough terrain
Stair-Climbing Capability	12–15 cm
Leg Mobility	Full range without interference
Chassis Rigidity	High
Component Accessibility	Modular
Manufacturing Method	FDM 3D Printing

Table 4.17 Mechanical Design Advantages

Design Feature	Benefit
Lightweight PLA Structure	Reduced actuator load
Reinforced Servo Mounts	Improved durability
Centralized Battery Placement	Better stability
Modular Leg Design	Easy maintenance
Internal Cable Routing	Improved reliability
Optimized Leg Geometry	Efficient walking and climbing
Low Center of Gravity	Increased balance
Expandable Chassis	Supports future upgrades

4.12 Chapter Summary

This chapter presented the complete mechanical design and hardware implementation of the proposed intelligent quadruped robotic dog. The objective of this chapter was to describe the transformation of the conceptual system architecture presented in the previous chapter into a fully integrated physical robotic platform capable of autonomous locomotion, environmental perception, and intelligent human–robot interaction.

The chapter began with an overview of the mechanical design philosophy, emphasizing the importance of developing a lightweight, modular, and structurally robust quadruped platform. The design requirements established the engineering criteria that guided the development process, including stability, manufacturability, terrain adaptability, stair-climbing capability, ease of maintenance, and future scalability. These requirements ensured that the mechanical structure could support the robot's intended applications while maintaining reliable performance under varying operating conditions.

A detailed Computer-Aided Design (CAD) model of the robotic dog was developed using **Autodesk Fusion 360**. The modular design approach enabled accurate placement of mechanical and electronic components, verification of assembly compatibility, collision detection, and motion analysis before fabrication. The use of CAD significantly reduced design errors, improved manufacturing efficiency, and facilitated future modifications to the robotic platform.

The chassis was designed as the primary structural framework, providing mechanical support for the leg assemblies, embedded controllers, sensors, battery, and communication modules. Special attention was given to achieving a low and centrally located center of gravity by strategically positioning heavy components such as the **NVIDIA Jetson Orin Nano, ESP32 microcontroller**, and **11.1 V LiPo battery** within the central compartment of the robot. This optimized weight distribution contributes to improved static and dynamic stability during locomotion, turning, rough-terrain traversal, and stair climbing.

The locomotion system employs a **12-degree-of-freedom (12-DOF)** leg mechanism, with four independently actuated joints per leg. This configuration enables smooth and adaptive movement across a variety of environments while providing sufficient flexibility for balance control, obstacle negotiation, and autonomous stair climbing. The leg geometry and joint arrangement were optimized to maximize ground clearance, stride length, and body stability while remaining within the torque capabilities of the selected actuators.

Actuator selection was based on the performance requirements of the robotic platform. **Sixteen MG996R metal-gear servo motors** were selected to actuate the leg joints due to their high torque output and durability, while **two MG90S servo motors** were incorporated into the head and tail assemblies to provide expressive movement and improve human–robot

interaction. The actuator configuration provides sufficient mechanical performance for autonomous walking, rough-terrain locomotion, and stair-climbing tasks.

Material selection and manufacturing were accomplished using **Polylactic Acid (PLA)** and **Fused Deposition Modeling (FDM) 3D printing**. This combination provided an effective balance between structural strength, lightweight construction, low manufacturing cost, and rapid prototyping capability. The modular printed components simplify maintenance, reduce fabrication time, and allow damaged parts to be replaced individually without reconstructing the entire robot.

The chapter also described the complete electronic hardware integration process. The **NVIDIA Jetson Orin Nano** serves as the high-level processing unit responsible for artificial intelligence, reinforcement learning, computer vision, and offline speech recognition, while the **ESP32** performs real-time embedded control, sensor acquisition, and servo management. Additional hardware including the **GY-87 IMU**, **VL53L0X Time-of-Flight sensor**, **RGB camera**, **OLED display**, **I2S digital microphone**, and **dual PCA9685 PWM drivers** were successfully integrated into the chassis to provide sensing, communication, and actuation capabilities.

A structured wiring architecture was implemented to ensure reliable communication between all electronic subsystems. Standard communication protocols including **UART**, **I2C**, **SPI**, **USB**, and **PWM** were employed to facilitate efficient data transfer while separate regulated power rails improved electrical stability and minimized interference between high-current servo motors and sensitive electronic components.

The complete assembly procedure demonstrated the systematic integration of mechanical, electrical, and embedded subsystems into a fully functional robotic platform. Component alignment, wiring organization, servo calibration, and sensor initialization were performed before conducting preliminary functional tests. The modular assembly approach simplifies future maintenance and provides flexibility for integrating additional sensors and hardware upgrades.

Finally, the mechanical characteristics of the robotic dog were evaluated through structural assessment, weight distribution analysis, joint motion verification, and stability evaluation. The analysis confirmed that the optimized chassis, reinforced leg mechanism, centralized

center of gravity, and modular construction satisfy the mechanical requirements necessary for autonomous locomotion, rough-terrain traversal, obstacle avoidance, human following, and stair climbing over steps with heights ranging from **12 cm to 15 cm**.

Overall, the mechanical design and hardware implementation provide a robust foundation for the intelligent capabilities of the robotic dog. The combination of optimized structural design,

distributed embedded computing, modular electronics, and additive manufacturing establishes a reliable research platform capable of supporting advanced artificial intelligence, reinforcement learning, and autonomous navigation in real-world environments.

The next chapter presents the **software implementation and control system**, including firmware development, ROS2 integration, reinforcement learning algorithms, computer vision, offline voice recognition, locomotion control, and the software framework responsible for coordinating all robotic functions.

CHAPTER 5

SOFTWARE IMPLEMENTATION AND CONTROL SYSTEM

5.1 Software Implementation Overview

The software implementation of the proposed quadruped robotic dog provides the intelligence required for autonomous locomotion, environmental perception, human–robot interaction, and real-time decision making. Unlike conventional robotic systems that rely on a single controller, the proposed robot employs a **distributed software architecture** in which computationally intensive tasks are executed on the **NVIDIA Jetson Orin Nano**, while deterministic real-time control is handled by the **ESP32CEU6 microcontroller**. This separation ensures high computational performance without compromising the real-time requirements of locomotion and hardware control.

The software architecture has been designed in a modular manner to improve maintainability, scalability, and ease of future development. Individual software modules perform independent tasks such as object detection, reinforcement learning, voice recognition, navigation, sensor processing, gait generation, and actuator control. These modules communicate through well-defined interfaces, allowing each subsystem to be developed, tested, and upgraded independently. Such a modular design simplifies debugging and enables future integration of additional sensors, robotic manipulators, cloud connectivity, and advanced autonomous navigation algorithms.

The high-level software executes on the NVIDIA Jetson Orin Nano running **Ubuntu 22.04 LTS** and **ROS2 Humble Hawksbill**. This processing unit is responsible for artificial intelligence and perception tasks including **YOLOv8-based object detection**, **Proximal Policy Optimization (PPO) reinforcement learning**, **offline voice recognition**, **navigation planning**, and overall system coordination. The Jetson continuously receives sensor information from the camera, microphone, and

embedded controller, processes this information using artificial intelligence algorithms, and generates motion commands according to the surrounding environment.

Low-level control functions are executed on the **ESP32CEU6**, which operates as the real-time embedded controller. The firmware continuously receives motion commands from the Jetson through UART communication and converts them into synchronized PWM signals for the servo motors using two PCA9685 servo driver modules. In addition to actuator control, the ESP32 is responsible for acquiring data from the GY-87 Inertial Measurement Unit (IMU), the VL53L0X Time-of-Flight sensor, monitoring battery status, executing safety routines, and maintaining deterministic timing during locomotion.

The robotic dog incorporates multiple software modules that operate simultaneously during robot operation. The **computer vision module** continuously analyzes camera images to detect humans, obstacles, staircases, and surrounding objects using the YOLOv8 deep learning model. The **voice recognition module** interprets spoken commands received through the I2S digital microphone and converts them into executable robot actions. The **navigation module** combines information from computer vision and onboard sensors to determine safe movement trajectories, while the **motion planning module** generates coordinated joint trajectories for walking, turning, rough-terrain traversal, and stair climbing.

Locomotion is achieved using a **12-degree-of-freedom (12-DOF)** quadruped mechanism consisting of three servo-actuated joints in each leg: the hip joint, thigh joint, and knee joint. The robot employs **Proximal Policy Optimization (PPO)** as the primary reinforcement learning algorithm to generate adaptive gait patterns. Instead of relying on predefined trajectories, the reinforcement learning controller continuously adjusts joint movements based on body orientation, terrain characteristics, and environmental conditions, allowing the robot to maintain stability while traversing uneven surfaces and climbing staircases with heights ranging from **12 cm to 15 cm**.

Sensor fusion forms another important component of the software architecture. Measurements obtained from the GY-87 IMU are combined with distance information from the VL53L0X sensor to estimate body orientation, detect external disturbances, and maintain dynamic balance during locomotion. This continuous feedback enables the robot to compensate for uneven terrain and maintain stable movement even when subjected to unexpected disturbances.

Communication between the software modules is achieved through multiple interfaces. ROS2 manages communication between artificial intelligence modules executing on the Jetson Orin Nano, while UART provides bidirectional communication between the Jetson and the ESP32. The embedded controller communicates with peripheral devices using I²C, SPI, and PWM protocols, ensuring efficient interaction with sensors, displays, and servo motors. This layered communication architecture separates high-level decision making from low-level hardware control, improving reliability and simplifying software maintenance.

The software architecture also incorporates several safety mechanisms to ensure reliable robot operation. Continuous monitoring of battery voltage, communication status, servo activity, and sensor feedback enables the embedded controller to detect abnormal operating conditions. If communication

is interrupted or unsafe conditions are detected, the firmware automatically halts robot movement and places the actuators into a safe state, preventing mechanical damage and improving operational safety.

Overall, the proposed software implementation provides an intelligent, modular, and scalable control framework capable of supporting autonomous locomotion, adaptive terrain traversal, human following, obstacle avoidance, offline voice interaction, and real-time environmental perception. By combining artificial intelligence, reinforcement learning, embedded control, and standardized communication protocols, the software architecture establishes a robust platform for the advanced capabilities demonstrated by the proposed robotic dog.

Table 5.1 Software Modules of the Proposed Robotic Dog

Software Module	Function
ROS2 Middleware	Communication between software nodes
YOLOv8	Object, human, and obstacle detection
PPO Controller	Adaptive locomotion and gait generation
Voice Recognition	Offline speech command processing
Navigation Module	Path planning and obstacle avoidance
Motion Planner	Joint trajectory generation
ESP32 Firmware	Real-time embedded control
Sensor Fusion	IMU and ToF data processing
Communication Module	UART, I ² C, SPI, USB, and PWM management
Safety Module	Fault detection and emergency handling

5.2 Software Development Environment

The software development environment of the proposed quadruped robotic dog was carefully designed to support real-time embedded control, artificial intelligence, computer vision, reinforcement learning, and autonomous navigation. Since the robotic platform consists of both high-level computing and low-level embedded control, multiple software tools and programming environments were integrated to facilitate efficient system development, testing, debugging, and deployment.

The high-level software was developed for the **NVIDIA Jetson Orin Nano**, which operates as the primary artificial intelligence processing unit. The Jetson runs **Ubuntu 22.04 LTS** as its operating

system, providing a stable Linux-based environment for robotics and machine learning applications. Ubuntu offers excellent compatibility with ROS2, OpenCV, CUDA, PyTorch, and various deep learning frameworks required for autonomous robotic applications.

The complete robotic software architecture was implemented using **ROS2 Humble Hawksbill**, which serves as the middleware responsible for communication between different software modules. ROS2 enables modular software development through independent nodes that exchange information using publishers, subscribers, services, and actions. This architecture allows perception, navigation, locomotion, voice recognition, and hardware communication modules to operate concurrently while maintaining synchronized data exchange.

The primary programming languages used in the development of the robotic dog are **Python** and **C++**. Python was selected for implementing artificial intelligence algorithms, computer vision applications, reinforcement learning models, and high-level robot behaviors due to its extensive machine learning ecosystem and rapid development capabilities. C++ was employed for performance-critical ROS2 nodes and embedded software modules where low execution latency and deterministic performance are required.

Computer vision functionalities were developed using the **OpenCV** library together with the **Ultralytics YOLOv8** framework. OpenCV provides efficient image acquisition, preprocessing, and visualization functions, while YOLOv8 performs real-time object detection for identifying humans, obstacles, and staircases. The integration of these libraries enables the robotic dog to perceive and interpret its surroundings with high computational efficiency.

Artificial intelligence models were developed using **PyTorch**, which provides a flexible deep learning framework for both object detection and reinforcement learning. The **Proximal Policy Optimization (PPO)** algorithm was implemented within the PyTorch environment to train and execute adaptive locomotion policies. The trained neural network is deployed directly on the NVIDIA Jetson Orin Nano, allowing the robot to generate stable walking patterns in real time without relying on external computing resources.

To accelerate deep learning inference, the Jetson Orin Nano utilizes **CUDA** and GPU acceleration. CUDA enables parallel execution of neural network computations on the embedded GPU, significantly reducing inference time for YOLOv8 object detection and PPO-based locomotion control. This hardware acceleration allows the robot to process sensor data and generate control decisions with minimal latency.

The embedded firmware for the **ESP32CEU6** microcontroller was developed using **ESP32CubeIDE**. This integrated development environment provides comprehensive support for peripheral configuration, firmware development, debugging, and firmware uploading. The ESP32 firmware manages real-time tasks including servo motor control, sensor acquisition, UART communication, and system monitoring.

The **PCA9685 PWM servo drivers** are controlled through firmware developed using the ESP32 Hardware Abstraction Layer (HAL) libraries. These libraries simplify peripheral initialization while

providing reliable communication with I²C devices and precise timing required for synchronized servo control.

Development and debugging of high-level software were performed using **Visual Studio Code (VS Code)**. VS Code provides integrated support for Python, C++, ROS2, Git, remote SSH access, and debugging utilities, allowing efficient software development directly on the Jetson Orin Nano. Version control was managed using **Git**, enabling systematic tracking of software modifications and collaborative development.

Simulation and algorithm validation were conducted prior to deployment on the physical robot whenever possible. Individual software modules, including object detection, voice recognition, and locomotion control, were tested independently before being integrated into the complete robotic system. This incremental development strategy reduced debugging complexity and improved overall software reliability.

Communication testing was performed using serial monitoring utilities and ROS2 command-line tools to verify correct data exchange between the NVIDIA Jetson Orin Nano and the ESP32 microcontroller. Sensor outputs, servo responses, and communication latency were continuously monitored throughout development to ensure reliable operation under real-time conditions.

The software development environment also incorporates comprehensive debugging and diagnostic tools. ROS2 visualization utilities, log files, terminal monitoring, and serial debugging interfaces were used to identify software errors, communication failures, and hardware synchronization issues during system integration. This structured development methodology significantly reduced implementation time and improved overall software robustness.

Overall, the selected software development environment provides a powerful and flexible platform for implementing intelligent robotic applications. The combination of Ubuntu Linux, ROS2, Python, C++, OpenCV, PyTorch, CUDA, ESP32CubeIDE, and Visual Studio Code enables efficient integration of artificial intelligence, embedded control, computer vision, reinforcement learning, and real-time communication into a unified software framework suitable for autonomous quadruped locomotion.

Table 5.2 Software Development Tools

Software Tool	Purpose
Ubuntu 22.04 LTS	Operating system for Jetson Orin Nano
ROS2 Humble Hawksbill	Robotics middleware
Python 3	Artificial intelligence and high-level programming

C++	Performance-critical software modules
OpenCV	Image acquisition and processing
Ultralytics YOLOv8	Real-time object detection
PyTorch	Deep learning and reinforcement learning
CUDA	GPU acceleration
ESP32CubeIDE	ESP32 firmware development
Visual Studio Code	Software development and debugging
Git	Version control

Table 5.3 Programming Languages Used

Language	Application
Python	YOLOv8, PPO, Voice Recognition, Navigation
C++	ROS2 Nodes and Performance-Critical Modules
C	ESP32 Firmware Development

5.3 Software Architecture

The software architecture of the proposed quadruped robotic dog has been developed using a modular and layered approach to achieve high reliability, scalability, maintainability, and real-time performance. The architecture integrates artificial intelligence, computer vision, reinforcement learning, embedded control, and hardware communication into a unified framework capable of supporting autonomous locomotion and intelligent decision-making. The modular organization allows each software component to perform a dedicated task while communicating efficiently with other subsystems through standardized interfaces.

The complete software system is divided into two primary computational layers. The **high-level processing layer**, executed on the **NVIDIA Jetson Orin Nano**, is responsible for computationally intensive operations such as object detection, reinforcement learning, navigation, and offline voice recognition. The **low-level embedded control layer**, implemented on the **ESP32CEU6**

microcontroller, performs deterministic real-time tasks including servo motor control, sensor acquisition, PWM generation, safety monitoring, and communication with peripheral devices. This distributed architecture ensures that artificial intelligence computations do not interfere with the timing requirements of the locomotion controller.

The Jetson Orin Nano executes **Ubuntu 22.04 LTS** together with **ROS2 Humble Hawksbill**, providing a middleware framework that coordinates communication between the different software modules. Individual functionalities are implemented as independent ROS2 nodes, allowing perception, locomotion, navigation, and communication processes to execute concurrently. This modular node-based architecture simplifies debugging, improves software maintainability, and allows future algorithms to be integrated with minimal modification to the existing system.

The perception subsystem forms the first stage of the software architecture. Images captured by the RGB camera are processed using the **YOLOv8 deep learning model** to identify humans, obstacles, staircases, and surrounding objects. Simultaneously, audio captured by the I2S digital microphone is processed by the offline speech recognition module to interpret voice commands. Distance measurements from the VL53L0X Time-of-Flight sensor and orientation data from the GY-87 IMU provide additional environmental information that supports navigation and balance control.

The outputs of the perception subsystem are transferred to the decision-making layer, where the robot determines the appropriate response according to the current operating mode. Voice commands initiate user-requested actions such as walking, stopping, turning, sitting, or following a person. During autonomous operation, detected obstacles and environmental features are analyzed to generate safe movement strategies while maintaining the desired navigation objective.

The locomotion subsystem is based on a **12-degree-of-freedom quadruped mechanism**, consisting of three independently actuated joints in each leg: the thigh joint, and knee joint. Motion generation is performed using a **Proximal Policy Optimization (PPO)** reinforcement learning controller that continuously calculates optimal joint positions according to the robot's body orientation, terrain characteristics, and navigation objectives. Unlike conventional predefined gait algorithms, the reinforcement learning policy adapts the robot's movement in real time, improving stability during rough-terrain traversal and stair climbing.

Once joint trajectories have been generated, motion commands are transmitted from the Jetson Orin Nano to the ESP32 microcontroller through a UART communication interface. The ESP32 interprets these commands and generates synchronized PWM signals using two PCA9685 servo driver modules. These PWM signals control the twelve MG996R servo motors responsible for locomotion and the two MG90S servo motors that actuate the head and tail mechanisms.

A dedicated sensor fusion module continuously monitors feedback from the GY-87 IMU and VL53L0X Time-of-Flight sensor. Roll, pitch, and yaw measurements obtained from the IMU are combined with obstacle distance information to estimate the robot's current posture and surrounding environment. This information enables the controller to compensate for external disturbances, maintain body balance, and adjust gait parameters during locomotion over uneven surfaces.

Safety has been integrated into the software architecture through continuous system monitoring. The ESP32 firmware supervises battery voltage, communication integrity, servo operation, and sensor status throughout robot operation. In the event of abnormal operating conditions, communication failure, or unstable body orientation, the embedded controller immediately executes emergency protection routines by stopping locomotion and placing the actuators in a safe position. These safety mechanisms protect both the robotic platform and nearby users.

The layered software architecture also facilitates future system expansion. Additional ROS2 nodes can be incorporated to support technologies such as LiDAR-based SLAM, GPS navigation, robotic arm manipulation, cloud robotics, multi-robot collaboration, and autonomous charging without requiring significant changes to the existing software framework. This scalability makes the proposed robotic dog suitable for continued research and future technological development.

Overall, the implemented software architecture successfully integrates perception, artificial intelligence, reinforcement learning, embedded control, and hardware communication into a cohesive robotic system. The combination of modular software design, distributed processing, ROS2 middleware, and real-time embedded control provides a reliable platform capable of supporting intelligent quadruped locomotion and autonomous operation in complex environments.

5.3.1 Software Layers

The software architecture is organized into five functional layers:

Application Layer

Responsible for user interaction and high-level robot functions, including:

- Voice command execution
- Human following
- Autonomous navigation
- Stair climbing
- Rough-terrain locomotion

Artificial Intelligence Layer

Implements intelligent algorithms such as:

- YOLOv8 object detection
- PPO reinforcement learning
- Decision-making logic

Motion Planning Layer

Generates coordinated locomotion commands for:

- Walking
- Turning
- Standing
- Sitting
- Climbing
- Obstacle avoidance

Embedded Control Layer

Executed on the ESP32 and responsible for:

- Servo control
- Sensor acquisition
- Communication
- Safety monitoring

Hardware Layer

Includes:

- Servo motors
- Sensors
- Camera
- Microphone
- OLED display
- Power management system

Table 5.4 Software Architecture Layers

Layer	Primary Function
Application Layer	User interaction and robot behavior
AI Layer	YOLOv8, PPO, decision making
Motion Planning Layer	Gait generation and navigation
Embedded Layer	Real-time control and communication
Hardware Layer	Sensors, actuators, and electronics

Table 5.5 Major Software Modules

Module	Function
Vision Module	Object and human detection
Voice Module	Offline speech recognition
Navigation Module	Path planning and obstacle avoidance
Locomotion Module	Walking and gait generation
Sensor Fusion Module	IMU and ToF processing
Communication Module	UART, I ² C, SPI, USB, PWM
Safety Module	Fault detection and emergency stop

5.4 ROS2 Framework Implementation

The proposed quadruped robotic dog utilizes the **Robot Operating System 2 (ROS2) Humble Hawksbill** as the primary middleware for integrating perception, artificial intelligence, locomotion, and communication into a unified software framework. ROS2 enables distributed processing by allowing independent software modules, known as **nodes**, to communicate efficiently through topics, services, and actions. This architecture simplifies software development while ensuring scalability, modularity, and reliable real-time communication between different robotic subsystems.

In the proposed system, ROS2 is executed on the **NVIDIA Jetson Orin Nano**, which serves as the high-level processing unit. The Jetson runs Ubuntu 22.04 LTS together with ROS2 Humble, providing a robust environment for coordinating artificial intelligence algorithms, sensor processing, navigation, and user interaction. Each major functionality of the robotic dog is implemented as an independent ROS2 node, allowing individual processes to execute concurrently without interfering with one another.

The **camera node** continuously acquires image frames from the RGB camera and publishes them to the ROS2 communication network. These images are received by the **YOLOv8 object detection node**, which identifies humans, obstacles, staircases, and other relevant objects within the robot's surroundings. The detected objects are then published as structured detection messages, allowing other software modules to utilize this information for navigation and decision making.

Simultaneously, the **voice recognition node** captures audio from the I2S digital microphone and performs offline speech recognition directly on the Jetson Orin Nano. Once a valid command is recognized, the interpreted command is published to the ROS2 network. The navigation and

locomotion modules subscribe to this information and determine the corresponding robot behavior, such as walking, stopping, turning, or initiating human-following mode.

The **sensor interface node** receives orientation data from the GY-87 Inertial Measurement Unit and distance measurements from the VL53L0X Time-of-Flight sensor through the ESP32 microcontroller. These measurements are published periodically and utilized by the locomotion controller to maintain balance, estimate body posture, and detect nearby obstacles. Continuous sensor feedback enables adaptive locomotion across uneven terrain and improves stability during stair climbing.

The **locomotion node** serves as the central decision-making component for robot movement. It receives information from the object detection, sensor interface, navigation, and voice recognition nodes before generating desired body motions. Instead of producing servo commands directly, the locomotion node calculates desired joint positions using the reinforcement learning controller and transmits these commands to the communication node.

Communication between the Jetson Orin Nano and the ESP32 is handled by a dedicated **UART communication node**. This node packages the desired joint angles into serial data packets and transmits them to the embedded controller. The ESP32 firmware interprets the received commands, generates synchronized PWM signals using the PCA9685 servo drivers, and controls the twelve MG996R locomotion servos together with the two MG90S servos responsible for head and tail movement.

The ROS2 framework also manages communication timing between software modules. Each node executes independently while exchanging data asynchronously through publishers and subscribers. This architecture prevents computationally intensive operations, such as YOLOv8 inference, from blocking other critical processes including locomotion control and voice recognition. As a result, perception, navigation, and movement can operate simultaneously without degrading overall system performance.

Several ROS2 topics were defined to organize communication between software modules. Camera images, object detection results, voice commands, navigation targets, sensor measurements, locomotion commands, and system status information are transmitted through dedicated topics. The modular organization of these communication channels simplifies software debugging and allows additional functionalities to be incorporated without redesigning the existing architecture.

ROS2 services are employed for operations that require immediate responses, such as system initialization, calibration procedures, and hardware diagnostics. During startup, service calls initialize the camera, communication interfaces, sensors, and locomotion controller before normal operation begins. This ensures that all software modules are synchronized prior to executing autonomous behaviors.

The framework also incorporates comprehensive logging and diagnostic capabilities. ROS2 logging utilities continuously record node activity, communication events, and software warnings, providing valuable information during system development and debugging. These diagnostic tools facilitate rapid identification of software faults while improving the reliability of the complete robotic system.

The modular nature of ROS2 significantly enhances the future scalability of the proposed robotic dog. New functionalities such as LiDAR-based SLAM, GPS navigation, robotic arm control, cloud connectivity, autonomous charging, or multi-robot coordination can be incorporated by simply adding additional ROS2 nodes without modifying the existing software architecture. This flexibility makes the platform highly suitable for future research and advanced robotic applications.

Overall, the implementation of ROS2 provides a reliable communication infrastructure that coordinates perception, artificial intelligence, navigation, locomotion, and embedded control into a unified robotic system. The distributed node-based architecture enables efficient execution of multiple software processes while maintaining real-time responsiveness, modularity, and expandability.

5.4.1 ROS2 Nodes Used in the Proposed System

The implemented ROS2 framework consists of the following primary nodes:

- Camera Node
- YOLOv8 Detection Node
- Voice Recognition Node
- Sensor Interface Node
- Navigation Node
- PPO Locomotion Node
- UART Communication Node
- System Monitoring Node

Each node performs a dedicated function while communicating with other nodes through ROS2 topics.

Table 5.6 ROS2 Nodes

ROS2 Node	Function
Camera Node	Captures images from RGB camera
YOLOv8 Node	Detects humans, obstacles, and stairs
Voice Recognition Node	Processes offline speech commands
Sensor Interface Node	Receives IMU and ToF sensor data
Navigation Node	Generates movement objectives
PPO Locomotion Node	Computes joint trajectories
UART Communication Node	Sends commands to ESP32

System Monitoring Node	Monitors software and hardware status
------------------------	---------------------------------------

5.5 Embedded Firmware Implementation (ESP32)

The **ESP32CEU6 microcontroller** serves as the real-time embedded controller of the proposed quadruped robotic dog. While the NVIDIA Jetson Orin Nano performs computationally intensive tasks such as object detection, reinforcement learning, and voice recognition, the ESP32 is responsible for deterministic hardware control. This separation ensures that high-level artificial intelligence processing does not affect the precise timing required for servo actuation and sensor acquisition.

The firmware was developed using **ESP32CubeIDE** in the C programming language. The implementation follows a modular architecture in which independent firmware modules manage servo control, sensor acquisition, communication, safety monitoring, and system diagnostics. This organization improves software readability, simplifies debugging, and allows individual modules to be updated without affecting the remainder of the embedded software.

After power-up, the firmware performs a complete system initialization sequence. All communication peripherals, timers, GPIO pins, I²C interfaces, UART channels, and system clocks are configured before entering the main execution loop. The initialization routine also verifies communication with the PCA9685 servo driver modules and connected sensors. If a hardware initialization failure is detected, the robot remains in a safe standby mode until the fault is resolved.

The ESP32 continuously communicates with the **NVIDIA Jetson Orin Nano** through a UART serial interface. High-level motion commands generated by the Jetson are transmitted as structured data packets containing the desired joint positions, operating mode, and motion parameters. After validating the received packet, the ESP32 converts the requested joint positions into PWM pulse widths and transmits these values to the dual PCA9685 servo driver modules via the I²C communication bus.

The locomotion system consists of **twelve MG996R servo motors**, with **three actuators assigned to each leg**: the hip joint, thigh joint, and knee joint. Two additional MG90S servo motors control the movement of the robot's head and tail. The ESP32 synchronizes all fourteen actuators to ensure smooth and coordinated motion during standing, walking, turning, rough-terrain traversal, and stair climbing.

To maintain stable locomotion, the firmware executes a cyclic control loop at a fixed update frequency. During each cycle, the controller receives new motion commands from the Jetson, reads sensor data, updates servo positions, verifies communication status, and monitors system safety.

Executing these operations at regular intervals ensures deterministic timing and prevents irregular servo movement that could compromise robot stability.

The **GY-87 Inertial Measurement Unit (IMU)** is connected to the ESP32 through the I²C interface. The firmware periodically acquires acceleration, angular velocity, and orientation data, which are forwarded to the Jetson Orin Nano for higher-level balance estimation and locomotion control. Continuous IMU monitoring also enables the firmware to detect excessive body inclination and initiate safety procedures when necessary.

The **VL53L0X Time-of-Flight (ToF) sensor** provides distance measurements that assist with obstacle detection and stair recognition. The ESP32 periodically samples the sensor and transmits the measured distances to the Jetson through the UART interface. This information complements the computer vision system during autonomous navigation.

Servo control is implemented using two **PCA9685 16-channel PWM driver modules**, which generate accurate PWM signals independently of the ESP32 processor. Offloading PWM generation to dedicated hardware reduces processor workload and ensures stable servo timing even when multiple communication and sensing tasks are executed simultaneously. Each servo receives an individual pulse width corresponding to the desired joint angle calculated by the locomotion controller.

The firmware incorporates several safety mechanisms to protect both the robot and its hardware components. Communication with the Jetson is continuously monitored using a timeout mechanism. If valid motion commands are not received within the specified interval, the ESP32 immediately stops updating servo positions and places the robot into a safe standing posture. Battery voltage and communication status are also monitored throughout operation, allowing early detection of abnormal operating conditions.

To facilitate software debugging, the firmware provides diagnostic information through the UART interface. Servo positions, sensor measurements, communication status, and system errors can be monitored in real time during development. This diagnostic capability significantly simplified software integration and reduced troubleshooting time during hardware testing.

The modular firmware architecture also supports future hardware expansion. Additional sensors, communication interfaces, or actuator systems can be integrated by extending the existing firmware modules without restructuring the entire software system. This flexibility makes the ESP32 firmware suitable for future research involving LiDAR sensors, GPS receivers, robotic manipulators, or advanced environmental sensing.

Overall, the ESP32 firmware provides reliable real-time control of the robotic dog by coordinating actuator movement, sensor acquisition, communication, and safety monitoring. Its deterministic execution, modular structure, and efficient peripheral management form the foundation upon which the robot's intelligent behaviors are implemented.

5.5.1 Firmware Modules

The ESP32 firmware is divided into the following functional modules:

- **System Initialization Module**
- **UART Communication Module**
- **I²C Communication Module**
- **Servo Control Module**
- **Sensor Acquisition Module**
- **Safety Monitoring Module**
- **Diagnostic Module**

Each module performs an independent task while sharing information through the main control loop.

5.5.2 Firmware Execution Sequence

The firmware follows the execution sequence below:

1. Initialize system clock and peripherals.
2. Configure GPIO, UART, I²C, and timers.
3. Initialize PCA9685 servo driver modules.
4. Verify communication with sensors.
5. Move servos to the default standing posture.
6. Enter the main control loop.
7. Receive commands from the Jetson Orin Nano.
8. Read IMU and ToF sensor data.
9. Generate PWM signals for the servo motors.
10. Monitor battery voltage and communication status.
11. Repeat the control loop continuously.

Table 5.8 ESP32 Firmware Modules

Module	Function
System Initialization	Configures peripherals and communication interfaces
UART Module	Exchanges data with Jetson Orin Nano
I ² C Module	Controls PCA9685, IMU, and ToF sensor

Servo Control Module	Generates joint commands for all servos
Sensor Module	Reads IMU and ToF sensor measurements
Safety Module	Monitors communication and battery status
Diagnostic Module	Reports system information and faults

Table 5.9 ESP32 Peripheral Utilization

Peripheral	Purpose
GPIO	General hardware control
UART	Communication with Jetson Orin Nano
I ² C	PCA9685, IMU, and ToF sensor interface
Timers	Control loop timing
PWM (via PCA9685)	Servo motor control
Interrupts	Event handling and communication

5.6 Artificial Intelligence Processing on NVIDIA Jetson Orin Nano

The **NVIDIA Jetson Orin Nano** serves as the primary artificial intelligence and high-level processing unit of the proposed quadruped robotic dog. It is responsible for executing computationally intensive algorithms that require significant processing power, including computer vision, reinforcement learning, offline voice recognition, autonomous navigation, and decision-making. By separating these tasks from the low-level embedded controller, the system achieves both high computational performance and reliable real-time hardware control.

The Jetson Orin Nano operates on **Ubuntu 22.04 LTS** with **ROS2 Humble Hawksbill**, providing a robust software platform for robotics and artificial intelligence applications. All high-level software modules execute as independent ROS2 nodes, enabling parallel execution and efficient communication between perception, navigation, locomotion, and user interaction subsystems.

The processing pipeline begins with data acquisition from the robot's sensors. The RGB camera continuously captures images of the surrounding environment, while the I2S digital microphone records user voice commands. Simultaneously, the ESP32 transmits orientation information from the GY-87 IMU and distance measurements from the VL53L0X Time-of-Flight sensor through the UART communication interface. These data streams are synchronized and processed by the Jetson to maintain an accurate understanding of both the robot's internal state and its external environment.

The **computer vision module** utilizes the YOLOv8 deep learning model to detect humans, obstacles, staircases, and other objects in real time. The detected objects are used by the navigation module to determine safe movement directions and by the human-following algorithm to maintain a desired distance from the target person. The vision module continuously publishes detection results through ROS2 topics, allowing other software modules to access the processed information without direct coupling.

The locomotion intelligence is provided by the **Proximal Policy Optimization (PPO)** reinforcement learning controller. Rather than relying on predefined walking trajectories, the trained PPO model receives the robot's current state—including body orientation, terrain information, and navigation objectives—and generates appropriate joint commands for the twelve leg actuators. This adaptive approach enables the robot to maintain stable locomotion over uneven terrain, negotiate obstacles, and climb staircases with heights between **12 cm and 15 cm**.

The Jetson also executes the **offline voice recognition module**, allowing the robot to respond to spoken commands without requiring an internet connection. Once a command is recognized, it is converted into a high-level robot action such as standing, walking, turning, sitting, following a person, or initiating stair-climbing mode. Offline processing reduces communication latency, improves user privacy, and enables reliable operation in environments where network connectivity is unavailable.

A dedicated **decision-making module** coordinates information received from the perception, navigation, and voice recognition systems. This module determines the current operating mode of the robot and selects the appropriate locomotion strategy. For example, if a human is detected and the "follow me" command has been activated, the navigation system generates a trajectory that maintains a safe following distance while avoiding nearby obstacles. Similarly, when a staircase is identified, the controller switches from normal walking to the stair-climbing gait.

The Jetson continuously communicates with the ESP32 through a high-speed UART interface. Instead of transmitting raw sensor data, the Jetson sends processed motion commands that specify the desired joint positions and operating mode. This communication strategy reduces data traffic while allowing the embedded controller to focus exclusively on deterministic hardware control.

To achieve real-time performance, the Jetson utilizes its integrated **GPU** to accelerate deep learning inference. CUDA-enabled processing significantly reduces the execution time of the YOLOv8 object detection network and the PPO locomotion policy, allowing the robot to respond rapidly to changes in the environment. GPU acceleration also enables multiple artificial intelligence algorithms to execute simultaneously without significantly affecting overall system responsiveness.

The modular software architecture implemented on the Jetson simplifies future development. Additional ROS2 nodes can be incorporated to support advanced capabilities such as LiDAR-based SLAM, GPS-assisted navigation, facial recognition, autonomous charging, robotic manipulation, cloud robotics, or multi-robot collaboration. Since each function operates independently within the ROS2 framework, new features can be integrated with minimal modification to the existing software.

Extensive testing was performed to verify the stability and reliability of the Jetson-based software. Individual AI modules were validated independently before complete system integration. The successful interaction between computer vision, reinforcement learning, voice recognition, and navigation demonstrates the effectiveness of the distributed software architecture in supporting intelligent autonomous behavior.

Overall, the NVIDIA Jetson Orin Nano provides the computational capability required for advanced robotic intelligence. Its ability to execute multiple artificial intelligence algorithms concurrently enables the robotic dog to perceive its surroundings, interpret user commands, make autonomous decisions, and generate adaptive locomotion commands while maintaining efficient communication with the embedded control system.

5.6.1 AI Modules Executed on the Jetson Orin Nano

The Jetson Orin Nano executes the following major artificial intelligence modules:

- YOLOv8 Object Detection
- PPO Reinforcement Learning Controller
- Offline Voice Recognition
- Navigation and Path Planning
- Decision-Making Module
- ROS2 Communication Manager
- Human-Following Algorithm
- System Monitoring Module

These modules operate concurrently and exchange information through ROS2 topics.

5.6.2 AI Processing Pipeline

The artificial intelligence processing sequence is summarized below:

1. Capture images from the RGB camera.
2. Acquire voice input from the I2S microphone.
3. Receive IMU and ToF sensor data from the ESP32.
4. Perform YOLOv8 object detection.
5. Execute offline speech recognition.
6. Determine the current operating mode.

- 7. Generate locomotion commands using the PPO controller.
- 8. Transmit joint commands to the ESP32.
- 9. Receive feedback from the embedded controller.
- 10. Repeat the processing cycle continuously.

Table 5.10 AI Modules Executed on Jetson Orin Nano

AI Module	Function
YOLOv8 Detection	Detect humans, obstacles, and staircases
PPO Controller	Generate adaptive locomotion commands
Voice Recognition	Process offline speech commands
Navigation	Generate safe movement trajectories
Decision-Making	Select robot operating mode
ROS2 Manager	Coordinate communication between nodes
Human Following	Maintain a safe distance from the target
System Monitoring	Monitor AI module performance

Table 5.11 Jetson Hardware Utilization

Hardware Resource	Primary Task
CPU	ROS2 nodes, navigation, decision making
GPU	YOLOv8 inference and PPO execution
RAM	Neural network models and sensor data
USB Interface	RGB camera communication
UART	Communication with ESP32
Storage	Operating system, software, and AI models

5.7 Object Detection Using YOLOv8

The proposed quadruped robotic dog utilizes the **YOLOv8 (You Only Look Once Version 8)** deep learning model as its primary computer vision algorithm for real-time environmental perception. The object detection system enables the robot to recognize humans, obstacles, staircases, furniture, and other relevant objects while navigating in indoor and outdoor environments. By providing continuous visual feedback to the navigation and locomotion modules, YOLOv8 plays a critical role in autonomous operation and intelligent human–robot interaction.

The YOLOv8 model is deployed on the **NVIDIA Jetson Orin Nano**, where it performs real-time image inference using the embedded GPU. The RGB camera continuously captures images of the surrounding environment, which are transferred directly to the object detection module. Each captured frame is preprocessed before being supplied to the trained YOLOv8 neural network for inference. The detection results include object classes, confidence scores, and bounding box coordinates, which are then published through the ROS2 communication framework.

Unlike conventional image processing techniques that rely on manually designed features, YOLOv8 performs object detection using a deep convolutional neural network capable of simultaneously identifying multiple objects within a single image. The single-stage detection architecture provides high processing speed while maintaining accurate object localization, making it particularly suitable for autonomous robotic applications that require rapid environmental perception.

The object detection module continuously analyzes the robot's surroundings during operation. When a human is detected, the detected position is transmitted to the navigation module, allowing the robot to initiate or maintain human-following behavior. If an obstacle is identified within the planned trajectory, the navigation module generates an alternative path to prevent collisions. Similarly, the detection of staircases triggers the stair-climbing locomotion mode, enabling the robot to transition from normal walking to adaptive climbing behavior.

To improve computational efficiency, image frames are resized before inference while maintaining sufficient resolution for reliable object detection. GPU acceleration provided by the Jetson Orin Nano significantly reduces inference latency, enabling the robotic dog to perform object detection continuously during locomotion without interrupting other artificial intelligence processes such as reinforcement learning and voice recognition.

The detection results generated by YOLOv8 are not used independently but are integrated with information obtained from the GY-87 IMU and the VL53L0X Time-of-Flight sensor. The combination of visual information and sensor measurements improves environmental awareness, reduces false detections, and enhances navigation reliability. For example, if an obstacle is detected visually, the ToF sensor provides additional distance information that assists the navigation controller in determining a safe avoidance maneuver.

The object detection module operates continuously throughout robot operation. Each inference cycle consists of image acquisition, preprocessing, neural network inference, post-processing, and

communication of detection results through ROS2 topics. The navigation and locomotion modules subscribe to these topics and utilize the received information to update the robot's movement strategy in real time.

The modular implementation of YOLOv8 allows additional object classes to be incorporated without significant modification to the software architecture. Future versions of the robotic dog may include recognition of doors, charging stations, traffic signs, emergency exits, tools, or other application-specific objects simply by retraining the detection model with additional datasets.

During software development, the object detection system was evaluated under different lighting conditions and viewing angles to verify its robustness. The use of GPU acceleration together with the efficient YOLOv8 architecture enabled stable real-time inference suitable for autonomous robotic operation. Continuous testing confirmed reliable communication between the perception module, navigation controller, and embedded control system.

Overall, the implementation of YOLOv8 provides the robotic dog with reliable visual perception capabilities required for autonomous navigation, obstacle avoidance, human following, and stair detection. The integration of deep learning, GPU acceleration, and ROS2 communication establishes an efficient computer vision framework capable of supporting intelligent behavior in dynamic environments.

5.7.1 YOLOv8 Processing Pipeline

The object detection module performs the following sequence during each processing cycle:

1. Capture an image from the RGB camera.
2. Resize and preprocess the image.
3. Execute YOLOv8 inference on the Jetson GPU.
4. Identify object classes.
5. Calculate confidence scores.
6. Generate bounding boxes.
7. Publish detection results through ROS2 topics.
8. Update navigation and locomotion modules.
9. Repeat continuously during robot operation.

5.7.2 Detected Object Classes

The implemented YOLOv8 model is configured to detect objects relevant to the robotic dog's operation, including:

- Humans
- Staircases
- Chairs
- Tables
- Doors
- Walls
- Obstacles
- Boxes
- Bags
- Indoor furniture

Additional object classes can be incorporated by retraining the detection model.

Table 5.12 YOLOv8 Detection Pipeline

Processing Stage	Function
Image Acquisition	Capture image from RGB camera
Image Preprocessing	Resize and normalize image
Neural Network Inference	Execute YOLOv8 model
Object Classification	Identify detected objects
Localization	Generate bounding boxes
Confidence Estimation	Calculate detection confidence
ROS2 Publishing	Transmit detection results
Navigation Update	Assist path planning

Table 5.13 Object Detection Applications

Detected Object	Robot Action
Human	Human-following mode
Staircase	Activate stair-climbing gait

Obstacle	Avoid obstacle
Chair	Navigate around object
Table	Recalculate path
Wall	Prevent collision
Door	Continue navigation through opening
Unknown Object	Slow down and monitor

5.8 Reinforcement Learning-Based Locomotion Control

Reinforcement learning forms the foundation of the autonomous locomotion system implemented in the proposed quadruped robotic dog. Instead of relying on predefined walking trajectories or manually designed gait equations, the robot employs **Proximal Policy Optimization (PPO)** to generate adaptive leg movements based on its interaction with the surrounding environment. This learning-based control strategy enables the robot to maintain stable locomotion while traversing uneven terrain, climbing staircases, avoiding obstacles, and recovering from external disturbances.

The locomotion controller was designed to continuously observe the robot's current state, evaluate environmental conditions, and determine the optimal joint movements required to accomplish the desired task. During each control cycle, the reinforcement learning agent receives sensory information describing the robot's posture, orientation, and navigation objective. Based on these observations, the trained neural network predicts the most suitable joint commands for the twelve locomotion servos, allowing the robot to adapt its gait according to changing environmental conditions.

Unlike conventional quadruped robots that execute predefined walking sequences, the reinforcement learning controller continuously adjusts the movement of each leg in real time. This adaptive behavior improves balance, increases walking stability, and enables the robot to negotiate irregular terrain that cannot be accurately modeled using fixed mathematical trajectories. The learning-based controller also improves robustness against unexpected disturbances, allowing the robot to maintain locomotion even when small slips or body inclinations occur.

The proposed robotic dog utilizes the **Proximal Policy Optimization (PPO)** algorithm because of its stability, computational efficiency, and excellent performance in continuous control problems. PPO belongs to the family of policy-gradient reinforcement learning algorithms and optimizes the locomotion policy through gradual updates that prevent excessive changes in robot behavior during

training. This characteristic makes PPO particularly suitable for quadruped locomotion where unstable policy updates may cause the robot to lose balance.

The reinforcement learning policy operates on the **NVIDIA Jetson Orin Nano**, where the trained neural network performs inference in real time. The output generated by the policy consists of desired joint positions for the hip, thigh, and knee joints of each leg. These commands are transmitted to the ESP32 microcontroller through UART communication, where they are converted into synchronized PWM signals that drive the twelve MG996R servo motors.

The locomotion controller continuously receives feedback from the GY-87 Inertial Measurement Unit and the VL53L0X Time-of-Flight sensor. Orientation measurements including roll, pitch, and angular velocity provide information about body stability, while distance measurements assist in identifying terrain changes and approaching obstacles. This continuous feedback enables the reinforcement learning policy to generate adaptive joint movements that improve stability during walking, turning, and stair climbing.

Several locomotion behaviors are supported by the reinforcement learning framework. During normal walking, the policy generates coordinated leg trajectories that maintain forward movement while minimizing unnecessary body oscillation. When rough terrain is encountered, the controller modifies joint positions to improve stability and maintain foot contact with the ground. During stair climbing, the policy increases leg lift and body clearance to enable the robot to climb steps with heights ranging from **12 cm to 15 cm**. Turning maneuvers are achieved by adjusting stride lengths independently for the left and right legs, allowing smooth changes in heading direction.

To improve the reliability of locomotion, the reinforcement learning controller operates together with traditional safety constraints implemented within the ESP32 firmware. Joint angle limits, servo velocity limits, and communication timeout mechanisms prevent unsafe robot behavior even if abnormal control commands are generated. This hybrid approach combines the adaptability of reinforcement learning with the reliability of deterministic embedded safety mechanisms.

The reinforcement learning framework was designed to be modular, allowing future improvements through additional training or algorithm replacement. Alternative reinforcement learning techniques such as **Soft Actor-Critic (SAC)**, **Deep Deterministic Policy Gradient (DDPG)**, or **Twin Delayed Deep Deterministic Policy Gradient (TD3)** can be incorporated with minimal modification to the overall software architecture. This flexibility supports continued research into intelligent quadruped locomotion.

The implementation of PPO significantly improves the adaptability of the robotic dog compared to conventional rule-based controllers. Rather than following a fixed gait regardless of terrain conditions, the robot continuously modifies its movement according to sensory feedback and navigation objectives. This capability enables smoother locomotion, improved obstacle negotiation, enhanced stair-climbing performance, and greater robustness in unstructured environments.

Overall, reinforcement learning provides the robotic dog with an intelligent locomotion controller capable of generating adaptive gait patterns for real-world operation. By combining PPO with

continuous sensor feedback, embedded safety mechanisms, and GPU-accelerated inference on the NVIDIA Jetson Orin Nano, the proposed system achieves stable and efficient quadruped locomotion suitable for autonomous navigation across a variety of terrains.

5.8.1 Reinforcement Learning Workflow

The locomotion controller follows the sequence below:

1. Receive sensor information from the IMU and ToF sensor.
2. Estimate the robot's current state.
3. Receive the navigation objective.
4. Process the observation using the PPO neural network.
5. Predict the desired joint positions.
6. Transmit joint commands to the ESP32.
7. Generate PWM signals for the servo motors.
8. Execute robot movement.
9. Receive updated sensor feedback.
10. Repeat the control loop continuously.

5.8.2 Observation Space

The PPO policy receives the following observations during each control cycle:

- Roll angle
- Pitch angle
- Yaw angle
- Angular velocity
- Linear acceleration
- Current joint positions
- Previous joint positions
- Robot velocity
- Desired walking direction
- Obstacle distance (VL53L0X)
- Current operating mode
- Body orientation

These observations provide sufficient information for the policy to estimate the current state of the robot and generate appropriate locomotion commands.

5.8.3 Action Space

The action generated by the PPO controller consists of desired joint positions for the twelve locomotion joints:

- Front Left Hip
- Front Left Thigh
- Front Left Knee
- Front Right Hip
- Front Right Thigh
- Front Right Knee
- Rear Left Hip
- Rear Left Thigh
- Rear Left Knee
- Rear Right Hip
- Rear Right Thigh
- Rear Right Knee

The ESP32 converts these desired joint positions into PWM signals for the servo motors.

5.8.4 Reward Function

The reinforcement learning agent is trained to maximize a reward function that encourages stable and efficient locomotion.

The reward consists of several objectives:

- Maintain forward movement.
- Keep the body balanced.
- Minimize excessive roll and pitch.
- Reduce energy consumption.
- Avoid slipping.
- Maintain smooth joint motion.
- Successfully climb stairs.
- Avoid collisions.
- Reach navigation targets.

A simplified reward formulation can be expressed as:

$$R = w_1 R_{\text{forward}} + w_2 R_{\text{balance}} + w_3 R_{\text{stability}} + w_4 R_{\text{energy}} + w_5 R_{\text{terrain}} - w_6 R_{\text{collision}}$$

where:

- R_{forward} rewards forward progress.
- R_{balance} rewards upright posture.
- $R_{\text{stability}}$ penalizes oscillation.
- R_{energy} encourages efficient actuator usage.
- R_{terrain} rewards successful traversal of rough terrain and stairs.
- $R_{\text{collision}}$ penalizes collisions with obstacles.

Table 5.14 PPO Observation Variables

Observation	Source
Roll, Pitch, Yaw	GY-87 IMU
Angular Velocity	GY-87 IMU
Linear Acceleration	GY-87 IMU
Joint Angles	Servo Feedback / Command State
Obstacle Distance	VL53L0X ToF
Motion Goal	Navigation Module

5.9 Voice Recognition and Human–Robot Interaction

The proposed quadruped robotic dog incorporates an **offline voice recognition system** that enables users to interact with the robot through natural spoken commands. Voice control provides an intuitive and hands-free method of communication, allowing users to operate the robotic dog without requiring a remote controller, keyboard, or mobile application. By processing speech directly on the **NVIDIA Jetson Orin Nano**, the robot remains fully functional even in environments where internet connectivity is unavailable.

The voice recognition system consists of an **I2S digital microphone**, a speech recognition module running on the Jetson Orin Nano, a command interpretation module, and the locomotion controller. The microphone continuously captures audio from the surrounding environment and streams the digital audio signal to the Jetson for processing. The captured speech is filtered to reduce background noise before being analyzed by the speech recognition engine.

Unlike cloud-based voice assistants, the proposed system performs **offline speech recognition**, ensuring low latency, improved privacy, and reliable operation in remote locations. This approach eliminates dependence on external servers and allows the robotic dog to respond immediately to user commands. The recognized speech is converted into text and compared against a predefined command dictionary to determine the intended robot action.

Once a valid command is identified, the command interpretation module generates a corresponding high-level instruction that is transmitted to the navigation and locomotion modules through the ROS2 communication framework. These modules determine the appropriate motion strategy and generate joint commands for the embedded controller. The ESP32 then converts the received commands into synchronized PWM signals that actuate the twelve MG996R servo motors responsible for locomotion and the two MG90S servos controlling the head and tail.

The implemented voice recognition system supports a variety of operational commands that allow users to control the robot during normal operation. Commands such as **"walk forward," "stop," "turn left," "turn right," "sit," "stand," "follow me,"** and **"climb stairs"** enable the user to switch between different operating modes without manual intervention. Additional commands may be incorporated in future software updates by expanding the command database.

To improve reliability, the command interpretation module verifies the confidence score produced by the speech recognition engine before executing any action. Commands with confidence values below the predefined threshold are ignored to reduce the probability of incorrect robot behavior caused by background noise or incomplete speech. This validation process enhances operational safety and improves the overall user experience.

The voice recognition module operates concurrently with the object detection and navigation systems. While the robot is following a user or navigating autonomously, it continues to listen for new commands without interrupting ongoing locomotion. This parallel execution is achieved through the ROS2 node architecture, where the voice recognition node publishes recognized commands independently of other software modules.

Human–robot interaction is further enhanced through coordinated head movement and visual feedback. When a user issues a voice command, the head mechanism may rotate toward the detected speaker, improving the perception of interaction. The OLED display provides immediate status information by indicating the currently recognized command or operating mode, allowing users to confirm that the robot has correctly interpreted their instructions.

Safety remains an important consideration in voice-controlled operation. Certain commands, such as emergency stop, receive the highest execution priority and immediately override all ongoing robot

activities. Furthermore, locomotion commands are only executed after confirming that the robot is in a stable operating condition and that no critical hardware faults have been detected by the ESP32 firmware.

The modular design of the voice recognition system allows future expansion through natural language processing techniques and multilingual support. Future versions of the robotic dog may include conversational interaction, speaker identification, personalized command sets, and integration with large language models to enable more advanced human–robot communication.

Overall, the offline voice recognition system significantly improves the usability and accessibility of the proposed robotic dog. By combining speech recognition, ROS2 communication, intelligent command interpretation, and real-time embedded control, the robot is capable of responding naturally to spoken instructions while maintaining reliable and safe autonomous operation.

5.9.1 Voice Command Processing Pipeline

The voice interaction system performs the following sequence during robot operation:

1. Capture speech using the I2S digital microphone.
 2. Apply noise reduction and audio preprocessing.
 3. Perform offline speech recognition.
 4. Convert speech into text.
 5. Compare the recognized text with the command database.
 6. Validate the recognition confidence.
 7. Publish the command through ROS2.
 8. Generate the corresponding robot action.
 9. Transmit motion commands to the ESP32.
 10. Execute the requested robot behavior.
-

5.9.2 Supported Voice Commands

The robotic dog supports the following voice commands:

- Stand
- Sit
- Walk Forward
- Walk Backward

- Turn Left
- Turn Right
- Stop
- Follow Me
- Climb Stairs
- Return Home
- Sleep Mode
- Wake Up

Additional commands can be added by updating the command recognition database.

Table 5.16 Voice Recognition Modules

Module	Function
I2S Digital Microphone	Captures voice input
Audio Preprocessing	Noise reduction and filtering
Speech Recognition Engine	Converts speech to text
Command Interpreter	Matches recognized commands
ROS2 Voice Node	Publishes command messages
Navigation Module	Determines required movement
ESP32 Controller	Executes locomotion commands

Table 5.17 Supported Voice Commands

Voice Command	Robot Action
Stand	Move to standing posture
Sit	Move to sitting posture
Walk Forward	Begin forward locomotion

Walk Backward	Move backward
Turn Left	Rotate left
Turn Right	Rotate right
Stop	Halt all movement
Follow Me	Activate human-following mode
Climb Stairs	Switch to stair-climbing gait
Return Home	Navigate to the predefined home position
Sleep Mode	Enter low-power standby mode
Wake Up	Resume normal operation

5.10 Motion Planning and Gait Generation

Motion planning is one of the most critical functions of the proposed quadruped robotic dog, as it directly determines how the robot moves safely and efficiently in different environments. The motion planning system converts high-level navigation objectives into coordinated joint movements that enable stable walking, turning, rough-terrain traversal, stair climbing, and human-following behavior. The implemented motion planning framework combines artificial intelligence, reinforcement learning, real-time sensor feedback, and embedded control to generate adaptive locomotion while maintaining body stability.

The proposed robotic dog employs a **12-degree-of-freedom locomotion mechanism**, with each leg consisting of three independently actuated joints: the hip, thigh, and knee. Coordinated control of these twelve joints enables the robot to generate smooth and balanced movement across various operating conditions. Unlike conventional robots that rely on predefined gait tables, the proposed system dynamically adjusts joint trajectories based on sensory feedback and navigation requirements.

The motion planning process begins with the navigation module running on the NVIDIA Jetson Orin Nano. Based on information received from the object detection module, voice recognition system, and onboard sensors, the navigation module determines the desired movement objective. These objectives may include forward walking, turning, obstacle avoidance, human following, or stair climbing. The desired motion is then transmitted to the reinforcement learning controller, which calculates appropriate joint trajectories for each leg.

The **Proximal Policy Optimization (PPO)** controller generates desired joint positions for the hip, thigh, and knee joints using the robot's current observation state. These joint commands are

transmitted to the ESP32 through the UART communication interface. The embedded controller converts the received joint positions into PWM signals using the PCA9685 servo driver modules, allowing synchronized control of all twelve locomotion servos.

To ensure smooth locomotion, motion planning operates continuously in a closed-loop control system. During every control cycle, the robot receives updated information regarding body orientation, obstacle locations, terrain characteristics, and navigation objectives. The reinforcement learning controller uses this feedback to modify joint trajectories in real time, allowing the robot to adapt to changes in the surrounding environment while maintaining dynamic stability.

During **normal walking**, the controller generates coordinated leg movements that maintain continuous body support while minimizing unnecessary oscillations. The robot follows a stable gait sequence in which diagonal leg pairs move in coordination, providing efficient forward locomotion and improved balance. The movement of each leg is synchronized to ensure smooth weight transfer between supporting limbs.

When a turning command is received, the locomotion controller modifies the stride length and joint trajectories of the left and right legs independently. Differential leg movement enables the robot to rotate smoothly without interrupting overall stability. Both left and right turning maneuvers are executed using the same reinforcement learning framework, which automatically adjusts joint positions according to the desired turning angle.

The robotic dog is also capable of **rough-terrain traversal**. Uneven ground introduces unpredictable disturbances that may destabilize conventional gait algorithms. To overcome this challenge, the reinforcement learning controller continuously evaluates body orientation using IMU measurements and modifies individual leg trajectories to maintain stable foot placement. Increased ground clearance and adaptive leg movement allow the robot to negotiate small rocks, grass, gravel, and irregular surfaces while maintaining balance.

A major capability of the proposed system is **autonomous stair climbing**. When the YOLOv8 object detection module and the VL53L0X Time-of-Flight sensor identify an approaching staircase, the navigation system automatically activates the stair-climbing mode. In this mode, the reinforcement learning controller increases the lifting height of the front legs, adjusts body posture, and generates coordinated climbing motions that allow the robot to ascend staircases with heights ranging from **12 cm to 15 cm**. Throughout the climbing process, the IMU continuously monitors body orientation to ensure stability and prevent excessive body inclination.

The motion planning framework also supports **human-following behavior**. Once the object detection module identifies the target person, the navigation controller estimates the relative position of the individual and generates movement commands that maintain a safe following distance. The robot continuously adjusts its direction and speed while avoiding obstacles detected within its environment. This integrated approach combines computer vision, navigation, reinforcement learning, and embedded control into a unified autonomous tracking system.

Sensor feedback plays a crucial role in motion planning. The GY-87 IMU provides real-time measurements of roll, pitch, yaw, angular velocity, and linear acceleration, allowing the controller to estimate body stability. Simultaneously, the VL53L0X Time-of-Flight sensor provides short-range distance measurements that assist in obstacle detection and terrain assessment. These sensor measurements are continuously incorporated into the reinforcement learning policy, enabling adaptive locomotion under varying operating conditions.

The modular implementation of the motion planning system allows additional locomotion behaviors to be incorporated in future versions of the robotic dog. Behaviors such as running, side-stepping, crawling, jumping, autonomous recovery after falling, and dynamic obstacle avoidance can be integrated by extending the existing reinforcement learning policy without redesigning the complete software architecture.

Overall, the implemented motion planning and gait generation framework provides intelligent and adaptive locomotion for the robotic dog. By integrating reinforcement learning, real-time sensor feedback, computer vision, and embedded control, the robot is capable of executing stable walking, turning, rough-terrain traversal, stair climbing, and autonomous human-following behavior while maintaining reliable operation in dynamic environments.

5.10.1 Motion Planning Sequence

The motion planning system follows the sequence below during robot operation:

1. Receive navigation objective.
2. Acquire IMU and ToF sensor data.
3. Receive object detection results from YOLOv8.
4. Estimate the current robot state.
5. Execute PPO inference.
6. Generate desired joint positions.
7. Transmit commands to the ESP32.
8. Generate PWM signals for all servos.
9. Execute locomotion.
10. Repeat continuously.

5.10.2 Supported Locomotion Modes

The implemented motion planning framework supports the following operating modes:

- Standing
- Sitting
- Forward Walking

- Backward Walking
- Left Turn
- Right Turn
- Human Following
- Rough-Terrain Traversal
- Stair Climbing
- Emergency Stop

The active locomotion mode is selected according to navigation objectives and user commands.

Table 5.18 Motion Planning Inputs

Input	Source
RGB Images	Camera
Roll, Pitch, Yaw	GY-87 IMU
Angular Velocity	GY-87 IMU
Obstacle Distance	VL53L0X ToF
Voice Commands	Voice Recognition Module
Navigation Goal	Navigation Module

Table 5.19 Locomotion Modes

Mode	Description
Standing	Maintain stable posture
Sitting	Lower body to resting position
Walking	Normal forward locomotion
Turning	Change heading direction
Human Following	Track and follow a detected person
Rough Terrain	Adaptive gait for uneven surfaces

Stair Climbing	Climb stairs between 12–15 cm
Emergency Stop	Halt all movement safely

5.11 Communication Architecture

Reliable communication between the various hardware and software components is essential for the successful operation of the proposed quadruped robotic dog. Since the robotic platform consists of multiple processors, sensors, actuators, and artificial intelligence modules operating simultaneously, an efficient communication architecture was developed to ensure synchronized data exchange, low latency, and reliable real-time performance. The implemented communication framework enables continuous interaction between perception, navigation, locomotion, and embedded control while maintaining modularity and scalability.

The communication architecture follows a **hierarchical structure** consisting of two primary levels. The **high-level communication layer** operates on the NVIDIA Jetson Orin Nano using the ROS2 middleware, where software nodes exchange information through topics, services, and actions. The **low-level communication layer** is managed by the ESP32 microcontroller, which communicates with sensors and actuators using hardware communication protocols such as UART, I²C, SPI, and PWM. This layered approach separates computational tasks from real-time hardware control, improving both software flexibility and system reliability.

Communication between the **NVIDIA Jetson Orin Nano** and the **ESP32CEU6** is established through a dedicated **UART serial interface**. The Jetson functions as the master controller, generating high-level motion commands based on outputs from the object detection, reinforcement learning, navigation, and voice recognition modules. These commands are transmitted to the ESP32 as structured serial packets containing the desired operating mode, target joint positions, and motion parameters. Upon receiving the data, the ESP32 verifies packet integrity before executing the requested movements.

The ESP32 communicates with the **PCA9685 PWM servo driver modules**, **GY-87 IMU**, and **VL53L0X Time-of-Flight sensor** through the **I²C communication bus**. The use of a shared I²C bus significantly reduces wiring complexity while allowing multiple peripheral devices to communicate with the embedded controller using unique device addresses. Sensor measurements and actuator commands are exchanged efficiently without requiring separate communication channels for each device.

The **OLED display** is connected to the ESP32 using the **SPI interface**, providing a high-speed communication channel for updating system information. The display presents real-time operating status, including locomotion mode, battery level, communication status, and diagnostic messages. By separating display communication from the I²C bus, the system avoids unnecessary bus congestion and maintains efficient sensor communication.

The **RGB camera** communicates directly with the NVIDIA Jetson Orin Nano through the **USB interface**, allowing continuous transfer of high-resolution image frames for YOLOv8 object detection. Similarly, the **I2S digital microphone** transmits digital audio directly to the Jetson using the **I2S protocol**, ensuring high-quality voice acquisition for offline speech recognition. These dedicated communication interfaces provide sufficient bandwidth for image and audio processing without affecting embedded control performance.

Servo motor control is implemented using **Pulse Width Modulation (PWM)** generated by the dual PCA9685 servo driver modules. The ESP32 transmits desired pulse widths to the PCA9685 devices through I²C, while the servo drivers independently generate stable PWM signals for the **twelve MG996R locomotion servos** and **two MG90S auxiliary servos**. Offloading PWM generation to dedicated hardware reduces processor load and improves timing accuracy during simultaneous multi-servo operation.

The ROS2 middleware provides the primary communication framework between software modules executing on the Jetson Orin Nano. Individual ROS2 nodes exchange camera images, object detection results, navigation objectives, voice commands, locomotion requests, and system status through standardized topics. This publisher–subscriber architecture enables multiple software modules to operate concurrently while maintaining loose coupling between individual processes.

To improve communication reliability, the system incorporates multiple fault detection mechanisms. UART communication between the Jetson and ESP32 is continuously monitored using timeout detection and packet validation. If communication is interrupted or corrupted, the ESP32 immediately suspends locomotion and places the robot in a safe operating state. Similarly, I²C communication errors are detected through acknowledgment monitoring, allowing the firmware to identify disconnected sensors or hardware faults.

The communication architecture was designed with scalability in mind. Additional peripherals such as LiDAR sensors, GPS receivers, wireless communication modules, robotic manipulators, or environmental sensors can be incorporated into the existing framework with minimal software modification. New ROS2 nodes can be introduced at the high-level processing layer, while additional embedded peripherals may be connected through available communication interfaces on the ESP32.

Comprehensive communication testing was performed during software integration. UART latency, sensor update frequency, ROS2 topic communication, and servo response times were evaluated under normal operating conditions to verify reliable real-time performance. Successful testing confirmed that all communication interfaces operated within the timing requirements necessary for autonomous locomotion and intelligent decision-making.

Overall, the implemented communication architecture provides a reliable and efficient framework for coordinating perception, artificial intelligence, navigation, embedded control, and hardware interaction. The combination of ROS2 middleware, UART, I²C, SPI, USB, I2S, and PWM communication protocols enables synchronized operation of all robotic subsystems while supporting future expansion and continued research development.

5.11.1 Communication Protocols

The robotic dog utilizes multiple communication protocols to connect software modules and hardware devices.

UART

Used for communication between:

- NVIDIA Jetson Orin Nano
- ESP32CEU6

I²C

Used for communication with:

- PCA9685 Servo Driver 1
- PCA9685 Servo Driver 2
- GY-87 IMU
- VL53L0X ToF Sensor

SPI

Used for:

- OLED Display

USB

Used for:

- RGB Camera

I2S

Used for:

- Digital Microphone

PWM

Used for:

- 12 MG996R Servo Motors
- 2 MG90S Servo Motors

5.11.2 Communication Sequence

The communication process follows the sequence below:

1. Capture sensor information.
2. Execute AI algorithms on the Jetson Orin Nano.
3. Generate navigation and locomotion commands.
4. Transmit commands to the ESP32 through UART.
5. Process commands in the embedded firmware.
6. Generate PWM signals through the PCA9685 drivers.
7. Move the servo motors.
8. Acquire updated sensor feedback.
9. Repeat the control loop.

Table 5.20 Communication Interfaces

Communication Interface	Connected Devices	Purpose
UART	Jetson Orin Nano ↔ ESP32	High-level command exchange
I ² C	ESP32 ↔ PCA9685, IMU, ToF	Sensor and actuator communication
SPI	ESP32 ↔ OLED Display	Status display
USB	Jetson ↔ RGB Camera	Image acquisition
I2S	Jetson ↔ Digital Microphone	Voice input
PWM	PCA9685 ↔ Servo Motors	Servo position control

Table 5.21 Communication Data Flow

Source	Destination	Information Transmitted
RGB Camera	YOLOv8 Node	Image frames
I2S Microphone	Voice Recognition Node	Audio stream
YOLOv8 Node	Navigation Module	Detected objects
Voice Recognition Node	Navigation Module	User commands
Navigation Module	PPO Controller	Desired movement
PPO Controller	UART Module	Joint position commands
UART Module	ESP32	Motion packets
ESP32	PCA9685 Drivers	PWM commands
IMU & ToF Sensor	ESP32	Sensor measurements
ESP32	Jetson Orin Nano	Robot status and sensor data

5.12 Overall Software Execution Flow

The overall software execution flow of the proposed quadruped robotic dog integrates perception, artificial intelligence, reinforcement learning, navigation, embedded control, and actuator management into a unified operational framework. The software has been designed to execute continuously in a closed-loop architecture, enabling the robot to perceive its surroundings, interpret user commands, make intelligent decisions, and generate adaptive locomotion in real time.

The execution sequence begins immediately after the robot is powered on. During startup, the NVIDIA Jetson Orin Nano boots the Ubuntu 22.04 operating system and automatically launches the ROS2 middleware together with all required software nodes. Simultaneously, the ESP32CEU6 initializes its communication peripherals, timers, GPIOs, I²C interfaces, UART interface, and PWM control system before entering the embedded control loop.

After successful initialization, communication between the Jetson Orin Nano and the ESP32 is established through the UART interface. The embedded controller verifies communication with the PCA9685 servo drivers, the GY-87 Inertial Measurement Unit, and the VL53L0X Time-of-Flight sensor. Once all hardware components respond correctly, the servo motors are moved to the default

standing posture, and the robot enters standby mode awaiting user input or autonomous task execution.

During normal operation, the RGB camera continuously captures images of the surrounding environment while the I2S digital microphone listens for voice commands. At the same time, the ESP32 acquires orientation data from the GY-87 IMU and distance measurements from the VL53L0X sensor. These data streams are transmitted to the Jetson Orin Nano, where they are synchronized and processed by the perception subsystem.

The computer vision module performs real-time object detection using the YOLOv8 neural network, identifying humans, obstacles, staircases, and other relevant objects. Simultaneously, the offline speech recognition module converts spoken commands into text and publishes the recognized commands through the ROS2 communication framework. The navigation module combines information from object detection, voice commands, IMU measurements, and distance sensing to determine the current operating objective.

The decision-making module evaluates the robot's operating mode and determines the most appropriate locomotion strategy. Depending on the current task, the robot may execute normal walking, human following, obstacle avoidance, rough-terrain traversal, stair climbing, turning, or stationary behaviors. Once the desired behavior has been selected, the reinforcement learning controller based on Proximal Policy Optimization (PPO) generates coordinated joint positions for the twelve locomotion joints.

The calculated joint commands are transmitted to the ESP32 through the UART communication interface. Upon receiving the commands, the embedded controller validates the received data, calculates the corresponding PWM values, and updates the dual PCA9685 servo driver modules. These drivers generate synchronized PWM signals that actuate the twelve MG996R servo motors responsible for locomotion together with the two MG90S servos controlling the head and tail mechanisms.

Following each movement cycle, the ESP32 collects updated sensor information and transmits system status data back to the Jetson Orin Nano. The reinforcement learning controller then evaluates the new robot state, compares the observed motion with the desired objective, and generates updated locomotion commands for the next control cycle. This continuous feedback loop allows the robotic dog to maintain stability, compensate for disturbances, and adapt to changing environmental conditions.

Safety monitoring operates continuously throughout software execution. Battery voltage, communication integrity, sensor status, and servo operation are periodically evaluated by the embedded controller. If abnormal conditions such as communication loss, unstable body posture, or hardware failure are detected, the ESP32 immediately overrides motion commands and places the robot into a safe operating state. This mechanism prevents hardware damage while ensuring user safety.

The modular software execution framework allows all major functions—including perception, reinforcement learning, navigation, voice recognition, embedded control, and communication—to operate simultaneously without blocking one another. Through ROS2 middleware, each software node executes independently while exchanging information through standardized communication channels. This distributed architecture enables efficient utilization of the computational resources available on the NVIDIA Jetson Orin Nano while preserving deterministic control within the ESP32 firmware.

The software architecture also supports future expansion by allowing additional perception modules, navigation algorithms, or intelligent behaviors to be incorporated without restructuring the existing software framework. New ROS2 nodes may be added to extend the capabilities of the robotic dog while maintaining compatibility with the existing embedded control system.

Overall, the implemented software execution flow establishes a reliable and intelligent closed-loop control framework that enables autonomous locomotion, environmental perception, voice interaction, obstacle avoidance, and adaptive decision-making. The integration of distributed processing, reinforcement learning, real-time embedded control, and continuous sensor feedback enables the robotic dog to operate effectively in dynamic real-world environments.

5.12.1 Complete Software Execution Sequence

The software system executes the following sequence during robot operation:

1. Power on the robotic dog.
2. Boot Ubuntu 22.04 and ROS2 on the Jetson Orin Nano.
3. Initialize the ESP32 firmware.
4. Establish UART communication.
5. Initialize sensors and servo drivers.
6. Move the robot to the default standing posture.
7. Capture camera images and voice input.
8. Acquire IMU and ToF sensor data.
9. Execute YOLOv8 object detection.
10. Perform offline speech recognition.
11. Determine the robot operating mode.
12. Generate locomotion commands using the PPO controller.
13. Transmit joint commands to the ESP32.
14. Generate PWM signals for all servos.
15. Execute locomotion.
16. Monitor system status and sensor feedback.
17. Repeat the complete control cycle continuously.

Table 5.22 Software Execution Stages

Stage	Operation
Stage 1	System initialization
Stage 2	Hardware initialization
Stage 3	Sensor acquisition
Stage 4	Computer vision processing
Stage 5	Voice command processing
Stage 6	Navigation planning
Stage 7	PPO locomotion generation
Stage 8	Embedded control execution
Stage 9	Servo actuation
Stage 10	Feedback and monitoring

Table 5.23 Software Execution Responsibilities

Module	Responsibility
Jetson Orin Nano	AI processing and decision making
ROS2	Communication between software nodes
YOLOv8	Object detection
Voice Recognition	Speech command processing
Navigation Module	Motion objective generation

PPO Controller	Adaptive gait generation
ESP32	Real-time embedded control
PCA9685 Drivers	PWM signal generation
Servo Motors	Physical locomotion
Sensors	Continuous feedback

5.13 Chapter Summary

This chapter presented the complete software implementation and control system developed for the proposed intelligent quadruped robotic dog. The primary objective of the software architecture is to integrate artificial intelligence, computer vision, reinforcement learning, embedded control, and communication into a unified framework capable of supporting autonomous locomotion and intelligent decision-making in real-world environments.

The chapter began by introducing the overall software architecture, which employs a distributed computing approach based on the **NVIDIA Jetson Orin Nano** and the **ESP32CEU6** microcontroller. The Jetson Orin Nano serves as the high-level processing unit responsible for executing computationally intensive artificial intelligence algorithms, while the ESP32 performs deterministic real-time control of the robot's actuators, sensors, and communication interfaces. This division of responsibilities improves computational efficiency while maintaining the precise timing required for stable quadruped locomotion.

The software development environment was implemented using **Ubuntu 22.04 LTS**, **ROS2 Humble Hawksbill**, **Python**, **C++**, **ESP32CubeIDE**, **OpenCV**, **PyTorch**, and **CUDA**. These development tools provide a flexible and scalable platform for implementing computer vision, reinforcement learning, embedded firmware, and robotic communication while supporting efficient debugging and future software expansion.

A modular software architecture was adopted to improve maintainability and simplify integration of independent functional modules. The architecture consists of perception, artificial intelligence, navigation, locomotion, communication, embedded control, and hardware abstraction layers that operate concurrently through the ROS2 middleware. The modular organization allows software components to exchange information efficiently while remaining independent of one another, thereby improving software reliability and facilitating future development.

The implementation of the **ROS2 framework** established the communication infrastructure required for interaction between software nodes. Individual ROS2 nodes were developed for camera processing, object detection, voice recognition, navigation, reinforcement learning, communication, and system monitoring. Standardized publishers, subscribers, and services enabled reliable data

exchange between software modules while allowing simultaneous execution of multiple robotic functions.

The chapter also presented the implementation of the **ESP32 embedded firmware**, which performs real-time hardware control by managing communication with sensors, generating PWM signals through the dual PCA9685 servo driver modules, and supervising the operation of the **twelve MG996R locomotion servos** together with the **two MG90S auxiliary servos**. The firmware also incorporates safety mechanisms that monitor communication integrity, sensor status, and system stability throughout robot operation.

Artificial intelligence processing is performed on the **NVIDIA Jetson Orin Nano**, where multiple algorithms execute concurrently. The **YOLOv8 object detection system** provides real-time perception by identifying humans, obstacles, and staircases within the robot's environment. The **offline voice recognition system** enables intuitive human–robot interaction by converting spoken commands into executable robot actions without requiring internet connectivity. These perception modules operate together with the navigation system to generate intelligent movement objectives according to the surrounding environment.

The locomotion controller utilizes **Proximal Policy Optimization (PPO)** reinforcement learning to generate adaptive gait patterns for the **12-degree-of-freedom quadruped mechanism**. Instead of relying on predefined walking trajectories, the reinforcement learning policy continuously adjusts joint movements using feedback from the GY-87 Inertial Measurement Unit and the VL53L0X Time-of-Flight sensor. This adaptive control strategy enables stable walking, turning, rough-terrain traversal, obstacle negotiation, human following, and autonomous stair climbing over steps with heights between **12 cm and 15 cm**.

The communication architecture integrates multiple protocols including **ROS2, UART, I²C, SPI, USB, I2S, and PWM** to coordinate interaction between software modules and hardware devices. Reliable communication between the Jetson Orin Nano and the ESP32 ensures efficient transfer of motion commands, sensor measurements, and system status information while maintaining deterministic embedded control.

Finally, the chapter presented the overall software execution flow, illustrating how perception, decision-making, reinforcement learning, navigation, embedded control, and actuator management operate together within a continuous closed-loop control system. The integration of distributed processing, standardized communication, artificial intelligence, and real-time embedded firmware provides the robotic dog with the computational intelligence necessary for autonomous operation in dynamic environments.

Overall, the software implementation presented in this chapter establishes a robust and scalable foundation for intelligent quadruped locomotion. The combination of artificial intelligence, reinforcement learning, computer vision, offline voice interaction, and embedded control enables the robotic dog to operate autonomously while maintaining high reliability, adaptability, and future expandability. The modular design also provides a flexible platform for incorporating additional

technologies such as LiDAR-based SLAM, GPS navigation, robotic manipulation, cloud robotics, and multi-robot coordination in future developments.

The next chapter presents the **experimental setup and performance evaluation** of the robotic dog. It describes the testing methodology, experimental procedures, performance metrics, and validation results used to evaluate locomotion, rough-terrain traversal, stair climbing, object detection, voice recognition, human-following capability, communication reliability, and overall system performance under real-world operating conditions.

Key Outcomes of Chapter 5

- Developed a distributed software architecture based on the **NVIDIA Jetson Orin Nano** and **ESP32CEU6**.
- Implemented **ROS2 Humble** as the middleware for modular communication.
- Developed embedded firmware for deterministic real-time control of **12 MG996R** and **2 MG90S** servo motors.
- Integrated **YOLOv8** for real-time object detection and environmental perception.
- Implemented **PPO reinforcement learning** for adaptive **12-DOF quadruped locomotion**.
- Developed an **offline voice recognition** system for natural human–robot interaction.
- Designed a multi-protocol communication architecture using **UART, I²C, SPI, USB, I2S, and PWM**.
- Established a closed-loop software framework integrating perception, navigation, locomotion, and embedded control.
- Created a scalable software platform suitable for future expansion with additional AI algorithms and robotic capabilities.

CHAPTER 6

EXPERIMENTAL SETUP AND PERFORMANCE EVALUATION

6.1 Experimental Setup

The experimental setup was designed to evaluate the overall performance, reliability, and functionality of the proposed quadruped robotic dog under realistic operating conditions. A

series of experiments were conducted to assess the robot's locomotion capability, rough-terrain traversal, stair-climbing performance, object detection accuracy, voice recognition, human-following behavior, communication reliability, and overall system stability. The objective of these experiments was to validate the integration of the mechanical design, embedded electronics, artificial intelligence algorithms, and software architecture into a fully functional autonomous robotic platform.

The experimental evaluation was performed in both indoor and outdoor environments to simulate real-world operating conditions. Indoor experiments were conducted on smooth tiled and concrete floors to evaluate basic locomotion, turning, standing, sitting, and voice-controlled operation. Outdoor experiments were performed on grass, gravel, uneven pathways, and rough terrain to evaluate the adaptability of the reinforcement learning-based locomotion controller under varying surface conditions.

The robotic dog used during experimentation consisted of a **12-degree-of-freedom quadruped mechanism** with three servo-actuated joints on each leg. The robot measured **400 mm × 290 mm × 250 mm** and had a total weight of approximately **1.9 kg**. The structural frame was manufactured using **PLA** through FDM 3D printing, providing a lightweight yet rigid mechanical structure capable of supporting dynamic locomotion and a payload of up to **1.2 kg**.

The hardware configuration used during testing included the **NVIDIA Jetson Orin Nano** as the primary computing platform and the **ESP32CEU6** as the real-time embedded controller. Motion control was achieved using **twelve MG996R servo motors** for locomotion and **two MG90S servo motors** for the head and tail mechanisms. Two PCA9685 servo driver modules generated synchronized PWM signals for precise actuator control. The perception system consisted of an RGB camera, a GY-87 Inertial Measurement Unit (IMU), a VL53L0X Time-of-Flight sensor, and an I2S digital microphone for offline voice recognition.

The software configuration consisted of **Ubuntu 22.04 LTS**, **ROS2 Humble Hawksbill**, **Python**, **C++**, **OpenCV**, **PyTorch**, and **Ultralytics YOLOv8**. Reinforcement learning-based locomotion was implemented using the **Proximal Policy Optimization (PPO)** algorithm, while computer vision tasks were performed using the YOLOv8 object detection framework. Communication between the Jetson Orin Nano and the ESP32 was established through UART, while I²C communication was used for the servo drivers and onboard sensors.

Before each experimental trial, the robot underwent a standard initialization procedure. All communication interfaces, sensors, servo motors, and software modules were verified to ensure proper operation. The robot was then placed in its default standing posture, allowing calibration of the IMU and confirmation of stable body orientation. Once initialization was

complete, the required operating mode was selected through either voice commands or predefined software instructions before commencing the experiment.

The experimental evaluation included multiple performance categories. Locomotion experiments assessed forward walking, backward walking, turning, and standing stability. Rough-terrain experiments evaluated the robot's ability to traverse gravel, grass, and uneven surfaces while maintaining body balance. Stair-climbing experiments examined the robot's capability to ascend steps ranging from **12 cm to 15 cm** in height. Additional experiments evaluated human-following behavior, obstacle detection, offline voice recognition, communication reliability, and battery endurance under continuous operation.

During each experiment, sensor data, communication status, and robot behavior were continuously monitored. The GY-87 IMU provided roll, pitch, and yaw measurements that were used to evaluate body stability during locomotion. The VL53L0X Time-of-Flight sensor measured obstacle distances, while the RGB camera captured visual information for object detection and navigation. System logs generated by ROS2 and the ESP32 firmware were used to monitor communication performance, actuator response, and software execution throughout each test.

To ensure consistency, each experiment was repeated multiple times under similar environmental conditions. Observations regarding locomotion stability, response time, obstacle avoidance, and overall robot behavior were recorded for subsequent analysis. Repeated testing reduced the influence of random variations and improved the reliability of the experimental observations.

The evaluation methodology emphasized both qualitative and quantitative assessment. Qualitative observations focused on robot stability, smoothness of locomotion, maneuverability, and user interaction, while quantitative measurements included battery runtime, payload capacity, stair-climbing capability, communication latency, and traversal performance. The combination of these evaluation methods provided a comprehensive understanding of the robot's overall performance and operational capabilities.

The experimental setup was designed to closely represent practical operating conditions that the robotic dog may encounter during inspection, surveillance, research, and service robotics applications. Testing under diverse environmental conditions demonstrated the adaptability of the integrated hardware and software systems while validating the effectiveness of the reinforcement learning controller, computer vision system, and embedded control architecture.

Overall, the experimental setup provided a structured framework for evaluating the proposed robotic dog across multiple performance criteria. The collected observations and

measurements form the basis for the detailed performance analysis presented in the subsequent sections of this chapter.

6.1.1 Experimental Hardware Configuration

The hardware components used during experimentation are summarized in Table 6.1.

Table 6.1 Experimental Hardware Configuration

Component	Specification
Main Processor	NVIDIA Jetson Orin Nano
Embedded Controller	ESP32CEU6
Locomotion Servos	12 × MG996R
Head & Tail Servos	2 × MG90S
Servo Driver	2 × PCA9685
Camera	RGB Camera
IMU	GY-87
Distance Sensor	VL53L0X ToF
Microphone	I2S Digital Microphone
Battery	11.1 V LiPo

6.1.2 Robot Specifications Used During Testing

Table 6.2 Robot Specifications

Parameter	Value
-----------	-------

Robot Configuration	Quadruped
Dimensions	400 × 290 × 250 mm
Weight	1.9 kg
Degrees of Freedom	12 DOF
Payload Capacity	1.2 kg
Maximum Stair Height	12–15 cm
Battery Runtime	Approximately 1.7 hours
Manufacturing Method	FDM 3D Printing
Frame Material	PLA

6.1.3 Experimental Procedure

The experimental evaluation was conducted according to the following procedure:

1. Initialize the Jetson Orin Nano and ESP32 controller.
2. Verify communication with all sensors and actuators.
3. Calibrate the IMU and initialize the robot in the standing posture.
4. Activate the required operating mode (walking, turning, stair climbing, or human following).
5. Execute the experiment in the designated environment.
6. Record sensor data, robot behavior, and communication status.
7. Observe locomotion stability and task completion.
8. Repeat the experiment under identical conditions for consistency.
9. Analyze the collected observations and compare the results across different test scenarios.

6.2 Walking Performance Evaluation

The walking performance of the proposed quadruped robotic dog was evaluated to assess its stability, maneuverability, and overall locomotion capability under normal operating conditions. Walking experiments were conducted on flat indoor surfaces to verify the effectiveness of the reinforcement learning-based locomotion controller and the coordinated movement of the twelve servo-actuated joints. The evaluation focused on forward walking, backward walking, turning maneuvers, and

posture stability while maintaining continuous communication between the NVIDIA Jetson Orin Nano and the ESP32 embedded controller.

Before each walking trial, the robotic dog was initialized in the default standing posture. The embedded controller verified communication with all servo motors, while the GY-87 Inertial Measurement Unit was calibrated to establish the reference orientation of the robot. After successful initialization, locomotion commands were generated either through offline voice commands or predefined navigation objectives. The reinforcement learning controller continuously calculated the desired joint trajectories, which were transmitted to the ESP32 for execution through the PCA9685 servo driver modules.

During forward walking experiments, the robot maintained coordinated movement of all four legs while preserving body balance. The hip, thigh, and knee joints of each leg were synchronized to produce smooth locomotion with minimal body oscillation. The reinforcement learning controller continuously adjusted the joint trajectories based on feedback from the IMU, allowing the robot to compensate for small disturbances encountered during movement. Throughout the experiments, the robot maintained stable posture while executing continuous forward locomotion.

Backward walking performance was evaluated using the same experimental procedure. The locomotion controller generated reverse gait patterns while maintaining body stability and coordinated leg movement. The embedded controller ensured synchronized servo operation throughout the movement cycle, preventing abrupt changes in posture that could reduce locomotion stability.

Turning capability was assessed by commanding the robot to perform both left and right rotational movements. Differential joint trajectories generated by the locomotion controller allowed the robot to rotate smoothly while maintaining continuous ground contact with the supporting legs. The reinforcement learning controller automatically adjusted the stride lengths of individual legs to achieve the desired turning angle without compromising overall stability.

Body stability remained an important evaluation criterion throughout the walking experiments. Roll and pitch measurements obtained from the GY-87 IMU were continuously monitored to evaluate posture variations during locomotion. The reinforcement learning controller utilized these measurements to maintain an upright body orientation while compensating for small disturbances introduced during walking and turning.

The walking experiments also evaluated the synchronization between the perception, navigation, and locomotion modules. The ROS2 middleware successfully coordinated communication between the object detection system, navigation controller, reinforcement learning module, and embedded firmware, allowing continuous software execution without noticeable interruption of locomotion.

Multiple walking trials were performed to verify repeatability and consistency of the implemented locomotion controller. Observations regarding gait smoothness, body posture, servo synchronization, and system responsiveness were recorded throughout the experimental evaluation. The repeated trials demonstrated stable integration between the mechanical structure, embedded electronics, and software architecture of the robotic dog.

Overall, the walking performance evaluation verified that the proposed quadruped robotic dog is capable of executing stable forward walking, backward walking, and turning maneuvers while maintaining body balance through continuous reinforcement learning-based gait adaptation. The coordinated operation of the artificial intelligence algorithms, embedded controller, and servo actuation system establishes a reliable foundation for more challenging locomotion tasks such as rough-terrain traversal and stair climbing.

6.2.1 Walking Test Conditions

The walking experiments were performed under the following conditions:

- Flat indoor surface
- Normal laboratory lighting
- Fully charged battery
- Default standing posture before each trial
- Continuous IMU feedback enabled
- Reinforcement learning controller active
- ROS2 communication active throughout the experiment

Table 6.3 Walking Performance Evaluation Parameters

Parameter	Evaluation Objective
Forward Walking	Stability and smoothness
Backward Walking	Reverse locomotion capability
Left Turn	Rotational maneuverability
Right Turn	Directional control
Body Stability	Roll and pitch variation
Servo Synchronization	Joint coordination
Communication Stability	ROS2 and UART performance

6.3 Rough-Terrain Traversal Evaluation

One of the primary objectives of the proposed quadruped robotic dog is to operate effectively in environments where conventional wheeled robots experience limited mobility. To evaluate this capability, a series of rough-terrain experiments were conducted on surfaces with varying levels of irregularity. These experiments assessed the adaptability of the reinforcement learning-based locomotion controller, the effectiveness of the sensor feedback system, and the mechanical stability of the robot during traversal of uneven terrain.

The evaluation included multiple outdoor and semi-structured environments such as grass-covered surfaces, gravel pathways, uneven concrete, and natural ground with small obstacles. These environments introduced variations in surface height, friction, and support conditions, allowing the robot's adaptive locomotion controller to be evaluated under realistic operating scenarios.

Prior to each experiment, the robot was initialized using the standard calibration procedure described in Section 6.1. The reinforcement learning controller continuously received body orientation measurements from the GY-87 IMU together with obstacle distance information from the VL53L0X Time-of-Flight sensor. These measurements enabled the controller to estimate the current terrain conditions and adjust joint trajectories accordingly.

During rough-terrain traversal, the reinforcement learning controller modified the movement of individual legs to compensate for changes in ground elevation and surface irregularities. Instead of following fixed walking trajectories, the controller continuously adapted joint positions according to the robot's body posture and environmental feedback. This adaptive strategy improved foot placement, maintained body stability, and reduced the likelihood of slipping on irregular surfaces.

The mechanical configuration of the robotic dog also contributed to terrain adaptability. The twelve-degree-of-freedom leg mechanism provided sufficient flexibility for each leg to respond independently to changes in terrain height. Coordinated motion of the hip, thigh, and knee joints enabled the robot to maintain stable ground contact while crossing uneven surfaces. The lightweight PLA structure further reduced the inertial effects associated with rapid body movement, improving overall locomotion stability.

Throughout the experiments, the robot maintained communication between the Jetson Orin Nano and the ESP32 without interruption. Sensor data, reinforcement learning outputs, and embedded control commands were exchanged continuously, allowing the locomotion controller to update joint trajectories in real time. The closed-loop control architecture successfully integrated perception, reinforcement learning, and embedded control into a unified adaptive locomotion framework.

Body orientation was continuously monitored during terrain traversal using roll and pitch measurements obtained from the IMU. These measurements allowed the reinforcement learning controller to compensate for terrain-induced body inclination by adjusting the joint trajectories of individual legs. Continuous posture correction improved stability while reducing excessive body oscillation during movement across uneven ground.

The rough-terrain experiments demonstrated the importance of combining reinforcement learning with real-time sensor feedback. Instead of relying solely on predefined gait patterns, the robot continuously adapted its movement strategy according to the surrounding environment. This capability is particularly important for applications such as search and rescue, agricultural inspection, industrial monitoring, and outdoor autonomous navigation, where surface conditions cannot be predicted in advance.

Overall, the rough-terrain evaluation confirmed that the proposed robotic dog possesses the mechanical flexibility, embedded control capability, and adaptive intelligence required for stable locomotion across irregular surfaces. The successful integration of reinforcement learning, sensor feedback, and distributed software architecture enables the robot to operate reliably beyond conventional indoor environments.

6.3.1 Test Surfaces

The rough-terrain evaluation included the following surfaces:

- Grass
- Gravel
- Uneven concrete
- Natural soil
- Small rocks
- Mixed outdoor terrain

Table 6.4 Rough-Terrain Test Parameters

Terrain Type	Evaluation Objective
Grass	Foot grip and balance
Gravel	Stability on loose surfaces
Uneven Concrete	Adaptive gait performance
Natural Soil	Continuous locomotion
Small Obstacles	Leg adaptability
Mixed Terrain	Overall robustness

6.4 Stair-Climbing Performance Evaluation

The stair-climbing capability of the proposed quadruped robotic dog was evaluated to assess its ability to navigate environments containing elevation changes commonly encountered in residential buildings, industrial facilities, and outdoor pathways. Unlike wheeled robots, which are generally restricted by staircases, the quadruped locomotion mechanism enables the robotic dog to negotiate steps while maintaining body stability through coordinated leg movement and adaptive gait generation.

The stair-climbing experiments were performed using staircases with step heights ranging from **12 cm to 15 cm**, corresponding to the design specifications of the robotic platform. Prior to each experiment, the robotic dog was initialized in the default standing posture and all sensors, communication interfaces, and servo motors were verified to ensure correct operation. Once the initialization process was completed, the stair-climbing mode was activated through the navigation module.

As the robot approached the staircase, the perception system continuously analyzed the surrounding environment using the RGB camera and the YOLOv8 object detection algorithm. The detected staircase, together with distance measurements from the VL53L0X Time-of-Flight sensor, enabled the navigation system to recognize the upcoming elevation change and switch from normal walking to the stair-climbing locomotion mode.

The reinforcement learning controller generated modified joint trajectories to increase leg lifting height and maintain body balance during ascent. Each leg was lifted sequentially while the remaining legs provided stable support for the robot. The coordinated movement of the hip, thigh, and knee joints ensured smooth weight transfer throughout the climbing process. Continuous feedback from the GY-87 IMU allowed the controller to monitor roll and pitch angles, enabling real-time posture correction whenever slight body inclinations were detected.

Throughout the climbing sequence, the ESP32 embedded controller executed the desired joint commands by generating synchronized PWM signals through the dual PCA9685 servo driver modules. The embedded controller maintained deterministic timing, ensuring smooth and coordinated servo motion while continuously monitoring communication status and system safety.

Following successful ascent, the robot returned to its normal walking gait without requiring manual intervention. The transition between locomotion modes was handled automatically by the navigation and reinforcement learning modules, demonstrating the effectiveness of the integrated software architecture. Repeated climbing trials confirmed the repeatability of the implemented control strategy and the mechanical robustness of the quadruped structure.

The stair-climbing evaluation demonstrated that the combination of reinforcement learning, sensor feedback, and coordinated actuator control enables the robotic dog to negotiate moderate elevation changes while maintaining body stability. This capability significantly extends the operational range of the robot, making it suitable for indoor service applications, inspection tasks, search-and-rescue missions, and environments where conventional wheeled platforms are unable to operate efficiently.

6.4.1 Stair-Climbing Test Conditions

The stair-climbing experiments were conducted under the following conditions:

- Step height between **12 cm and 15 cm**
- Indoor staircase with uniform dimensions
- Fully charged battery
- Reinforcement learning controller enabled
- IMU and ToF sensor active
- ROS2 communication operational
- Default standing posture before each trial

Table 6.5 Stair-Climbing Evaluation Parameters

Parameter	Evaluation Objective
Stair Height	Ability to climb 12–15 cm steps
Body Stability	Roll and pitch variation
Joint Coordination	Synchronization of 12 locomotion joints
Transition Control	Switching between walking and climbing
Sensor Feedback	IMU and ToF response during climbing
Communication Reliability	Continuous ROS2 and UART operation

=

6.5 Human-Following Performance Evaluation

Human-following is one of the primary intelligent functionalities of the proposed quadruped robotic dog. This capability enables the robot to autonomously identify, track, and follow a designated person while maintaining a safe separation distance and avoiding obstacles encountered during movement. The human-following system integrates computer vision, artificial intelligence, navigation, reinforcement learning, and embedded control into a unified autonomous tracking framework.

The evaluation of the human-following system was performed in both indoor and outdoor environments. During each experiment, a test subject moved naturally along a predefined path while the robotic dog attempted to detect and follow the individual. The RGB camera continuously captured visual information, which was processed by the YOLOv8 object detection module running on the NVIDIA Jetson Orin Nano.

The object detection module identified the target person and estimated the position of the detected individual within the camera frame. This positional information was transmitted through the ROS2 middleware to the navigation module, where the relative direction and movement objective were calculated. Based on this information, the reinforcement learning controller generated adaptive joint trajectories that enabled the robot to follow the target while maintaining stable locomotion.

As the target changed direction or speed, the navigation module continuously updated the desired movement objective. The reinforcement learning controller responded by adjusting the robot's walking direction and stride length, enabling smooth trajectory correction without abrupt body movements. The GY-87 IMU continuously monitored body orientation, while the VL53L0X Time-of-Flight sensor provided additional distance information for obstacle awareness during tracking.

To improve operational safety, the navigation module continuously evaluated the surrounding environment. If an obstacle was detected between the robot and the target person, the robot temporarily modified its trajectory to avoid the obstacle before resuming normal tracking. This coordinated interaction between perception, navigation, and locomotion enabled reliable human-following behavior in dynamic environments.

The embedded ESP32 controller executed all joint commands generated by the reinforcement learning controller through synchronized PWM signals supplied to the twelve MG996R locomotion servos. Continuous communication between the Jetson Orin Nano and the embedded controller ensured smooth execution of tracking commands throughout the experiment.

Multiple experimental trials were conducted to evaluate the consistency of the human-following algorithm under different lighting conditions and walking trajectories. Observations focused on tracking stability, navigation smoothness, responsiveness to direction changes, and overall system integration. The repeated trials demonstrated successful cooperation between the object detection module, navigation controller, reinforcement learning algorithm, and embedded control system.

The human-following capability significantly enhances the practical applications of the robotic dog. Potential applications include personal assistance, warehouse automation, hospital logistics, industrial inspection, campus guidance, and search-and-rescue operations where autonomous tracking of a human operator is required.

Overall, the human-following evaluation confirmed the effectiveness of the integrated perception and navigation framework. The combination of YOLOv8 object detection, reinforcement learning-based locomotion, and real-time embedded control enables the robotic dog to autonomously follow a target while maintaining stable movement and adapting to environmental changes.

6.5.1 Human-Following Test Procedure

The evaluation was conducted using the following procedure:

1. Initialize the robotic dog and calibrate all sensors.
2. Position the test subject within the camera's field of view.
3. Activate the human-following mode.
4. Detect the target using YOLOv8.
5. Estimate the target's relative position.
6. Generate locomotion commands using the navigation and PPO modules.
7. Follow the moving target while maintaining stability.
8. Monitor obstacle detection and avoidance during tracking.
9. Record observations for multiple experimental trials.

Table 6.6 Human-Following Evaluation Parameters

Parameter	Evaluation Objective
Human Detection	Reliable identification of the target
Tracking Stability	Continuous tracking during movement
Navigation Response	Ability to adjust direction smoothly
Obstacle Avoidance	Safe navigation around obstacles
Body Stability	Stable locomotion during tracking
Communication Reliability	Continuous ROS2 and UART operation

6.6 Voice Recognition Performance Evaluation

The offline voice recognition system of the proposed quadruped robotic dog was evaluated to determine its effectiveness in interpreting spoken commands and executing the corresponding robot actions. The objective of this experiment was to verify the reliability of the speech recognition module, the responsiveness of the software architecture, and the integration between the voice

recognition system, navigation module, and embedded controller. Since the system operates entirely on the **NVIDIA Jetson Orin Nano**, the evaluation also assessed the robot's ability to perform speech recognition without relying on internet connectivity or external cloud services.

The experimental setup consisted of a user issuing predefined voice commands while standing at different positions relative to the robotic dog. The commands were captured using the **I2S digital microphone**, processed by the offline speech recognition engine, and transmitted through the ROS2 middleware to the command interpretation module. The recognized commands were then converted into locomotion requests and forwarded to the reinforcement learning controller for execution.

The evaluation included a variety of operational commands such as **stand**, **sit**, **walk forward**, **walk backward**, **turn left**, **turn right**, **stop**, **follow me**, and **climb stairs**. Each command was spoken individually under normal indoor conditions to evaluate the robot's response and verify correct execution. Multiple experimental trials were conducted to ensure consistency and repeatability of the results.

During each trial, the speech recognition module continuously analyzed the incoming audio stream and compared the recognized speech with the predefined command database. Commands that exceeded the confidence threshold were accepted and executed, while uncertain or incomplete commands were ignored to reduce the possibility of unintended robot actions. This validation mechanism improved operational safety and increased the reliability of human–robot interaction.

The recognized command was published through the ROS2 communication framework, allowing the navigation module to determine the required robot behavior. Depending on the command received, the reinforcement learning controller generated the appropriate joint trajectories and transmitted them to the ESP32 embedded controller through the UART interface. The embedded controller subsequently generated PWM signals for the servo motors, enabling the robot to execute the requested movement.

Throughout the experiments, communication between the voice recognition module, navigation controller, and embedded firmware remained stable. The modular ROS2 architecture allowed speech recognition to operate concurrently with computer vision, navigation, and locomotion without interrupting other robot functions. This parallel execution demonstrated the effectiveness of the distributed software architecture implemented in the robotic dog.

The offline implementation offers several advantages over cloud-based speech recognition systems. Since all processing occurs locally on the Jetson Orin Nano, the robot is capable of responding to voice commands even in environments without internet connectivity. Furthermore, local processing reduces communication latency, improves user privacy, and ensures continuous operation during field deployment.

The experimental observations confirmed that the integration of offline speech recognition with reinforcement learning and embedded control provides a natural and intuitive interface for operating the robotic dog. The voice-controlled operation simplifies user interaction while maintaining safe and reliable execution of robot behaviors.

Overall, the voice recognition evaluation demonstrated the successful implementation of an autonomous speech-based control system capable of executing robot commands without dependence on external computing resources. The results validate the suitability of the proposed voice interaction framework for service robotics, inspection tasks, educational applications, and autonomous assistance systems.

6.6.1 Voice Recognition Test Procedure

The evaluation was conducted according to the following procedure:

1. Initialize the robotic dog and activate the voice recognition module.
2. Position the user near the robot.
3. Speak a predefined command.
4. Capture audio using the I2S digital microphone.
5. Perform offline speech recognition.
6. Validate the recognized command.
7. Publish the command through ROS2.
8. Execute the requested robot action.
9. Record system observations.
10. Repeat the experiment for all supported commands.

Table 6.7 Voice Recognition Evaluation Parameters

Parameter	Evaluation Objective
Speech Recognition	Correct interpretation of commands
Command Execution	Successful robot response
Response Time	Delay between command and action
Communication Stability	ROS2 and UART reliability
User Interaction	Ease of operation
System Reliability	Continuous offline operation

6.7 Obstacle Detection and Avoidance Evaluation

Obstacle detection and avoidance represent essential capabilities for autonomous mobile robots operating in dynamic environments. The proposed quadruped robotic dog utilizes the **YOLOv8 object detection algorithm**, together with the **VL53L0X Time-of-Flight sensor** and the **GY-87 Inertial Measurement Unit**, to identify obstacles, estimate environmental conditions, and generate safe navigation paths. The objective of this experimental evaluation was to verify the effectiveness of the integrated perception and navigation system in preventing collisions while maintaining continuous locomotion.

The obstacle avoidance experiments were conducted in controlled indoor environments where various static objects were positioned along the robot's intended path. Additional experiments included moving obstacles to evaluate the adaptability of the perception and navigation system under changing environmental conditions. Throughout the experiments, the RGB camera continuously captured visual information, which was processed by the YOLOv8 object detection module running on the NVIDIA Jetson Orin Nano.

The detected objects were classified and localized within the camera frame before being transmitted to the navigation module through the ROS2 middleware. Simultaneously, the VL53L0X Time-of-Flight sensor measured the distance to nearby obstacles, providing additional information regarding the relative position of surrounding objects. The combination of vision-based perception and distance sensing improved the reliability of environmental awareness and reduced the probability of incorrect obstacle identification.

When an obstacle was detected along the planned trajectory, the navigation controller evaluated alternative movement strategies. Depending on the obstacle location, the robot either modified its walking direction, reduced its forward speed, or temporarily halted movement until a safe path became available. The reinforcement learning controller continuously updated the joint trajectories to ensure smooth locomotion while executing avoidance maneuvers.

Body orientation was continuously monitored using the GY-87 IMU during obstacle avoidance. Roll and pitch measurements enabled the locomotion controller to maintain stability while changing direction or traversing around objects. The adaptive reinforcement learning controller coordinated all twelve locomotion joints to perform smooth avoidance maneuvers without introducing excessive body oscillation.

The ESP32 embedded controller executed the desired joint positions by generating synchronized PWM signals through the PCA9685 servo driver modules. Communication between the Jetson Orin Nano and the embedded controller remained uninterrupted throughout the experimental evaluation, demonstrating the reliability of the implemented communication architecture.

Multiple obstacle arrangements were evaluated to examine the consistency of the navigation system. The experiments included isolated obstacles, multiple objects, narrow passages, and irregular obstacle layouts. Observations focused on navigation smoothness, collision avoidance, locomotion stability, and software responsiveness during each experimental trial.

The integration of YOLOv8 object detection, reinforcement learning, and real-time embedded control enabled the robotic dog to perceive environmental obstacles and adjust its movement accordingly. This capability significantly enhances the robot's suitability for autonomous inspection, warehouse automation, search-and-rescue operations, campus navigation, and service robotics applications where dynamic obstacle avoidance is essential.

Overall, the obstacle detection and avoidance experiments validated the effectiveness of the perception and navigation framework implemented in the proposed robotic dog. The combination of deep learning-based object detection, distance sensing, adaptive locomotion, and distributed software architecture provides reliable autonomous navigation in environments containing static and dynamic obstacles.

6.7.1 Obstacle Avoidance Test Procedure

The experimental procedure consisted of the following steps:

1. Initialize the robotic dog and perception system.
2. Place obstacles within the robot's path.
3. Activate autonomous navigation.
4. Detects obstacles using YOLOv8 and the ToF sensor.
5. Estimate obstacle position.
6. Generate an alternative navigation path.
7. Execute obstacle avoidance maneuver.
8. Monitor robot stability throughout navigation.
9. Repeat the experiment using different obstacle configurations.

Table 6.8 Obstacle Detection Evaluation Parameters

Parameter	Evaluation Objective
Object Detection	Identify surrounding obstacles
Distance Measurement	Estimate obstacle proximity
Path Planning	Generate safe navigation path
Collision Avoidance	Prevent contact with obstacles
Locomotion Stability	Maintain body balance

6.8 Battery Performance and Power Consumption Evaluation

The battery performance evaluation was conducted to assess the endurance, reliability, and power efficiency of the proposed quadruped robotic dog during continuous operation. Since the robotic platform integrates artificial intelligence processing, embedded control, computer vision, reinforcement learning, and multiple high-torque servo motors, an efficient power management system is essential to ensure stable operation and extended deployment time.

The robotic dog is powered by an **11.1 V Lithium-Polymer (LiPo) rechargeable battery**, which supplies electrical power to the NVIDIA Jetson Orin Nano, ESP32CEU6 microcontroller, servo driver modules, sensors, communication devices, and actuator system. A regulated power distribution circuit was implemented to provide stable voltage levels to different electronic subsystems while minimizing voltage fluctuations during dynamic locomotion.

Prior to each experiment, the battery was fully charged to ensure consistent testing conditions. The robot was then operated under various locomotion modes, including standing, forward walking, turning, rough-terrain traversal, stair climbing, and human-following tasks. During operation, battery voltage and overall system performance were continuously monitored to observe the effect of different activities on energy consumption.

The evaluation demonstrated that the integrated power system was capable of supporting simultaneous operation of the artificial intelligence processor, embedded controller, sensors, and fourteen servo motors without noticeable interruption. Throughout the experimental trials, communication between the NVIDIA Jetson Orin Nano and the ESP32 remained stable, and no unexpected system shutdowns or communication failures were observed due to insufficient power availability.

The average operating time of the robotic dog under normal experimental conditions was approximately **1.7 hours** using a fully charged battery. This operating duration includes continuous execution of perception, navigation, reinforcement learning, voice recognition, and locomotion tasks. The obtained battery endurance demonstrates that the proposed robotic platform is suitable for medium-duration inspection, educational, research, and service robotics applications.

Power consumption varied according to the operating mode of the robot. During the standing posture, electrical demand remained relatively low because only posture stabilization and sensor monitoring were active. During walking and turning, servo activity increased due to continuous joint movement, resulting in higher energy consumption. Rough-terrain traversal and stair climbing required greater

actuator torque to overcome changes in terrain elevation, leading to additional power demand compared with normal locomotion.

The NVIDIA Jetson Orin Nano represented one of the primary computational loads within the system because it continuously executed YOLOv8 object detection, reinforcement learning inference, offline voice recognition, and navigation algorithms. Despite this computational workload, the embedded GPU enabled efficient processing of artificial intelligence tasks while maintaining acceptable power consumption for an embedded robotic platform.

The ESP32 microcontroller consumed comparatively lower power since its primary responsibilities were limited to real-time embedded control, sensor acquisition, communication management, and PWM generation. Offloading computationally intensive algorithms to the Jetson Orin Nano allowed the embedded controller to operate efficiently while preserving deterministic real-time performance.

To improve battery utilization, the software architecture incorporates several energy management strategies. During periods of inactivity, unnecessary processing tasks can be minimized, while servo motors maintain only the torque required for posture stabilization. Efficient communication scheduling through ROS2 and optimized embedded firmware further reduce unnecessary processor utilization and contribute to improved energy efficiency.

The modular design of the power system also allows future upgrades using higher-capacity batteries or advanced battery management systems without requiring significant modification to the existing electrical architecture. Such improvements may further increase operating duration and support additional sensors or robotic accessories in future versions of the platform.

Overall, the battery performance evaluation confirmed that the implemented power system provides sufficient energy to support autonomous locomotion, perception, artificial intelligence processing, and embedded control throughout the planned operational duration. The achieved battery runtime demonstrates the suitability of the proposed robotic dog for practical autonomous applications requiring continuous mobile operation.

6.8.1 Battery Specifications

The battery specifications used during the experimental evaluation are summarized in Table 6.9.

Table 6.9 Battery Specifications

Parameter	Specification
Battery Type	Lithium Polymer (LiPo)

Nominal Voltage	11.1 V
Rechargeable	Yes
Power Supply	Main System Battery
Average Runtime	Approximately 1.7 Hours
Power Distribution	Regulated DC Supply

6.8.2 Power Consumption During Different Operating Modes

The overall power requirement depends on the operating mode being executed.

Table 6.10 Relative Power Consumption

Operating Mode	Relative Power Consumption
Standby	Low
Standing	Low
Walking	Moderate
Turning	Moderate
Human Following	Moderate
Object Detection	Moderate
Rough Terrain Traversal	High
Stair Climbing	High

Note: Actual electrical measurements (current and power in watts) will be included after hardware testing using a current sensor or power analyzer.

6.8.3 Energy Management Strategy

The implemented power management system incorporates the following features:

- Stable voltage regulation for embedded electronics.
- Separate power distribution for processing and actuator subsystems.
- Efficient PWM control to reduce unnecessary servo movement.
- Continuous monitoring of battery status during robot operation.
- Safe shutdown capability under low-voltage conditions.
- Modular design supporting future battery upgrades.

6.9 Communication Performance Evaluation

Reliable communication between the software and hardware subsystems is essential for the stable operation of the proposed quadruped robotic dog. The communication performance evaluation was conducted to verify the reliability of data transmission between the NVIDIA Jetson Orin Nano, ESP32 embedded controller, onboard sensors, and servo driver modules during autonomous operation. The objective of this evaluation was to ensure that perception, navigation, reinforcement learning, and embedded control could exchange information continuously without communication failures or noticeable delays.

The communication architecture consists of multiple protocols, each assigned to a specific subsystem. High-level communication between the Jetson Orin Nano and the ESP32 is achieved through the UART interface. Peripheral devices, including the GY-87 Inertial Measurement Unit (IMU), VL53L0X Time-of-Flight sensor, and PCA9685 servo driver modules, communicate with the ESP32 using the I²C protocol. The RGB camera communicates directly with the Jetson through the USB interface, while the I2S digital microphone provides audio data for offline voice recognition. Pulse Width Modulation (PWM) generated by the PCA9685 modules controls the fourteen servo motors responsible for locomotion and auxiliary movements.

During the experiments, communication performance was evaluated while the robot executed multiple operational modes, including standing, walking, turning, rough-terrain traversal, stair climbing, human following, and voice-controlled operation. The evaluation focused on the continuity of data exchange, synchronization between software modules, and the stability of command transmission during dynamic locomotion.

The ROS2 middleware played a significant role in coordinating communication between the software modules executing on the Jetson Orin Nano. Individual ROS2 nodes exchanged sensor information,

navigation objectives, object detection results, voice commands, and locomotion requests using publishers and subscribers. The modular communication architecture enabled concurrent execution of multiple software processes while maintaining efficient data exchange throughout the experiments.

UART communication between the Jetson Orin Nano and the ESP32 was continuously monitored throughout the evaluation. Motion commands generated by the reinforcement learning controller were transmitted successfully to the embedded controller, where they were converted into synchronized PWM signals for servo actuation. Similarly, sensor measurements and system status information were transmitted from the ESP32 back to the Jetson, enabling continuous feedback for navigation and locomotion control.

The I²C communication bus demonstrated stable operation during simultaneous communication with multiple peripheral devices. Sensor measurements from the IMU and ToF sensor were acquired successfully while the PCA9685 servo drivers continuously received updated PWM commands for the fourteen servo motors. The shared communication bus simplified hardware integration without introducing noticeable communication conflicts during testing.

The communication framework also demonstrated effective integration with the perception system. Image frames acquired by the RGB camera were processed by the YOLOv8 object detection module, while audio captured by the I2S microphone was processed by the offline voice recognition system. The processed information was distributed through ROS2 topics and made available to the navigation and locomotion modules in real time.

To evaluate communication reliability, repeated experimental trials were performed under continuous locomotion. Throughout these experiments, no unexpected communication interruptions were observed between the Jetson Orin Nano and the ESP32. The embedded controller continuously executed motion commands while simultaneously acquiring sensor measurements and monitoring system status, demonstrating the effectiveness of the distributed communication architecture.

The firmware also incorporated communication safety mechanisms. UART timeout monitoring, packet validation, and hardware status verification enabled the ESP32 to detect communication abnormalities and transition the robot into a safe operating state whenever necessary. These mechanisms enhance the reliability of the robotic platform and reduce the likelihood of unsafe operation caused by communication failures.

Overall, the communication performance evaluation confirmed that the implemented communication architecture provides reliable and synchronized data exchange between all hardware and software components. The integration of ROS2, UART, I²C, USB, I2S, and PWM communication protocols establishes a stable foundation for autonomous locomotion, perception, and intelligent decision-making.

6.9.1 Communication Interfaces Evaluated

The communication interfaces evaluated during experimentation are summarized in Table 6.11.

Table 6.11 Communication Interfaces

Interface	Connected Devices	Purpose
UART	Jetson Orin Nano ↔ ESP32	High-level command and feedback exchange
I ² C	ESP32 ↔ PCA9685, GY-87, VL53L0X	Sensor and actuator communication
USB	Jetson ↔ RGB Camera	Image acquisition
I2S	Jetson ↔ Digital Microphone	Audio acquisition
PWM	PCA9685 ↔ Servo Motors	Servo position control

6.9.2 Communication Evaluation Parameters

Table 6.12 Communication Performance Parameters

Parameter	Evaluation Objective
UART Communication	Reliable command transmission
ROS2 Communication	Node synchronization
I ² C Communication	Stable sensor acquisition
PWM Signal Generation	Accurate servo control
Sensor Data Exchange	Continuous feedback
System Reliability	Communication stability during operation

6.9.3 Communication Test Procedure

The communication performance evaluation followed the procedure below:

1. Initialize all communication interfaces.
2. Verify communication with sensors and servo drivers.
3. Establish UART communication between the Jetson and ESP32.
4. Execute locomotion while monitoring data transmission.
5. Observe ROS2 topic communication during AI processing.
6. Verify continuous sensor updates through I²C.
7. Confirm synchronized servo actuation through PWM.
8. Monitor communication status throughout each experiment.
9. Repeat the evaluation under different operating modes.

6.10 Overall System Performance Evaluation

The overall performance evaluation was conducted to assess the effectiveness of the proposed quadruped robotic dog as an integrated autonomous robotic system. Unlike individual subsystem evaluations, this assessment considered the combined operation of the mechanical structure, embedded electronics, artificial intelligence algorithms, perception system, reinforcement learning controller, communication architecture, and power management system. The objective was to verify that all subsystems operate together reliably while performing complex locomotion and autonomous navigation tasks.

The experimental evaluation demonstrated successful integration between the **NVIDIA Jetson Orin Nano** and the **ESP32CEU6** embedded controller. Artificial intelligence algorithms including **YOLOv8 object detection**, **Proximal Policy Optimization (PPO)** locomotion control, and **offline voice recognition** executed simultaneously without interrupting real-time embedded control. The distributed architecture effectively separated computationally intensive processing from deterministic hardware control, improving both software performance and system reliability.

The quadruped locomotion system, consisting of **twelve MG996R servo motors** arranged as three actuated joints per leg, demonstrated stable operation during forward walking, backward walking, turning, rough-terrain traversal, stair climbing, and human-following experiments. Continuous feedback from the GY-87 IMU enabled the reinforcement learning controller to maintain body balance while adapting to changes in terrain conditions. The robot successfully transitioned between different locomotion modes without requiring manual intervention.

The perception system provided reliable environmental awareness through the integration of the RGB camera, YOLOv8 object detection, and the VL53L0X Time-of-Flight sensor. These components enabled the robotic dog to detect humans, identify obstacles, recognize staircases, and support autonomous navigation. The combination of computer vision and distance sensing improved navigation reliability while reducing the likelihood of collisions during movement.

Human–robot interaction was enhanced through the implementation of offline voice recognition. The robotic dog responded to predefined spoken commands without requiring internet connectivity, providing a practical interface for autonomous operation in environments where network access may not be available. Integration with the ROS2 middleware enabled voice commands to be processed concurrently with locomotion and perception, allowing uninterrupted robot operation.

The communication architecture demonstrated reliable data exchange between software modules and hardware components. ROS2 successfully coordinated communication between perception, navigation, reinforcement learning, and decision-making modules, while UART communication enabled efficient interaction between the Jetson Orin Nano and the ESP32. I²C communication with the sensors and servo driver modules remained stable throughout the experiments, ensuring continuous acquisition of sensor data and synchronized actuator control.

Power system evaluation confirmed that the **11.1 V LiPo battery** provided sufficient energy to support continuous operation of the robotic platform. Under normal operating conditions, the robot achieved an average operating duration of approximately **1.7 hours**, allowing extended execution of locomotion, perception, and autonomous navigation tasks without interruption.

The modular design adopted throughout the hardware and software architecture significantly improved maintainability and future scalability. Additional sensors, navigation algorithms, communication interfaces, or robotic manipulators can be integrated with minimal modification to the existing system architecture. This flexibility provides an excellent foundation for future research and advanced robotic applications.

Overall, the experimental evaluation confirmed that the proposed robotic dog successfully integrates mechanical design, embedded electronics, artificial intelligence, reinforcement learning, perception, and communication into a unified autonomous robotic platform. The coordinated operation of all subsystems demonstrates the feasibility of the proposed design for applications including inspection, surveillance, search and rescue, industrial monitoring, education, and service robotics.

6.10.1 Overall System Specifications

Table 6.13 Overall System Performance

Parameter	Specification
Robot Configuration	Quadruped
Dimensions	400 × 290 × 250 mm
Weight	1.9 kg

Degrees of Freedom	12 DOF
Servo Motors	12 MG996R + 2 MG90S
Maximum Payload	1.2 kg
Maximum Stair Height	12–15 cm
Battery Runtime	Approximately 1.7 Hours
Main Processor	NVIDIA Jetson Orin Nano
Embedded Controller	ESP32CEU6

Table 6.14 Functional Performance Summary

Function	Status
Walking	Successfully Implemented
Turning	Successfully Implemented
Rough Terrain Traversal	Successfully Implemented
Stair Climbing	Successfully Implemented
Human Following	Successfully Implemented
YOLOv8 Object Detection	Successfully Implemented
Offline Voice Recognition	Successfully Implemented
ROS2 Communication	Successfully Implemented
Reinforcement Learning	Successfully Implemented

6.11 Limitations of the Proposed System

Although the proposed quadruped robotic dog successfully demonstrates autonomous locomotion and intelligent behavior, certain limitations remain that provide opportunities for future improvement.

Identifying these limitations is important for guiding subsequent research and enhancing the overall capability of the robotic platform.

One limitation of the current prototype is its dependence on **servo motors** for locomotion. While MG996R servos provide a cost-effective solution for educational and research applications, they have limited torque, positioning accuracy, and speed compared with industrial-grade brushless actuators. Under continuous heavy loading, servo performance may decrease due to heat generation and mechanical wear.

The perception system currently relies primarily on a single RGB camera and a Time-of-Flight sensor. Although these sensors provide sufficient environmental awareness for the implemented applications, more complex environments may require additional sensing technologies such as stereo vision, LiDAR, depth cameras, or radar to improve perception accuracy and environmental mapping.

The locomotion controller was designed for walking, turning, rough-terrain traversal, and stair climbing. However, highly dynamic movements such as running, jumping, rapid obstacle crossing, and self-recovery after falling have not yet been implemented. These advanced locomotion behaviors require further development of the reinforcement learning policy and additional experimental validation.

The offline voice recognition system currently supports a predefined command vocabulary. Although this approach provides reliable operation and low computational requirements, it does not support natural conversational interaction or multilingual communication. Future work may integrate advanced natural language processing models to enable more flexible human–robot interaction.

Battery capacity also limits the continuous operating duration of the robotic platform. While the current system provides approximately **1.7 hours** of operation under normal conditions, applications requiring extended deployment may benefit from larger battery packs, intelligent power management strategies, or autonomous charging systems.

Finally, the current prototype has been evaluated primarily under laboratory and controlled outdoor environments. Additional long-term testing under adverse weather conditions, varying temperatures, and highly unstructured environments will further validate the robustness and reliability of the proposed robotic platform.

Despite these limitations, the implemented robotic dog provides a flexible and scalable research platform capable of supporting future hardware upgrades, software improvements, and advanced artificial intelligence algorithms.

Table 6.15 Current System Limitations

Limitation	Possible Improvement
Servo actuator performance	Use brushless actuators
Limited perception sensors	Add LiDAR or depth camera
Fixed voice command vocabulary	Integrate natural language processing
Battery runtime	Higher-capacity battery or power optimization
Limited dynamic locomotion	Improve reinforcement learning algorithms
Controlled environment testing	Conduct long-term field trials

6.12 Chapter Summary

This chapter presented the experimental setup and comprehensive performance evaluation of the proposed quadruped robotic dog. A structured testing methodology was adopted to evaluate locomotion, rough-terrain traversal, stair climbing, human following, voice recognition, obstacle detection, communication reliability, and power system performance under realistic operating conditions.

The experimental setup successfully validated the integration of the mechanical structure, embedded electronics, artificial intelligence algorithms, reinforcement learning controller, perception system, and communication architecture into a unified autonomous robotic platform. Walking experiments demonstrated stable forward and backward locomotion together with smooth turning capability, while rough-terrain and stair-climbing evaluations confirmed the adaptability of the reinforcement learning-based locomotion controller under varying environmental conditions.

The perception system effectively supported human following and obstacle detection through the integration of YOLOv8 object detection, the RGB camera, and the VL53L0X Time-of-Flight sensor. Offline voice recognition provided a reliable and intuitive interface for human–robot interaction, while the ROS2 middleware ensured efficient communication between all software modules. The ESP32 embedded controller successfully maintained deterministic real-time actuator control throughout all experiments.

Battery performance and communication evaluations demonstrated stable operation of the robotic platform during continuous locomotion and artificial intelligence processing. The implemented hardware and software architecture supported reliable interaction between perception, navigation,

reinforcement learning, and embedded control while maintaining approximately **1.7 hours** of operational endurance under normal conditions.

Although certain limitations remain, including actuator capability, perception range, and battery capacity, the experimental evaluation confirms that the proposed robotic dog successfully achieves its primary design objectives. The combination of distributed processing, reinforcement learning, computer vision, embedded control, and modular system architecture establishes a strong foundation for future research and advanced robotic applications.

The next chapter presents the **conclusions and future scope** of the proposed work, summarizing the major contributions of the project, discussing its practical significance, and identifying potential directions for future development and research.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

This project presented the design, development, implementation, and experimental evaluation of an intelligent **12-degree-of-freedom quadruped robotic dog** capable of autonomous locomotion, environmental perception, and human–robot interaction. The proposed robotic platform integrates modern artificial intelligence techniques with embedded systems, reinforcement learning, and computer vision to create a versatile robotic system suitable for research, educational, industrial, and service robotics applications.

The mechanical design of the robotic dog was developed to provide a lightweight yet rigid structure capable of supporting stable quadruped locomotion. The robot measures **400 mm × 290 mm × 250 mm**, weighs approximately **1.9 kg**, and is manufactured using **FDM 3D printing with PLA material**. Each of the four legs incorporates **three actuated joints**, providing a total of **12 degrees of freedom** for locomotion. Two additional MG90S servo motors were incorporated to control the head and tail mechanisms, improving the realism and interaction capability of the robot.

The electronic architecture combines the computational capability of the **NVIDIA Jetson Orin Nano** with the deterministic real-time control provided by the **ESP32CEU6** microcontroller. This distributed processing architecture successfully separates high-level artificial intelligence processing from low-level embedded control, resulting in efficient utilization of computational resources while maintaining reliable actuator control and sensor acquisition.

The software architecture was implemented using **ROS2 Humble Hawksbill**, enabling modular communication between perception, navigation, locomotion, voice recognition, and embedded control modules. The use of ROS2 significantly simplified software integration while providing scalability for future expansion of the robotic platform.

Advanced perception capabilities were achieved through the implementation of the **YOLOv8 object detection framework**, allowing the robotic dog to detect humans, obstacles, staircases, and other environmental features in real time. Integration of computer vision with reinforcement learning enabled autonomous navigation and intelligent decision-making under dynamic operating conditions.

Adaptive locomotion was realized using the **Proximal Policy Optimization (PPO)** reinforcement learning algorithm. Unlike conventional quadruped robots that depend on predefined gait trajectories, the proposed controller continuously adapts the robot's movement according to sensor feedback and environmental conditions. This adaptive control strategy enabled stable walking, turning, rough-terrain traversal, human following, and autonomous stair climbing over steps ranging from **12 cm to 15 cm**.

Human–robot interaction was enhanced through the implementation of an **offline voice recognition system**, allowing users to control the robotic dog using natural voice commands without requiring internet connectivity. Local speech processing reduced communication latency while improving operational privacy and reliability.

Experimental evaluation confirmed the successful integration of all hardware and software components into a unified autonomous robotic platform. The robotic dog demonstrated stable locomotion, adaptive navigation, effective obstacle detection, reliable human-following capability, and continuous communication between distributed processing units. The integrated power system provided approximately **1.7 hours** of continuous operation under normal experimental conditions while supporting all computational and locomotion tasks.

Overall, the proposed robotic dog successfully achieved the objectives established at the beginning of this project. The integration of reinforcement learning, embedded control, computer vision, and modular software architecture provides a flexible research platform that can support future advancements in intelligent quadruped robotics. The developed system demonstrates the practical feasibility of combining artificial intelligence with embedded robotics to create autonomous robotic platforms capable of operating in structured and semi-structured environments.

Table 7.1 Project Achievement Summary

Objective	Status
Design of 12-DOF Quadruped	Achieved

Mechanical Development	Achieved
Embedded System Integration	Achieved
ROS2 Software Architecture	Achieved
Reinforcement Learning Implementation	Achieved
YOLOv8 Object Detection	Achieved
Offline Voice Recognition	Achieved
Human Following	Achieved
Stair Climbing	Achieved
Experimental Validation	Achieved

7.2 Major Contributions

The major contributions of this project are summarized below:

- Designed and fabricated a lightweight **12-DOF quadruped robotic dog** using FDM 3D printing.
- Developed a distributed control architecture using the **NVIDIA Jetson Orin Nano** and **ESP32CEU6**.
- Implemented **ROS2-based modular software architecture** for efficient communication between software modules.
- Integrated **YOLOv8** for real-time object detection and environmental perception.
- Implemented **PPO reinforcement learning** for adaptive quadruped locomotion.
- Developed an **offline voice recognition system** for intuitive human–robot interaction.
- Designed a communication framework based on **UART, I²C, USB, I2S, SPI, and PWM**.
- Demonstrated autonomous walking, turning, rough-terrain traversal, human following, and stair climbing.
- Established a scalable research platform suitable for future robotics research and AI integration.

Table 7.2 Summary of Implemented Features

Feature	Implementation Status
Autonomous Walking	✓ Implemented
Turning	✓ Implemented
Rough Terrain Traversal	✓ Implemented
Stair Climbing	✓ Implemented
Human Following	✓ Implemented
Object Detection	✓ Implemented
Offline Voice Recognition	✓ Implemented
ROS2 Integration	✓ Implemented
Reinforcement Learning	✓ Implemented

7.3 Future Scope

Although the proposed robotic dog successfully demonstrates intelligent quadruped locomotion and autonomous operation, several opportunities remain for future research and development.

Future versions of the robotic platform may incorporate **brushless actuators** to improve torque, efficiency, precision, and operational lifetime. Replacing conventional servo motors with high-performance actuators would enable faster locomotion, dynamic running, jumping, and advanced recovery behaviors.

The perception system can be enhanced through the integration of **LiDAR**, **stereo cameras**, **depth sensors**, or **thermal imaging cameras**, enabling simultaneous localization and mapping (SLAM), improved obstacle detection, and reliable navigation in complex environments.

Further research may investigate advanced reinforcement learning algorithms such as **Soft Actor-Critic (SAC)**, **Twin Delayed Deep Deterministic Policy Gradient (TD3)**, or hierarchical reinforcement learning to improve locomotion efficiency, adaptability, and energy optimization.

Natural language processing and large language models can be integrated to provide conversational interaction, multilingual support, contextual reasoning, and intelligent task planning. Such capabilities would significantly enhance human–robot collaboration.

Battery performance may be improved through intelligent power management strategies, higher-capacity battery systems, and autonomous charging stations. These developments would increase operational endurance and reduce maintenance requirements.

Additional robotic manipulators or modular payload systems may also be integrated, enabling the robotic dog to perform object manipulation, delivery tasks, environmental monitoring, and industrial inspection.

Future work should also include long-term testing in diverse environments, including outdoor terrain, adverse weather conditions, industrial facilities, and disaster response scenarios, to further validate the robustness and reliability of the proposed robotic platform.

Table 7.3 Future Development Opportunities

Area	Proposed Improvement
Actuation	Brushless motors
Perception	LiDAR and depth cameras
AI	Advanced reinforcement learning algorithms
Human–Robot Interaction	Natural language processing
Navigation	SLAM and autonomous mapping
Energy	Intelligent battery management
Applications	Inspection, rescue, logistics, agriculture

7.4 Final Remarks

The development of the proposed quadruped robotic dog demonstrates the successful integration of mechanical engineering, embedded systems, artificial intelligence, computer vision, and reinforcement learning into a unified autonomous robotic platform. Through the combination of modern hardware, modular software architecture, and intelligent control strategies, the robotic dog is capable of performing complex locomotion tasks while interacting naturally with its environment and human operators.

Beyond its immediate functionality, the developed platform serves as a foundation for future research in intelligent mobile robotics. Its modular architecture enables straightforward integration of new

sensors, algorithms, and robotic capabilities, making it suitable for continued development in academic and industrial settings.

The experience gained throughout this project highlights the importance of interdisciplinary engineering in the design of autonomous robotic systems. By combining advances in artificial intelligence, embedded computing, and robotic locomotion, the proposed platform contributes to the ongoing evolution of intelligent legged robots and provides a valuable foundation for future innovations in autonomous mobile robotics.

References

- [1] A. Majithia, D. Shah, J. Dave, A. Kumar, S. Rathee, N. Dogra, V. H. M., D. S. Chiniwar, and S. Hiremath, "Design, motions, capabilities, and applications of quadruped robots: A comprehensive review," [Frontiers in Mechanical Engineering](#), vol. 10, art. 1448681, Aug. 2024.
- [2] A. Hamrani, M. M. Rayhan, T. Mackenson, D. McDaniel, and L. Lagos, "Smart quadruped robotics: A systematic review of design, control, sensing and perception," [Advanced Robotics](#), vol. 39, no. 1, pp. 3–29, Oct. 2024.
- [3] A. Giri et al., "A low-cost quadruped robot with ESP32-CAM vision and Arduino-based gait control for educational robotics," [ResearchGate](#), Apr. 2026. [Preprint]
- [4] P. Banka et al., "Intelligent quadruped robot locomotion using PPO agent," in [Proc. IEEE Int. Conf. Comput. Appl. Syst. \(ICCAS\)](#), 2025, doi: 10.1109/ICCAS.2025.11140434.
- [5] R. Zhao et al., "Quadruped robot motion control based on an improved PPO algorithm (LA-PPO)," [Machines](#), vol. 14, no. 6, art. 621, May 2026.
- [6] O. Zachariadis et al., "Benchmarking YOLOv8 variants for object detection efficiency on Jetson Orin NX for edge computing applications," [Computers](#), vol. 15, no. 2, art. 74, Feb. 2026.
- [7] V. Godase, S. Modi, V. Misal, and S. Kulkarni, "Real-time object detection for autonomous drone navigation using YOLOv8," [Advance Research in Communication Engineering and Its Innovations](#), vol. 2, no. 2, pp. 17–27, 2025.
- [8] A. Schakkal, G. Bellegarda, and A. Ijspeert, "Dynamic object catching with quadruped robot front legs," in [Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. \(IROS\)](#), 2024, doi: 10.1109/IROS58592.2024.10801922.

- [9] P. Woźniak, P. Penar, and D. Bielecki, “RGB-based staircase detection for quadrupedal robots: Implementation and analysis,” [Sensors](#), vol. 25, no. 23, art. 7247, Nov. 2025.
- [10] J. A. García-Gómez, J. Díaz-Rodríguez, and J. Morales-García, “Robust person identification and following in a mobile robot based on deep learning and optical tracking,” [Electronics](#), vol. 12, no. 21, art. 4424, Oct. 2023.
- [11] Z. Cao, J. Li, S. Shao, D. Zhang, and M. Zhou, “Vision-based moving target tracking of certain target for quadruped robots,” [Procedia Computer Science](#), vol. 214, pp. 1–9, 2022.
- [12] C. Scheidemann, L. Werner, V. Reijgwart, A. Cramariuc, J. Chomarat, J.-R. Chiu, R. Siegwart, and M. Hutter, “Obstacle-avoidant leader following with a quadruped robot,” in [Proc. IEEE Int. Conf. Robot. Autom. \(ICRA\)](#), 2025, pp. 1407–1413.
- [13] V. Kurumbaparambil, S. Rajanayagam, and S. Twieg, “Vision-based person-following algorithm for assistive elderly-care quadruped robots,” [Sensors](#), vol. 26, no. 10, art. 3263, May 2026.
- [14] A. J. Becoy, K. Khomenko, L. Peternel, and R. T. Rajan, “Autonomous navigation of quadrupeds using coverage path planning with morphological skeleton maps,” [Frontiers in Robotics and AI](#), vol. 12, art. 1601862, Jul. 2025.